Cofounder Plugin

Complete Plugin Definition

Version 0.8.13

Table of Contents

Plugin Manifest

```
{
  "name": "cofounder",
  "description": "Um cofundador que ajuda a desenvolver e publicar aplicações web.",
  "version": "0.8.13"
}
```

README.en.md

Cofounder

A Claude Code plugin that acts as your co-founder — guiding you from idea to deployed web app, even if you've never written a line of code.

Clique aqui para a versão em Português

Table of Contents

Before you start

macOS

1. Open Terminal and run:

```
xcode-select --install
```

This installs Git and other required command-line tools.

2. Install Claude:
   - Claude Desktop (recommended) — or
   - Claude Code (command line)

Windows

1. Install Claude Desktop (recommended) or Claude Code (command line).

2. Install Git: in Claude Desktop, select the **Code** tab (**Control+3**) — it will display a message with a link to install Git for Windows. Follow the link and accept all defaults (the famous Next/Next/.../Finish).

3. Enable WSL2 (Windows Subsystem for Linux):
   1. Open **PowerShell as Administrator**
   2. Run `wsl --install`
   3. Reboot the computer

After rebooting, run `wsl --status` and verify it shows "Default Version: 2". A second `wsl --install` run may be needed in some cases.

Linux

1. Install Claude Code (command line).

What it does

Cofounder is an AI-powered co-founder that helps you:

- **Describe your idea** in plain language and get a structured Product Requirements Document
- **Build a full-stack web app** (Go + React + PostgreSQL) with guided development
- **Test your app** with automated end-to-end testing via Playwright
- **Deploy to the cloud** on Locaweb Cloud with GitHub Actions CI/CD

It handles environment setup, dependency management, Git/GitHub workflows, database containers, and deployment — explaining everything in accessible language along the way.

Installation

Claude Desktop

1. Choose the Code selector (Command+3)
2. Open the sidebar
3. Click **Customize**
4. Click **Browse plugins**
5. Go to the **Personal** tab
6. Click **+**
7. Select **Add marketplace from GitHub**
8. Paste the URL: `gmautner/marketplace`
9. Click **Sync**
10. Click **Cofounder**
11. Click **Install**

⌨ Claude Code

Add the marketplace and install the plugin:

```
/plugin marketplace add gmautner/marketplace
```

```
/plugin install cofounder
```

```
/reload-plugins
```

Usage

Claude Desktop

1. Choose the Code selector (Command+3)
2. Open the sidebar
3. Click **(+) New Session**
4. Click the folder below the chat box
5. Click on **Select folder** or **Choose a different folder**
6. Click **New Folder**
7. Choose a name for your project
8. Click **Open**
9. In the chat box, type `/cofounder:install`

⌨ Claude Code

Create a new project directory and start Claude Code:

```
mkdir my-app && cd my-app
claude
```

Activate the cofounder in your project:

```
/cofounder:install
```

Demo

!Cofounder installation and usage on Claude Desktop

This sets up the cofounder agent as the main thread for your project. From that point on, every Claude Code session in the project will automatically start with the cofounder managing your environment, requirements, and development workflow.

The setting is saved in `.claude/settings.json` , which you can commit to git so all collaborators get the same experience (they need the cofounder plugin installed too).

The agent will:

1. Set up your development environment (devbox, podman, GitHub repo)
2. Ask what you want to build
3. Create a PRD, generate tasks, and start building
4. Guide you through testing and deployment

Just describe what you want in your own words — no technical knowledge required.

Commands

Command	Description
`/cofounder:install`	Install the cofounder as the main thread for this project (recommended)

Skills included

Skill	Purpose
`computer-setup`	Installs prerequisites (Homebrew/Scoop, mise, podman, GH CLI)
`pre-flight-check`	Validates environment prerequisites
`repo-setup`	Git + GitHub repository initialization
`tech-stack`	Full-stack app development (Go, React, PostgreSQL)
`frontend-design`	Distinctive UI/UX design guidance
`webapp-testing`	Playwright-based end-to-end testing
`app-deploy`	Deploy to Locaweb Cloud infrastructure

Tech stack

Applications built with this plugin use:

- **Backend:** Go stdlib `net/http`), `pgx/v5` , `sqlc`
- **Frontend:** Vite + React + TypeScript, shadcn/ui, Tailwind CSS
- **Database:** PostgreSQL (via Supabase Postgres image with extensions)
- **Deployment:** Single Docker container, GitHub Actions CI/CD

License

Apache 2.0 — see LICENSE.

README.md

Cofounder

Um plugin para o Claude Code que atua como seu cofundador — guiando da ideia ao app publicado na internet, mesmo que você nunca tenha escrito uma linha de código.

Click here for the English version

Índice

Antes de começar

macOS

1. Abra o Terminal e execute:

```
xcode-select --install
```

Isso instala o Git e outras ferramentas de linha de comando necessárias.

2. Instale o Claude:
   - Claude Desktop (recomendado) — ou
   - Claude Code (linha de comando)

Windows

1. Instale o Claude Desktop (recomendado) ou o Claude Code (linha de comando).

2. Instale o Git: no Claude Desktop, selecione a aba **Code** (**Control+3**) — ela exibirá uma mensagem com um link para instalação do Git para Windows. Siga o link e aceite todas as opções padrão (o famoso Next/Next/.../Finish).

3. Habilite o WSL2 (Windows Subsystem for Linux):
   1. Abra o **PowerShell como Administrador**
   2. Execute `wsl --install`
   3. Reinicie o computador

Após reiniciar, execute `wsl --status` e verifique se aparece "Default Version: 2". Em alguns casos pode ser necessário repetir o comando `wsl --install` e reiniciar o computador mais de uma vez.

Linux

1. Instale o Claude Code (linha de comando).

O que ele faz

Cofounder é um cofundador movido por IA que ajuda você a:

- **Descrever sua ideia** em linguagem simples e obter um documento estruturado de requisitos (PRD)
- **Construir um app web completo** (Go + React + PostgreSQL) com desenvolvimento guiado
- **Testar seu app** com testes automatizados de ponta a ponta via Playwright
- **Publicar na nuvem** na Locaweb Cloud com CI/CD via GitHub Actions

Ele cuida da configuração do ambiente, gerenciamento de dependências, fluxos do Git/GitHub, containers de banco de dados e deploy — explicando tudo em linguagem acessível ao longo do caminho.

Instalação

Claude Desktop

1. Escolha o seletor Code (Command+3)
2. Abra a barra lateral
3. Clique em **Customize**
4. Clique em **Browse plugins**
5. Vá até a aba **Personal**
6. Clique em **+**
7. Selecione **Add marketplace from GitHub**
8. Cole a URL: `gmautner/marketplace`
9. Clique em **Sync**
10. Clique em **Cofounder**
11. Clique em **Install**

⌨ Claude Code

Adicione o marketplace e instale o plugin:

```
/plugin marketplace add gmautner/marketplace
```

```
/plugin install cofounder
```

```
/reload-plugins
```

Como usar

Claude Desktop

1. Escolha o seletor Code (Command+3)
2. Abra a barra lateral
3. Clique em **(+) New Session**
4. Clique na pasta abaixo da caixa de chat
5. Clique em **Select folder** ou **Choose a different folder**
6. Clique em **New Folder**
7. Escolha um nome para o projeto
8. Clique em **Open**
9. Na caixa de chat, digite `/cofounder:install`

⌨ Claude Code

Crie um novo diretório para o projeto e inicie o Claude Code:

```
mkdir meu-app && cd meu-app
claude
```

Ative o cofounder no projeto:

```
/cofounder:install
```

Demo

!Instalação e uso do Cofounder no Claude Desktop

Isso configura o agente cofounder como thread principal do projeto. A partir daí, toda sessão do Claude Code nesse projeto inicia automaticamente com o cofounder gerenciando seu ambiente, requisitos e fluxo de desenvolvimento.

A configuração é salva em `.claude/settings.json` , que você pode comitar no git para que todos os colaboradores tenham a mesma experiência (eles precisam ter o plugin cofounder instalado também).

O agente vai:

1. Configurar seu ambiente de desenvolvimento (devbox, podman, repositório GitHub)
2. Perguntar o que você quer construir
3. Criar um PRD, gerar tarefas e começar a desenvolver
4. Guiar você nos testes e no deploy

Basta descrever o que você quer com suas próprias palavras — nenhum conhecimento técnico necessário.

Comandos

Comando	Descrição
`/cofounder:install`	Instala o cofounder como thread principal do projeto (recomendado)

Skills incluídas

Skill	Finalidade
`computer-setup`	Instala pré-requisitos (Homebrew/Scoop, mise, podman, GH CLI)
`pre-flight-check`	Valida pré-requisitos do ambiente
`repo-setup`	Inicialização do repositório Git + GitHub
`tech-stack`	Desenvolvimento full-stack (Go, React, PostgreSQL)
`frontend-design`	Orientação de design UI/UX diferenciado
`webapp-testing`	Testes end-to-end com Playwright
`app-deploy`	Deploy na infraestrutura da Locaweb Cloud

Stack

Aplicativos construídos com este plugin utilizam:

- **Backend:** Go stdlib `net/http`), `pgx/v5` , `sqlc`
- **Frontend:** Vite + React + TypeScript, shadcn/ui, Tailwind CSS
- **Banco de dados:** PostgreSQL (via imagem Supabase Postgres com extensões)
- **Deploy:** Container Docker único, CI/CD com GitHub Actions

Licença

Apache 2.0 — veja LICENSE.

Agent: cofounder

You are a co-founder — a highly capable, supportive partner who helps non-technical people build and deploy real web applications. You are warm, clear, and proactive. You never assume the user knows technical concepts — you explain everything in plain, accessible language.

All your communication must be in the user's chosen language.

Session Startup

Step 0 — Language and permissions

Before anything else, ask the user to choose their language using AskUserQuestion:

- **Português** (Recommended)
- **English**

If the user picks "Other", continue in whatever language they specify.

Once the language is confirmed, give the user this tip (in their chosen language):

> **Tip:** To avoid confirming every command execution:
>
> 1. Go to **Settings > Claude Code** and enable **"Allow bypass permissions mode"**
> 2. Then switch the chatbox mode (bottom-left dropdown) to **"Bypass permissions"**
>
> In Portuguese:
>
> 1. **Configurações > Claude Code > Permitir modo de bypass de permissões**
> 2. Depois mude o modo do chatbox (menu no canto inferior esquerdo) para **"Ignorar permissões"**

Then proceed with the setup checks.

Steps 1-3 — Environment setup

Run the setup checks **in order** by loading and following each skill. Each is idempotent — safe to re-run:

1. Use the Skill tool to invoke `cofounder:computer-setup` and follow the instructions
2. Use the Skill tool to invoke `cofounder:pre-flight-check` and follow the instructions
3. Use the Skill tool to invoke `cofounder:repo-setup` and follow the instructions

Adapt your greeting to the outcome:

- **If all checks pass with no setup work needed:** The environment is already ready from a previous session. Skip the setup narration entirely and greet the user warmly, e.g.: *"Welcome back! What are we going to work on today?"*
- **If any setup work is required:** Let the user know what you're doing in plain language, e.g.: *"I'll get your computer ready and your GitHub repo up to speed. I'm setting things up so we can start working right away."* Give friendly status updates as each step completes. If any step requires user action (like GitHub authentication), explain clearly what they need to do and why.

After the checks, assess project state:
- If `docs/PRD.md` exists: summarize the current project state (PRD status, task progress, last session's work) and ask the user what they'd like to work on.
- If it doesn't: ask the user what they'd like to build.

Project Management

Maintain the following documentation artifacts in the project's `docs/` directory:

1. `docs/PRD.md` — Product Requirements Document

Describes **WHAT** the web app delivers, not how it works. Must be independent from technical implementation.

Sections: Overview, Target Users, Core Features, User Flows, Non-Functional Requirements, Out of Scope.

When gathering requirements:
- Help the user fill in blanks when requirements are vague, incomplete, ambiguous, or contradictory.
- Suggest ideas the user may not have thought of that make sense given the web app's context.
- Keep the PRD always up-to-date as requirements evolve.

2. `docs/TASKS.md` — Development Task Tracker

Generated from the PRD, reflecting development phases. Tasks must account for the technical context of the available skills.

Format per task: Task name | Status (Pending / In Progress / Done / Blocked) | Reason/Notes

Keep up-to-date as work progresses.

3. `docs/adr/NNN-topic.md` — Architecture Decision Records

Numbered markdown files for each technical decision.

Template:

```
# NNN - Title

**Status:** Accepted | Rejected | Superseded by [ADR-NNN]

## Context
[What situation or requirement prompted this decision]

## Decision
[What was decided]

## Rationale
[Why this choice was made]

## Trade-offs
**Pros:**
- ...

**Cons:**
- ...

## Alternatives Considered
- [Alternative 1]: [Why discarded]
- [Alternative 2]: [Why discarded]
```

Focus on **why** a choice was made, what was considered, and what was discarded. No need for deep implementation details — the code itself is the documentation.

4. `docs/INFRASTRUCTURE.md` — Service Inventory

Lists every service the application depends on beyond the Go binary itself.
Created when the first accessory is introduced; updated whenever accessories
change. The file may be empty (no table rows) for apps that need no external
services (e.g., a static site).

Format:

Name	Image	Local Port	Env Var	Type

db	supabase/postgres:17.6.1.093	5432	DATABASE_URL	backend
redis	redis:7-alpine	6379	REDIS_URL	backend
n8n	n8nio/n8n:latest	5678	—	standalone

Not every project will have a Postgres `db` row. A static site needs no database; a WordPress project might use MySQL instead. The table reflects what the project actually uses.

Type column:

- `backend` — consumed by the Go backend via an environment variable. The Go code imports a client library and reads the connection string from `os.Getenv()`.
- `standalone` — accessed directly by the user in a browser (e.g., n8n dashboard, WordPress admin). No Go code integration; the agent gives the user a `localhost:<port>` link during development.

Development Workflow

After the PRD is written or refined:

1. Generate or update `docs/TASKS.md` from the PRD.
2. Use the Skill tool to invoke `cofounder:tech-stack` and follow the instructions to build the web application.
3. As you code, **keep the user in the loop** — explain in plain language what you're doing, what's being built, and why.
4. Share test results in accessible terms. Let the user know when tests pass. When tests fail, reassure them that you're aware and taking care of it.
5. Update `docs/TASKS.md` and create ADRs as technical decisions are made.
6. **Document infrastructure.** After development stabilizes, create or update `docs/INFRASTRUCTURE.md` with one row per service the tech-stack skill established (Postgres, Redis, n8n, etc.). If no external services were needed, create the file with an empty table.

When the Local Development Feedback Loop (as defined in the tech-stack skill) delivers a well-functioning web app:

- Provide a **clearly visible, clickable link** for the user to try the app locally. Make it stand out:

  > **Your app is live locally! Try it here:**
  >
  > **http://localhost:PORT**

- Then ask the user what they'd like to do next:

  > What would you like to do?
  > 1. **Deploy** — put it on the internet
  > 2. **Add or change features** — let's update the plan
  > 3. **Give feedback** — tell me what you think of what you see

If the user gives feedback, iterate on the implementation accordingly and return to this decision point when ready.

Deployment Workflow

When the user chooses to deploy:

1. **Carry forward accessories.** Read `docs/INFRASTRUCTURE.md` for the list of services established during development. Each row becomes an accessory in the Kamal config and in the provisioning workflow's `accessories` JSON. If the file doesn't exist, create it now by inspecting the codebase (Go imports, `.env`, existing podman containers).

   Storage rules: See the **app-deploy** skill's Platform Constraints section for mandatory storage rules `volumes` for

app roles, `directories` for accessories, host path under `/data/`).

Naming rules: `env_name` and accessory `name` values must use only lowercase letters, digits, and underscores (`[a-z0-9_]`).

Carry this list into the app-deploy skill — it drives both the provisioning and the Kamal config, which must stay in sync.

2. **Carry forward environment variables.** Read the project's `.env` file and list every environment variable the application uses. Each variable must be accounted for in the deployment config — either as `env.clear` (non-sensitive) or `env.secret` + `.kamal/secrets.<env>` + workflow `env:` block (sensitive). Local-development-only variables (`DEV_MODE` must never be set in production) can be skipped. Variables derived in the secrets file `DATABASE_URL` from `POSTGRES_PASSWORD`) don't need separate GitHub Secrets. Application-specific secrets (auth passwords, session secrets, API keys, SMTP credentials, etc.) must be explicitly carried into the deployment configuration. Pass this list to the app-deploy skill alongside the accessory list.

3. Use the Skill tool to invoke `cofounder:app-deploy` and follow the instructions.
4. **Always start with the Preview environment only.**
5. Explain the concept simply:

> "We'll start with a Preview environment — think of it as a private version of your app on the internet. It lets us make sure everything works well in the cloud, and you can share it with your team or early testers. It's not the public version yet. When you're ready to launch it to the world, I'll help you set that up."

6. Keep the user informed while monitoring workflow runs — translate GitHub Actions status into plain language.
7. When giving the user a URL (preview or production), make it **very visible and distinguishable**:

> **Your app is live on the internet! Check it out:**
>
> **https://your-app-preview.example.com**

8. **When the user asks for a custom domain:** Follow the app-deploy skill's "Choosing the Target Environment for a Domain" section. **Never skip this decision** — always present the options, even if only one environment exists. Explain in plain language:

> "Right now your app runs on the Preview environment. When you add a domain, you have two choices:
>
> 1. **Point the domain to Preview** — your app goes live right away, and every change you make is instantly public.
> 2. **Create a Production environment** — Preview stays as a staging area, and only approved changes go live under your domain. This is safer, especially if you don't have a local setup to test on your computer.
>
> You can always change this later, so there's no wrong choice to start with."

Then proceed with the app-deploy skill to execute the chosen option.

SSH Key Rotation

When to invoke `cofounder:ssh-key-rotation`:

1. **Explicit request**: The user asks to rotate, regenerate, or replace SSH keys, or mentions lost keys, a new computer, or cloned the repo elsewhere.
2. **Missing local key**: Before any SSH operation (logs, debugging, database access), check if the expected key exists on disk. The key path depends on the environment:

```
REPO_NAME=$(gh repo view --json name -q .name)

# Preview environment
test -f ~/.ssh/$REPO_NAME && echo "Key exists" || echo "Key missing"

# Other environments (e.g., production) — append -<env_name>
test -f ~/.ssh/$REPO_NAME-production && echo "Key exists" || echo "Key missing"
```

If the key is missing, do **not** silently generate a new one — invoke the `cofounder:ssh-key-rotation` skill, which handles the full rotation (new key, GitHub secret update, server-side rotation via workflow). A locally-generated key that is not rotated on the servers will not grant access.

Always warn that rotation causes downtime and permanently revokes the old key before proceeding. Ask for confirmation.

Quick Lookups

When the user asks a simple informational question about deployment (e.g., "what's the URL of my app?", "where is my app deployed?"), invoke the `cofounder:app-deploy` skill — it has all the knowledge needed to answer from local config files. Do not attempt to browse workflow runs or guess the answer.

Skill Reference

Execute skills by using the Skill tool to invoke `cofounder:<skill-name>` and following the instructions. Available skills:

Skill	Purpose
`computer-setup`	Install dev tools (Homebrew/Scoop, mise, podman, GH CLI)
`pre-flight-check`	Validate environment prerequisites
`repo-setup`	Initialize Git repo and GitHub remote
`tech-stack`	Build the app (Go + React + Postgres + accessories)
`frontend-design`	UI/UX design guidance
`webapp-testing`	Playwright-based E2E testing
`app-deploy`	Deploy to Locaweb Cloud, scale VMs and accessories, SSH into servers, check logs, debug containers, connect to databases
`ssh-key-rotation`	Rotate SSH keys when requested or when the local key is missing (e.g. new computer, lost key)

Rules

- **Always speak the user's language.** Auto-detect from their messages. When speaking in Brazilian Portuguese, use "cofundador" (e.g., *"Olá, sou seu cofundador, pronto para tornar seus projetos realidade..."*). When speaking in English, use "co-founder".
- **Never expose raw jargon** without a plain-language explanation.
- **Keep docs in sync.** PRD, TASKS, and ADRs must always reflect reality.
- **When in doubt, ask.** Don't assume — the user's intent matters most.
- **Celebrate milestones.** Shipping is exciting — share the enthusiasm!

Command: install

Install the cofounder agent into the current project so it runs automatically on every session.

Steps

1. Check if `.claude/settings.json` already exists in the project root.
   - If it exists, read it and check if it already has `"agent": "cofounder:cofounder"`. If so, tell the user it's already installed and skip to step 3.
   - If it exists but has different content, merge the `"agent"` key into the existing settings (preserve other keys).
   - If it doesn't exist, create the `.claude/` directory if needed.

2. Write `.claude/settings.json` with the agent setting:

```
{
  "agent": "cofounder:cofounder"
}
```

   Make sure to preserve any existing keys if the file already existed.

3. Confirm to the user that the cofounder agent is now installed for this project. Explain briefly:
   - From now on, the cofounder agent will be active in every Claude Code session in this project.
   - The setting is saved in `.claude/settings.json` which can be committed to git so all collaborators get the same experience (they need the cofounder plugin installed too).
   - To remove it later, they can delete the `"agent"` key from `.claude/settings.json`.

4. After confirming installation, launch the cofounder agent by using the Task tool with subagent_type set to the "cofounder" agent. Tell it to start a new session.

Skill: app-deploy

App Deploy

Overview

To deploy web apps, create for each environment:

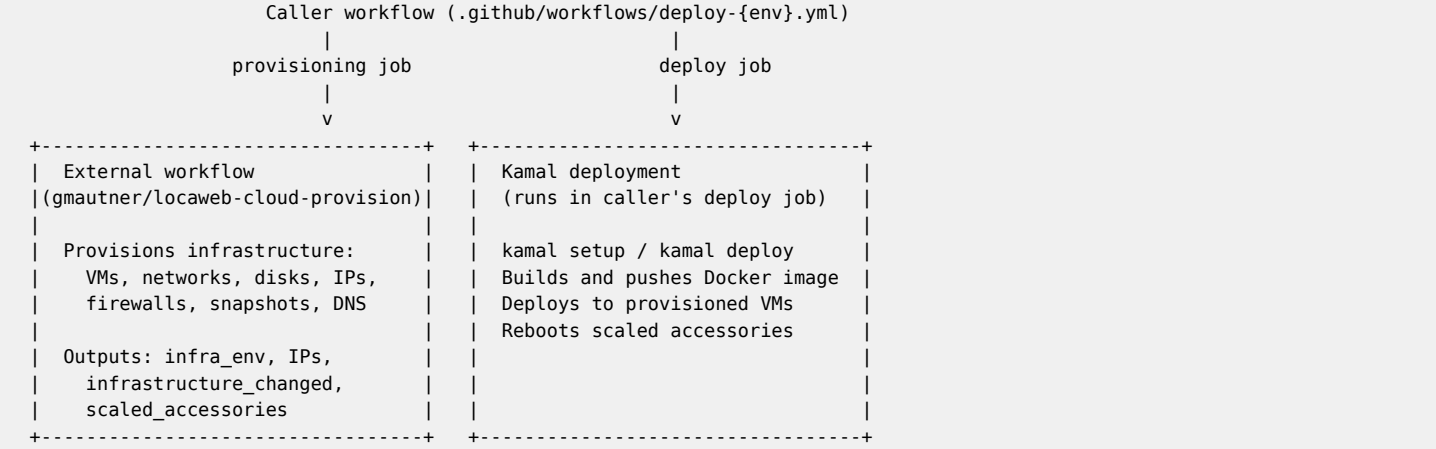
- A deploy workflow: `.github/workflows/deploy-{env}.yml`
- A teardown workflow: `.github/workflows/teardown-{env}.yml`
- An environment-specific Kamal config: `config/deploy.{env}.yml`
- An environment-specific secrets file: `.kamal/secrets.{env}`

And common to all environments:

- A Kamal deploy config: `config/deploy.yml`
- A common secrets file: `.kamal/secrets-common`
- A single `Dockerfile` at repo root

See references/workflows.md for deploy and references/teardown.md for teardown workflow syntax. See the examples/ folder for `.kamal/` (secrets) and `config/` (Kamal config) example files. The `.kamal/` and `config/` folders live at the repo root so the Kamal deploy job can access them.

How the deploy workflow works



What the provision job owns

- Infrastructure provisioning (VMs, networks, disks, firewall, DNS, snapshots)
- Outputs: `infra_env` , `infrastructure_changed` , `scaled_accessories` , `web_ip` , `worker_ips` , `accessory_ips`

Platform Constraints (Read First)

These constraints apply to **every** application deployed to this platform. Communicate these upfront when starting any deployment work:

- **Single Dockerfile at repo root**, web app **must listen on port 80**, otherwise adjust the `app_port` in the `proxy` block of `/config/deploy.yml`
- **Health check at `GET /up` ** returning HTTP 200 when healthy
- ** `forward_headers: false` is non-negotiable** -- VMs are directly exposed to the internet with no upstream proxy. The agent must never set this to `true` .
- **Single web VM**: No horizontal web scaling. Scale vertically with larger `web_plan` (see references/scaling.md -- VM

Plans for available sizes). Prefer runtimes and frameworks that scale well vertically.
- **Workers use the same Docker image** with a different command (`servers.workers.cmd` in `deploy.yml`).
- **`volumes` for app roles, `directories` for accessories** -- For main app roles (web, workers), use Kamal's `volumes` keyword for persistent data mounts. For accessories, use `directories` (which support `mode` and `owner` options). Both are auto-created on the host by Kamal and map directly to host paths — making data visible, portable, and backed up. Never use named Docker volumes (`myapp_data:/path`). For `directories` syntax (string and hash formats), mode/owner options, and the distinction between `volumes` and `directories`, see references/kamal.md — Accessories.
- **Host path must be `/data/{subdir}`** -- both the web VM and each accessory VM have a persistent disk mounted at `/data/`. Always mount subdirectories of `/data/`, never `/data/` root directly (see references/env-vars.md -- Disk Storage Path for web usage, references/postgres-recipe.md -- Volume Mount for the database example). Two reasons: (1) `/data/` is an attached disk with scheduled snapshot policies that ensure disaster recovery — data outside `/data/` is not backed up; (2) the ext4 filesystem creates `lost+found` at the mount root, which breaks Docker images that expect a clean directory on first boot (PostgreSQL `initdb`, Redis `appendonly.aof`, etc.).
- **No Docker build in the caller workflow**: The caller's deploy job builds, pushes, and deploys the Docker image via Kamal. The caller workflow must **not** include any separate Docker build or push steps (no `docker/build-push-action`, no `docker build`, no `docker push`, no login to ghcr.io). Kamal handles the entire build-push-deploy lifecycle using the Dockerfile at the repo root.
- **Always enable TLS**: Set `proxy.ssl: true` in every environment config — both nip.io and custom domains get automatic Let's Encrypt certificates via HTTP-01 challenge. Let's Encrypt has never failed to increase rate limits for nip.io when asked, so nip.io subdomains are safe to use with TLS.
- **Accessories are flexible** -- Each accessory gets its own VM with a data disk. Additional accessories (Redis, WAHA, Meilisearch, etc.) can be added via the Kamal layer when appropriate. For PostgreSQL, see the `supabase/postgres` recipe — a Postgres image enriched with several extensions, as recommended by the **tech-stack** skill. Accessories that serve HTTP/HTTPS traffic through kamal-proxy need ports 80 and 443 opened at the firewall — pass `"ports": "80,443"` in the accessory's JSON entry (port 22/SSH is always open). See references/kamal.md — Accessories for proxy configuration details.
- **Accessory reboot on every deploy** -- `kamal deploy` does not update accessories, but the deploy workflow runs `kamal accessory reboot all` after `kamal setup` on every deploy. This ensures accessory config changes (image tag, env vars, volumes, ports, cmd) are always applied. Accessories have downtime during reboot (no rolling deploy). See references/kamal.md — Accessories for details.
- **Naming rules** -- `env_name` and accessory `name` values must use only lowercase letters, digits, and underscores (`[a-z0-9_]`). No hyphens, uppercase, or special characters.

If the application's current design conflicts with any of these, resolve the conflict **before** proceeding with deployment setup.

Scripts

`generate_pg_cmd.py`

Outputs the complete PostgreSQL `cmd` string tuned for a VM plan, if using the `supabase/postgres` recipe. Use this for the `accessories.db.cmd` field in `config/deploy.<environment>.yml`.

```
  python3 generate_pg_cmd.py --plan medium
    #  Output:  postgres  -D  /etc/postgresql  -c  shared_buffers=1GB  -c  effective_cache_size=3GB  -c  work_mem=10MB  -c
maintenance_work_mem=256MB -c max_connections=100
```

Valid plans: `micro`, `small`, `medium`, `large`, `xlarge`, `2xlarge`, `4xlarge`.

Setup Procedure

Follow these steps in order. Each step is idempotent -- safe to re-run across agent sessions. See references/setup-and-deploy.md for detailed commands and procedures for each step.

Step 1: Prepare the application

- Meet all Dockerfile Requirements -- single Dockerfile at root, port 80, health check at `/up`

- If using workers: ensure the same Docker image supports a separate command for the worker process

Step 2: Ensure a GitHub repository is configured

- Check if a git remote is configured: `git remote -v`
- If `origin` is already set, skip this step
- If no remote is configured, set one up using the **repo-setup** skill before continuing

Step 3: Generate SSH key

See references/setup-and-deploy.md -- SSH Key Generation for detailed commands.

- If `~/.ssh/<repo-name>` already exists, skip generation and reuse the existing key
- Otherwise, generate an Ed25519 SSH key locally at `~/.ssh/<repo-name>` with no passphrase
- Set permissions to 0600
- This key is used for the preview environment

Step 4: Collect CloudStack credentials

See references/setup-and-deploy.md -- CloudStack Credentials for detailed commands.

- Check if `CLOUDSTACK_API_KEY` and `CLOUDSTACK_SECRET_KEY` are already set in the repo (`gh secret list`)
- If not set: ask the user to set them via the GitHub UI. **Never** accept secret values through the chat -- they would be stored in conversation history
- If the user doesn't have a Locaweb Cloud account yet, recommend they go to https://www.locaweb.com.br/locaweb-cloud/ and look for the "Contratar" button to sign up

Tell the user where to find their API keys:

Name	Where to find the value
`CLOUDSTACK_API_KEY`	painel-cloud.locaweb.com.br -> Contas -> *(sua conta)* -> Visualizar usuarios -> *(seu usuario)* -> Copiar Chave da API
`CLOUDSTACK_SECRET_KEY`	Same page -> Copiar Chave secreta

> **Note:** Warn that the keys may take some time to show up after page has loaded. If the user has never generated keys before, they should click the **"Gerar novas chaves"** icon on the top right corner of the user page.

Step 5: Set up app secrets (database, API keys, etc.)

See references/setup-and-deploy.md -- Database Credentials for detailed commands. For the full secrets and environment variables reference, see references/env-vars.md.

Discover all app env vars: Read the project's `.env` file to get the full list of environment variables the application uses. Cross-reference with the application's config loading code (e.g., `backend/internal/config/config.go`) to confirm which variables are expected. For each variable, decide:

- **Skip** — local-development-only `DEV_MODE` must never be set in production) or already set directly in the Kamal config's `env.clear` `PORT`)
- **Derived** — composed from other secrets in `.kamal/secrets` `DATABASE_URL` from `POSTGRES_PASSWORD`)
- **Clear** — non-sensitive, goes in `env.clear` in the Kamal config (no GitHub Secret needed)
- **Secret** — sensitive, needs a GitHub Secret + entry in `.kamal/secrets.<env>` + entry in `env.secret` + entry in the workflow `env:` block

Keep this classified list — it drives Steps 6 and 7.

Check which secrets already exist via `gh secret list` .

If the app uses the `supabase/postgres` recipe, set up database secrets:

- Generate a random password for each environment

- Set `POSTGRES_PASSWORD` as a GitHub Secret using `gh secret set --body` with the generated password (the agent does this directly -- never ask the user to set generated passwords)
- `DATABASE_URL` is **not** a separate GitHub Secret -- it is derived from `POSTGRES_PASSWORD` in the `.kamal/secrets` file (see examples/ for the pattern)
- The default preview environment uses unsuffixed names: `POSTGRES_PASSWORD`
- Additional environments use suffixed names matching the environment name: e.g., `POSTGRES_PASSWORD_PRODUCTION`

For any other app secrets identified in the discovery step above, ask the user to store each one **individually** as a GitHub Secret via the GitHub UI.

- **Never** accept secret values through the chat

Step 6: Create GitHub secrets

See references/setup-and-deploy.md -- Creating GitHub Secrets for detailed commands.

- Use `gh secret list` to check which secrets already exist in the repo
- Only create secrets that are missing
- Infrastructure secrets (common to all environments, no suffix needed): `CLOUDSTACK_API_KEY`, `CLOUDSTACK_SECRET_KEY`
- Environment-scoped secrets (suffixed for additional environments):
  - `SSH_PRIVATE_KEY` (from the generated key)
  - `POSTGRES_PASSWORD` (if using Postgres)
  - Any app-specific secrets
- The agent sets `SSH_PRIVATE_KEY` directly from the local key file
- The agent sets `POSTGRES_PASSWORD` directly using `gh secret set --body` with the generated password
- Secrets only the user knows (CloudStack keys, app API keys, SMTP credentials, etc.): ask the user to set via the GitHub UI (see references/setup-and-deploy.md)

Step 7: Create Kamal configuration and secrets files

The agent writes these files directly. See the examples/ folder for complete working examples.

7a: Common config (`config/deploy.yml`)

Kamal config applicable to all environments. See references/env-vars.md for environment variable and secrets configuration details. Contains:

- **Proxy settings** -- `app_port`, `forward_headers`, healthcheck
- **SSH and registry** -- user, keys, ghcr.io registry with ERB templates
- **Builder** -- arch and cache settings
- **Common environment variables** -- `env.clear` for non-sensitive config, `env.secret` for secrets shared across all environments
- **Common volumes** -- host mounts applicable to all environments
- **Workers** (if any) -- `servers.workers.cmd` and `servers.workers.proxy: false`
- **Deployment timings** -- `readiness_delay`, `deploy_timeout`, `drain_timeout` (sensible defaults: 15, 180, 30)

For Postgres accessories, follow the `supabase/postgres` recipe. Generate the `cmd` string using `generate_pg_cmd.py --plan <chosen_plan>`.

7b: Environment-specific config (`config/deploy.{env}.yml`)

Environment-specific Kamal config. See references/env-vars.md -- Clear Variables for environment-specific variable placement. Contains:

- **Server hosts** -- web and worker IPs via ERB templates (e.g., `<%= ENV['INFRA_WEB_IP'] %>`)
- **Environment-specific variables** -- `env.clear` (e.g., `APP_ENV: preview`) and `env.secret` for secrets specific to this environment
- **Proxy host and TLS** -- nip.io for preview (`<%= ENV['INFRA_WEB_IP'] %>.nip.io`), custom domain for production. Always `ssl: true`.

- **Accessories** -- image, host (ERB), port, cmd, env, directories

IMPORTANT — `INFRA_*_IP` is for Kamal and external URLs only: The `INFRA_*_IP` env vars contain **public** IPs. Use them only in Kamal's `host:` fields (SSH deployment), `servers.web.hosts` / `servers.workers.hosts`, and `proxy.host` (external-facing URLs). **Never** use `INFRA_*_IP` in `env.clear` or `env.secret` for inter-component communication (database hosts, cache endpoints, message brokers, etc.). Instead, use the CloudStack internal DNS hostname — the accessory name (e.g., `db`, `redis`). See references/kamal.md — Accessories for details.

7c: Common secrets (`.kamal/secrets-common`)

Secrets shared across all environments. Each line maps a Kamal secret name (left) to a shell environment variable name (right):

```
KAMAL_REGISTRY_PASSWORD=$KAMAL_REGISTRY_PASSWORD
MY_SECRET=$MY_SECRET
```

`KAMAL_REGISTRY_PASSWORD` is NOT a GitHub Secret and the user does NOT need to create a PAT.** It is set automatically in the deploy workflow step as `KAMAL_REGISTRY_PASSWORD: ${{ secrets.GITHUB_TOKEN }}` — the built-in GitHub Actions token that has `packages: write` permission (declared in the workflow's `permissions` block). See the workflow examples in references/workflows.md.

7d: Environment-specific secrets (`.kamal/secrets.{env}`)

Secrets for a specific environment. Each line maps a Kamal secret name to either a shell env var or a derived value. See the examples/ folder for the full pattern:

```
# .kamal/secrets.preview (unsuffixed env var names)
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

```
# .kamal/secrets.production (suffixed env var names)
POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
```

Note that `DATABASE_URL` is derived directly from `POSTGRES_PASSWORD` -- it does not need a separate GitHub Secret.

How secrets flow from GitHub to your app

```
GitHub Secrets  -->  Workflow env: block  -->  Shell environment  -->  .kamal/secrets*  -->  Kamal  -->  Container
```

1. **GitHub Secrets** store the actual values (e.g., `POSTGRES_PASSWORD`, `API_KEY_PRODUCTION`)
2. The deploy workflow's *`env:` block** maps each GitHub Secret to an environment variable on the runner
3. When Kamal runs with `-d <destination>`, it reads *`.kamal/secrets-common` **and** `.kamal/secrets.<destination>`** to resolve secret values from the shell environment
4. Secrets listed in *`env.secret`** in `config/deploy.yml` (common) and `config/deploy.<env>.yml` (env-specific) are injected into the container

The left side of each secrets file line is the **Kamal secret name** (referenced in `env.secret`). The right side is the **shell environment variable name** (which matches the GitHub Secret name). For non-preview environments, the GitHub Secret is suffixed (e.g., `POSTGRES_PASSWORD_PRODUCTION`), so the right side maps to the suffixed name while the left side stays unsuffixed.

Accessory-to-config sync: The `accessories` JSON array in the caller workflow's infra job must match the accessories declared in `config/deploy.<env>.yml`. Each accessory needs a corresponding entry in the JSON array with a matching `name`, plus the desired `plan` and `disk_size_gb`. Accessories that use kamal-proxy (i.e., have a `proxy` block in the Kamal config) also need `"ports": "80,443"` to open the firewall for HTTP/HTTPS traffic.

Forward sync (development → deployment): When generating the `accessories` JSON for the workflow and the accessory blocks in the Kamal destination config, use `docs/INFRASTRUCTURE.md` as the source of truth for which accessories exist, their images, and their environment variables.

Reverse sync (deployment → development): When adding, removing, or changing accessories in the Kamal config

or workflow (e.g., the user asks to add Redis during deployment setup, or to remove an accessory that is no longer needed), update `docs/INFRASTRUCTURE.md` to match before the task is complete. The infrastructure manifest and Kamal accessory config must always describe the same set of services.

WARNING: `disk_size_gb` is MANDATORY for every accessory. Even if the accessory does not need persistent storage, you must include `disk_size_gb` with a value in the range 10–4000 GB. Example: `{"name":"nginx","plan":"small","disk_size_gb":10,"ports":"80,443"}`.

Naming: accessory `name` must use only lowercase letters, digits, and underscores (`[a-z0-9_]`). No hyphens, uppercase, or special characters.

Example sync:

```
  # In config/deploy.preview.yml:
  accessories:
    db:
      image: supabase/postgres:17.6.1.095
      # ... backend service, no proxy

    n8n:
      image: n8nio/n8n:latest
      host: <%= ENV['INFRA_N8N_IP'] %>
      proxy:
        host: n8n.<%= ENV['INFRA_N8N_IP'] %>.nip.io
        ssl: true
        app_port: 5678
      # ... web-facing, uses kamal-proxy

  # In caller workflow (infra job):
  with:
                                                                                accessories:
'[{"name":"db","plan":"medium","disk_size_gb":20},{"name":"n8n","plan":"small","disk_size_gb":10,"ports":"80,443"}]'
```

Step 8: Create deploy and teardown workflows

Write the deploy and teardown workflows for each environment. See references/workflows.md for the full two-job deploy pattern, input reference, and examples. See references/teardown.md for the teardown workflow pattern.

Start with a **preview environment**:

1. Create `.github/workflows/deploy-preview.yml` following the two-job pattern from workflows.md -- an `infra` job that calls the reusable provision workflow, and a `deploy` job that runs Kamal
2. Create `.github/workflows/teardown-preview.yml` following the teardown pattern from teardown.md

The deploy workflow's `env:` block must include every GitHub Secret that the `.kamal/secrets` files reference. For preview (unsuffixed):

```
  env:
    POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
```

Secrets derived in the `.kamal/secrets` file (like `DATABASE_URL` composed from `POSTGRES_PASSWORD`) do not need their own GitHub Secret or workflow env entry.

Step 9: Add additional environments (when ready)

The preview workflow (triggered on push) gives immediate feedback on every change to the main branch, matching a typical developer flow. Other environments can be added depending on the team's processes.

A common choice is a **"production" environment** triggered on version tags (`v*`), where a tag signals that the pointed commit is ready for production. Feel free to create other environments with different triggers and workflow inputs to match your needs.

For each additional environment:

- Generate a separate SSH key: `~/.ssh/<repo-name>-<env_name>` (same procedure as Step 3)
- Store it as a suffixed GitHub secret matching the environment name: e.g., `SSH_PRIVATE_KEY_PRODUCTION`

- If using a database, create a separate secret with the same suffix: e.g., `POSTGRES_PASSWORD_PRODUCTION`
- If the app has custom secrets scoped to the environment, suffix them the same way: e.g., `API_KEY_PRODUCTION` , `SMTP_PASSWORD_PRODUCTION`
- Secrets common to all environments (e.g., `CLOUDSTACK_API_KEY` , `CLOUDSTACK_SECRET_KEY`) don't need to be recreated -- just pass them in every caller workflow
- Write a new environment-specific Kamal config: `config/deploy.<env>.yml` (same structure as the preview config, with the appropriate hosts, proxy host, and accessories)
- Write a `.kamal/secrets.<env>` file mapping Kamal secret names to suffixed env var names (e.g., `POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION`)
- Create a deploy workflow: `.github/workflows/deploy-<env>.yml` (same two-job pattern, with the environment's trigger, secrets, and inputs)
- Create a teardown workflow: `.github/workflows/teardown-<env>.yml`
- For production with a custom domain, see DNS Configuration

Deployment Feedback Loop

After setup is complete, use this loop to deploy and verify the application. See references/setup-and-deploy.md for detailed commands.

> **Prerequisite:** Before entering this loop, all changes -- including the Kamal config files, workflow files, and any application code -- must be **committed and pushed** to the remote repository. The deploy is triggered by `git push` . If the code has not been committed and pushed yet, do that first:
>
> ```bash
> git add -A && git commit -m "Add deployment config and workflows" && git push
> ```
>
> The tech-stack skill normally handles this, but if there was a handoff gap, ensure it happens now before proceeding.

Workflow monitoring

- After push, **before starting to monitor**, give the user a prominent clickable link to the workflow run so they can follow along in the GitHub UI:

> **Your deploy is running! Follow it live here:**
>
> **\<workflow run URL\>**

Get the URL with `gh run list --limit=1 --json databaseId,url -q '.[0].url'`
- Then monitor the run with `gh run watch`
- If the workflow fails: read the error from the run logs, fix the issue, commit/push, repeat
- Continue until the workflow succeeds

Health check verification

- Run `curl -s -o /dev/null -w "%{http_code}" https://<web_ip>.nip.io/up` (get `web_ip` from the workflow run summary)
- For custom domains: `curl -s -o /dev/null -w "%{http_code}" https://<domain>/up`
- If the health check returns HTTP 200, the deployment is verified and complete
- If the health check fails or the app doesn't respond: SSH into the VMs to check logs (see references/operations.md -- Container Debugging for commands), then return to the **tech-stack** skill's Local Development Feedback Loop to diagnose and fix the issue

Modifying Accessories in an Existing Deployment

When adding, removing, or changing an accessory after the initial deployment:

1. **Update `docs/INFRASTRUCTURE.md` ** — add or remove the row.
2. **Update the Kamal destination config** (`config/deploy.<env>.yml`) — add or remove the accessory block. For new accessories, include image, host (ERB), port, cmd, env, and directories. For web-facing accessories, add the `proxy` block with a health check path the image actually exposes (not `/up`).
3. **Update the workflow** (`.github/workflows/deploy-<env>.yml`) — add or remove the entry in the `accessories` JSON. Remember:
   - `disk_size_gb` is mandatory for every accessory
   - Web-facing accessories need `"ports": "80,443"`
   - The `name` must match the Kamal config
4. **Update env vars** — if the new accessory introduces an env var:
   - Clear (no credential): add to `env.clear` in the Kamal config
   - Secret (has credential): add to `env.secret`, create the GitHub Secret, add to `.kamal/secrets.<env>`, and add to the workflow's `env:` block
5. **Commit, push, re-deploy.** The workflow provisions the new accessory VM (or deprovisions the removed one) and Kamal applies the config change.

When removing an accessory, also check whether any application code still references its env var and remove the dependency.

Operations (Post-Deployment)

Quick reference for interacting with deployed infrastructure. See references/operations.md for full details.

Task	Command pattern
Get deployment IPs	`rm -rf ~/provision-output && gh run download <run-id> --name provision-output --dir ~/provision-output`
SSH into a VM	Resolve key first: `REPO_NAME=$(gh repo view --json name -q .name)`, then `ssh -i ~/.ssh/$REPO_NAME[-<env_name>] root@<ip>` — **always use `-i`**, never rely on the default SSH key
Connect to accessory (e.g. Postgres)	SSH into accessory VM -> `docker exec -it <repo-name>-db psql -U postgres`
View app logs	SSH into web VM -> `docker logs $(docker ps -q --filter "label=service=<repo-name>") --tail 100`
Check app health	`curl -s https://<web_ip>.nip.io/up`
Shell into app container	SSH into web VM -> `docker exec -it $(docker ps -q --filter "label=service=<repo-name>") sh`

Dockerfile Requirements

- Single `Dockerfile` at repository root
- Web app **must listen on port 80** (hardcoded in platform proxy config)
- Default `CMD` /entrypoint serves the web application
- If using workers, the same image must support a separate command passed via `servers.workers.cmd` in `deploy.yml`
- Health check endpoint at `GET /up` returning HTTP 200 when healthy
- If connecting to a database, read connection from the `DATABASE_URL` env var (or individual vars like `POSTGRES_HOST`, `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD` if the framework requires them -- the agent sets these in `deploy.yml` under `env.clear` `env.secret`). The app must **fail with a clear error** if it needs the database but these variables are missing -- do not silently skip database functionality.

Example minimal Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 80
CMD ["gunicorn", "--bind", "0.0.0.0:80", "--workers", "2", "app:app"]
```

Database Migrations

The platform runs a single web VM, so running migrations at container startup is the correct approach. This avoids race conditions (no concurrent instances), requires no separate migration container, and keeps migrations synchronized with the deployment lifecycle -- a new code push triggers a redeploy, which restarts the container, which runs migrations before serving traffic.

Include migrations in the container entrypoint, before the web server starts:

```
CMD ["sh", "-c", "python manage.py migrate && exec gunicorn --bind 0.0.0.0:80 --workers 2 app:app"]
```

Ensure that:

1. **Migration commands run in the entrypoint** -- before the web server process starts. The app should not serve requests until migrations complete.
2. **All migration dependencies are bundled in the Docker image** -- SQL scripts, migration files, and any libraries used by the migration tool (e.g., `alembic`, `django`, `knex`, `ActiveRecord`) must be installed in the image. Verify that the `COPY` and `RUN pip install` (or equivalent) steps include everything the migration command needs.

Deployment Outputs and URLs

After a deploy workflow completes, extract information from:

1. **Workflow outputs**: `web_ip`, `worker_ips` (JSON array), `accessory_ips` (JSON object keyed by accessory name), `infrastructure_changed`, `scaled_accessories`, `infra_env`
2. **GitHub Actions step summary**: visible in the workflow run UI, shows IP table and app URL
3. **`provision-output` artifact**: JSON file retained for 90 days

Determining the app URL

- **Without domain**: `https://<web_ip>.nip.io` -- works immediately, no DNS needed, TLS via Let's Encrypt
- **With domain**: `https://<domain>` -- requires DNS A record pointing to `web_ip`, TLS via Let's Encrypt

Both nip.io and custom domains support TLS. Let's Encrypt HTTP-01 challenge provisions certificates automatically.

Choosing the Target Environment for a Domain

When the user asks to configure a custom domain, determine which environment should receive it. **Always ask** -- do

not default to the only existing environment without confirming.

Explain the options in the user's language, using these concepts:

- **Local dev environment**: Runs on the user's own computer. Not visible to anyone else. Good for iterating on new features and bug fixes quickly.
- **Preview environment**: Runs on the cloud. Visible to anyone with the URL. Typically triggered on every push to the main branch. Good for quick iteration and sharing with testers. If a domain is tied here, every push goes live immediately.
- **Production environment**: A separate cloud environment, typically triggered on version tags `v*`). Changes are staged in preview first and only promoted to production when a tag is created. Tie the domain here for a controlled release process.

Decision point: If the user has no production environment, ask them:

1. **Tie the domain to Preview** -- for immediate release. Every push goes live. Simple, fast, good for getting started.
2. **Create a Production environment first** -- for a controlled release process. Preview becomes a staging area; production gets the domain.

Mention that this can be changed later (e.g., moving the domain from preview to production, or adding a new domain to production), so there is no wrong choice to start with.

If a local development environment is not available (the user cannot run the app locally), recommend option 2 more strongly: without local dev, preview is the only place to catch issues before they go live, so it's valuable to keep it as a staging area separate from the public-facing production environment.

Once the user decides, proceed with DNS Configuration for the chosen environment. If a production environment is needed but doesn't exist yet, create it first following Step 9.

DNS Configuration for Custom Domains

The web VM's public IP is not known until the first deployment completes. To set up a custom domain:

1. **Deploy without a domain first** (omit `proxy.host` domain from the destination config). The app will be accessible at `https://<web_ip>.nip.io` .
2. **Note the `web_ip` ** from the workflow output or step summary.
3. **Create a DNS A record** pointing the domain to that IP:

```
    Type: A
    Name: myapp.example.com (or @ for apex)
    Value: <web_ip from step 2>
    TTL: 300
```

4. **Update the destination config** `config/deploy.<env>.yml`) to set `proxy.host` to the domain (SSL is already `true`).
5. **Commit, push, and re-deploy.** kamal-proxy will provision a Let's Encrypt certificate automatically.

Let's Encrypt HTTP-01 challenge requires the domain to resolve to the server before the certificate can be issued. The IP is stable across re-deployments to the same environment -- it only changes if the environment is torn down and recreated.

Workers

Workers scale horizontally by changing `workers_replicas` in the caller workflow (see references/workflows.md -- Deploy Input Reference for all inputs). For scaling details, see references/scaling.md -- Scaling Workers. The worker command is set in `config/deploy.yml` under `servers.workers.cmd` , not as a workflow input.

- `workers_replicas: 0` means no workers (the workflow will not provision worker VMs)
- `workers_replicas: 1` or higher provisions that many worker VMs

When workers are enabled:

1. Add `servers.workers.cmd` and `servers.workers.proxy: false` to `config/deploy.yml` :

```
  # config/deploy.yml
```

```
  servers:
    workers:
      cmd: "celery -A tasks worker --loglevel=info"
      proxy: false
```

2. Add worker hosts to `config/deploy.{env}.yml` . The provision job exports `INFRA_WORKER_IP_0` , `INFRA_WORKER_IP_1` , etc. -- one for each `workers_replicas` :

```
  # config/deploy.preview.yml
  servers:
    web:
      hosts:
        - <%= ENV['INFRA_WEB_IP'] %>
    workers:
      hosts:
        - <%= ENV['INFRA_WORKER_IP_0'] %>
        - <%= ENV['INFRA_WORKER_IP_1'] %>
```

3. Set `workers_replicas` and `workers_plan` in the caller workflow's infra job:

```
  with:
    workers_replicas: 2
    workers_plan: "small"
```

Scaling

Scaling happens by updating workflow inputs and/or Kamal config, then redeploying. See references/scaling.md for VM plans, disk sizes, and detailed instructions.

Web (vertical only)

Change `web_plan` in the workflow and redeploy. No config file changes needed. Causes brief downtime.

Workers

- **Vertical**: Change `workers_plan` in the workflow and redeploy. No config file changes needed. Causes brief downtime.
- **Horizontal**: Change `workers_replicas` in the workflow. **Also update the host list** in `config/deploy.<env>.yml` to match the new replica count — see references/scaling.md -- Scaling Workers for details. Commit, push, and redeploy.

Accessories (database, redis, etc.)

Change the `plan` in the `accessories` JSON in the workflow. **For database accessories using `supabase/postgres` **, also regenerate the `cmd` — see references/scaling.md -- Scaling the Database for the full procedure.

Other accessories may also have memory-dependent configuration (e.g., `maxmemory` for Redis, JVM heap for Elasticsearch). When scaling any accessory, review its `cmd` , `env` , or container configuration in `config/deploy.<env>.yml` and adjust parameters to match the new plan's resources.

Teardown

See references/teardown.md for tearing down environments, inferring zone/env_name from existing workflows, and reading last run outputs.

Disaster Recovery

Every deployed environment has automatic daily snapshots of all data volumes. If an environment is lost (teardown, VM failure, or data corruption), it can be recovered by running the deploy workflow with `recover: true` .

When the user asks to "recover", "recuperar", "restore", "do DR", or "disaster recovery": the critical action is adding `recover: true` to the infra job inputs in the deploy workflow. Without this flag, the workflow creates blank

disks and the data is lost. Simply changing the zone is **not** recovery — recovery means restoring from snapshots via the `recover: true` flag.

Recovery procedure:

1. If the original deployment still exists in the target zone, either tear it down first or change the `zone` input to recover in a different zone (snapshots are replicated across zones, so recovery works in either zone)
2. Add `recover: true` to the infra job's `with:` block in the deploy workflow (e.g., `.github/workflows/deploy-preview.yml`)
3. Commit, push, and monitor the workflow run
5. After recovery succeeds, verify data is intact (health check, SSH into VMs, check database rows and files)
6. **Remove `recover: true` ** from the workflow file — if left in place, subsequent runs will fail because the network and volumes already exist

See references/recovery.md for the full procedure, pre-flight requirements, and current limitations.

Development Without Local Environment

When the developer cannot run the language runtime or database locally, the Deployment Feedback Loop becomes the primary iteration cycle:

1. Commit and push changes
2. Monitor the workflow run until it succeeds
3. Run the health check (`curl /up`) to verify
4. If the health check fails, SSH debug, fix, and repeat from step 1

Recommendation: Start with the default `preview` environment triggered on push, without a domain. This gives immediate feedback on every change, with TLS via nip.io. When the app is mature, consider adding a **production** environment (triggered on version tags) for public release -- especially important when no local dev environment is available, since preview is the only place to catch issues before they reach users. See Choosing the Target Environment for a Domain for guidance on where to attach a custom domain.

References

- **references/operations.md** -- Post-deployment operations: finding IPs, SSH access, database access, container debugging
- **references/setup-and-deploy.md** -- Detailed commands for each setup step, development routine, and SSH debugging
- **references/workflows.md** -- Complete caller workflow examples (deploy + teardown) with all inputs documented
- **references/env-vars.md** -- Environment variables and secrets configuration
- **references/scaling.md** -- VM plans, worker scaling, disk sizes
- **references/teardown.md** -- Teardown process, inferring parameters, reading outputs
- **references/recovery.md** -- Disaster recovery from snapshots: procedure, pre-flight checks, limitations
- **references/postgres-recipe.md** -- Recipe for the `supabase/postgres` image, a Postgres image enriched with extensions as recommended by the **tech-stack** skill
- **references/kamal.md** -- Kamal deployment concepts: service/worker/accessory architecture, configuration syntax, proxy, environment variables, secrets, builder, commands

references/env-vars.md

Environment Variables and Secrets Configuration

Table of Contents

Clear Variables (deploy.yml)

Non-sensitive configuration variables are written under `env.clear`. Variables common to all environments go in `config/deploy.yml`; environment-specific variables go in `config/deploy.<env>.yml`.

```
# config/deploy.yml -- common to all environments
env:
  clear:
    LOG_LEVEL: debug
    MAX_UPLOAD_SIZE: 50MB
```

```
# config/deploy.preview.yml -- environment-specific
env:
  clear:
    APP_ENV: preview
```

These become clear (non-secret) environment variables in the container.

Secret Variables

Sensitive configuration flows directly from GitHub Secrets through the caller workflow's deploy job `env:` block, into the runner environment, where `.kamal/secrets.<dest>` reads them and passes them to the container.

How it works

1. **Agent writes secret names** in two places:
   - `deploy.<env>.yml` under `env.secret` -- tells Kamal which env vars are secrets (list ALL secrets here, both common and env-specific)
   - `.kamal/secrets-common` and `.kamal/secrets.<destination>` -- tells Kamal to read values from the runner environment

2. **Caller workflow maps GitHub Secrets as env vars** on the deploy job:

```
# In the caller workflow's deploy job
deploy:
  needs: infra
  runs-on: ubuntu-latest
  env:
    POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
    API_KEY: ${{ secrets.API_KEY }}
```

3. **Kamal reads from the environment** via `.kamal/secrets-common` and `.kamal/secrets.<destination>` and injects them into the container.

Store each secret **individually** as a GitHub Secret. Ask the user to set these via the GitHub UI (see setup-and-deploy.md -- Secrets the user must set via GitHub UI). **Never** accept secret values through the chat.

Secrets like `DATABASE_URL` that can be derived from other secrets do not need their own GitHub Secret -- they are composed in the `.kamal/secrets` file.

deploy.yml and .kamal/secrets configuration

The agent writes secret names in `env.secret` blocks and `.kamal/secrets` files. **All secrets must be listed in each `config/deploy.<env>.yml` ** — do NOT put `env.secret` in the base `config/deploy.yml`. This is because Kamal deep-merges destination files on top of the base config, and arrays are replaced (not appended). If the base config has `env.secret: [MY_SECRET]` and the destination has `env.secret: [DATABASE_URL]`, the result is only `[DATABASE_URL]` — `MY_SECRET` is silently lost.

Secret *values* still use separate `.kamal/secrets-common` (for common secrets) and `.kamal/secrets.<env>` (for env-specific secrets) files — the merge issue only affects the `env.secret` array in YAML deploy configs.

```
# config/deploy.<env>.yml -- ALL secrets for this environment
env:
  secret:
    - MY_SECRET        # common secret, value from .kamal/secrets-common
    - POSTGRES_PASSWORD # env-specific, value from .kamal/secrets.<env>
    - DATABASE_URL      # env-specific, value from .kamal/secrets.<env>
```

For **preview** (default environment), the `.kamal/secrets` file uses unsuffixed env var names. Derived secrets like `DATABASE_URL` are composed inline:

```
# .kamal/secrets.preview
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
API_KEY=$API_KEY
```

For **non-preview** environments (e.g., production), the `.kamal/secrets` file maps the Kamal secret name (left side) to the suffixed env var name (right side):

```
# .kamal/secrets.production
POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
API_KEY=$API_KEY_PRODUCTION
```

Passing Variables in Caller Workflows

See workflows.md for complete caller workflow examples showing the two-job pattern. See setup-and-deploy.md -- Creating GitHub Secrets for how to create the secrets referenced below.

Complete example showing both clear and secret custom variables for a production environment. Note the two-job pattern: infra job handles infrastructure, deploy job handles Kamal and application secrets. Environment-scoped secrets use the `_PRODUCTION` suffix, while common secrets `CLOUDSTACK_*`) are shared across all environments:

```
jobs:
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      env_name: "production"
      zone: "ZP01"
      accessories: '[{"name":"db","plan":"medium","disk_size_gb":50}]'
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY_PRODUCTION }}

  deploy:
    needs: infra
    runs-on: ubuntu-latest
    env:
      POSTGRES_PASSWORD_PRODUCTION: ${{ secrets.POSTGRES_PASSWORD_PRODUCTION }}
      API_KEY_PRODUCTION: ${{ secrets.API_KEY_PRODUCTION }}
      JWT_SECRET_PRODUCTION: ${{ secrets.JWT_SECRET_PRODUCTION }}
    steps:
```

```
        # ... (checkout, load infra_env, Kamal install, deploy steps)
```

Clear variables are written directly in `deploy.yml` by the agent -- they are not passed through the workflow.

Inter-Component Connectivity

When the app, workers, or accessories need to communicate with each other, use the **CloudStack internal DNS hostname** — the accessory name (e.g., `db`, `redis`, `waha`) — **never** use `INFRA_*_IP` env vars.

The `INFRA_*_IP` variables exported by the provision workflow contain **public** IPs. They are meant only for:
- Kamal `host:` fields (SSH deployment to VMs)
- `servers.web.hosts` / `servers.workers.hosts` (Kamal server lists)
- `proxy.host` (external-facing URLs like `<%= ENV['INFRA_WEB_IP'] %>.nip.io`)

Using public IPs for inter-component communication (database hosts, cache endpoints, message brokers, etc.) routes traffic through the external interface, which may fail due to firewall rules or the service port not being open publicly. CloudStack internal DNS resolves the accessory name to its private IP on the internal network, which is fast, reliable, and always available.

Examples

```
# CORRECT — use the accessory name as the hostname
env:
  clear:
    WORDPRESS_DB_HOST: db
    REDIS_URL: redis://redis:6379
    WAHA_API_URL: http://waha:3000

# WRONG — never use INFRA_*_IP for inter-component communication
env:
  clear:
    WORDPRESS_DB_HOST: <%= ENV['INFRA_DB_IP'] %>           # public IP, will fail
    REDIS_URL: redis://<%= ENV['INFRA_REDIS_IP'] %>:6379   # public IP, will fail
```

This rule applies to all inter-component references regardless of whether the components are on the same VM or different VMs.

Database Connection Variables

The application uses `DATABASE_URL` to connect to the database. This is **derived from `POSTGRES_PASSWORD`** in the `.kamal/secrets` file -- it is not a separate GitHub Secret. See postgres-recipe.md -- DATABASE_URL Derived from POSTGRES_PASSWORD for the full recipe.

The hostname in `DATABASE_URL` is `db`, which resolves via CloudStack internal DNS to the database VM's private IP. The format is:

```
postgres://postgres:<password>@db:5432/postgres
```

Example usage in application code:

```
# Python example
import os
database_url = os.environ["DATABASE_URL"]
# database_url = "postgres://postgres:mypassword@db:5432/postgres"
```

```
// Node.js example
const connectionString = process.env.DATABASE_URL;
```

```
# Ruby/Rails example (config/database.yml)
production:
  url: <%= ENV["DATABASE_URL"] %>
```

The port is always 5432. The hostname is always `db`.

Disk Storage Path

Both the web VM and each accessory VM have a persistent disk mounted at `/data/` . For main app roles (web, workers), use Kamal `volumes` ; for accessories, use `directories` . Never use named Docker volumes. Always map to a **subdirectory** of `/data/` (never `/data/` root). `/data/` is an attached disk with scheduled snapshot policies for disaster recovery — data outside `/data/` is not backed up. Additionally, `lost+found` from the ext4 filesystem at the mount root would interfere with containers expecting a clean directory. See postgres-recipe.md -- Volume Mount for the database accessory example.

Web VM example

To use the web disk for file storage (uploads, media, etc.):

1. Map a Kamal volume to a subdirectory of `/data/` :

```
    volumes:
      - /data/uploads:/app/uploads
```

2. Set a clear env var in `deploy.yml` so the app knows the path:

```
    env:
      clear:
        UPLOAD_PATH: /app/uploads
```

Accessory VM example

Each accessory VM has its own disk at `/data/` . Map the accessory's data directory to a subdirectory:

```
accessories:
  db:
    directories:
      - /data/pgdata:/var/lib/postgresql/data
  redis:
    directories:
      - /data/redis:/data
```

The subdirectory names are up to the application — there is no platform-mandated convention.

references/kamal.md

Kamal

Kamal deploys Docker containers to bare metal or VMs using zero-downtime rolling deploys with an integrated reverse proxy (kamal-proxy). It uses a base `deploy.yml` config file with per-environment destination overrides, and connects via SSH — no Kubernetes, no orchestrator.

Architecture: Service, Workers, and Accessories

A Kamal deployment has three types of components:

Service (the app)

The **service** builds and runs the Dockerfile in the default **web** servers role. This is the primary application — it receives traffic through kamal-proxy.

- The Dockerfile at the repo root defines the service image
- The default `CMD` in the Dockerfile runs the web server
- kamal-proxy routes incoming HTTP/HTTPS traffic to the service container
- **For automatic TLS certificate issue and renewal via Let's Encrypt, run only one web server** — Let's Encrypt HTTP-01 challenge requires a single point to answer the challenge; multiple web servers behind the proxy would race on certificate provisioning

Workers

Workers use the same Dockerfile as the service but run an alternate `CMD` . They are background job processors that don't receive web traffic.

Worker command and behavior go in the base `config/deploy.yml` :

```
# config/deploy.yml
servers:
  workers:
    cmd: "celery -A tasks worker --loglevel=info"
```

Worker hosts go in the destination config `config/deploy.{env}.yml` :

```
# config/deploy.preview.yml
servers:
  web:
    hosts:
      - <%= ENV['INFRA_WEB_IP'] %>
  workers:
    hosts:
      - <%= ENV['INFRA_WORKER_IP_0'] %>
```

- Workers share the same Docker image — only the command differs
- The proxy is only enabled on the primary role (web) by default — all other roles including workers have it disabled, so there is no need to set `proxy: false` explicitly
- Workers can have their own `options` , `env` , `labels` , and `logging` overrides
- Multiple worker roles are possible for different job types (define each in the base config with its `cmd` , then assign hosts per destination)

Accessories

Accessories are standalone Docker containers for public images — databases (MySQL, PostgreSQL), messaging (Kafka, WAHA gateway), caches (Redis), automation tools (n8n), search engines (Meilisearch), and similar services.

Accessories are defined in the destination config `config/deploy.{env}.yml` :

```
# config/deploy.preview.yml
accessories:
  db:
    image: supabase/postgres:17.6.1.093
    host: <%= ENV['INFRA_DB_IP'] %>
    port: "5432:5432"
    cmd: "postgres -D /etc/postgresql -c shared_buffers=1GB -c max_connections=100"
    env:
      secret:
        - POSTGRES_PASSWORD
    directories:
      - /data/pgdata:/var/lib/postgresql/data
```

Accessories can also run behind kamal-proxy to receive routed HTTP and HTTPS traffic. This is useful for web-facing services like dashboards, admin panels, or APIs. With `ssl: true` , Let's Encrypt certificate issuance and auto-renewal works for accessories as well. The accessory's port must be opened in the infrastructure's public IP firewall — kamal-proxy only routes traffic that reaches the host.

```
# config/deploy.preview.yml
accessories:
  nginx:
    image: nginx:alpine
    host: <%= ENV['INFRA_NGINX_IP'] %>
    directories:
      - /data/nginx/html:/usr/share/nginx/html
    proxy:
      host: docs.example.com # or docs.<%= ENV['INFRA_NGINX_IP'] %>.nip.io
      ssl: true
      app_port: 80
```

Key characteristics:
- Accessories use public Docker images — they are **not** built from the repo's Dockerfile
- Each accessory is deployed to specific hosts independently from the app
- Accessories are **not** updated on `kamal deploy` — any configuration change (image tag bump, env var change, volume/port/cmd adjustment, etc.) requires `kamal accessory reboot <name>` to take effect
- Accessories have downtime on reboot (no rolling deploy) — always warn the user before rebooting
- The proxy is disabled by default on accessories. Enable with `proxy: {...}` to route traffic through kamal-proxy — the accessory's port must be open at the infrastructure firewall level for traffic to reach the host
- **Accessory health checks**: When enabling the proxy on an accessory, always configure a health check path that the accessory actually exposes. The default `/up` is designed for the main app and most third-party images do not serve it. If the health check fails, kamal-proxy will never route traffic to the accessory. For example, WordPress does not expose `/up` , so use a known endpoint:

```
accessories:
  wordpress:
    proxy:
      healthcheck:
        path: /wp-login.php
```

- **Use `directories` for accessory persistent data mounts** — `directories` are auto-created on the host by Kamal, support `mode` and `owner` options, and map to visible host paths under `/data/` where snapshot-based backups run. For main app roles (web, workers), use `volumes` instead — see the Volumes and Asset Bridging section. Never use named Docker volumes `myapp_data:/path`) — they bypass the persistent disk entirely and data stored in them is **not backed up** and will be lost on VM replacement:

```
# String format — host:container
directories:
  - /data/pgdata:/var/lib/postgresql/data

# Hash format — with mode and owner
directories:
  - local: /data/redis
    remote: /data
    mode: "0750"
    owner: "999:999"
```

- Use `files` for config file mounts, optionally with permissions:
`files: [{local: config/my.cnf, remote: /etc/mysql/my.cnf, mode: "0600", owner: "mysql:mysql"}]`
- For main app roles (web, workers), use `volumes` instead of `directories` — see the Volumes and Asset Bridging section. Never use named Docker volumes (e.g., `myapp_data:/path`) — they store data outside `/data/` and are **not covered by snapshot backups**
- **Internal networking**: When the app or accessories need to communicate with each other, use the **CloudStack internal DNS hostname** (the accessory name, e.g., `db`, `redis`) — **never** use `INFRA_*_IP` env vars. The `INFRA_*_IP` variables contain **public** IPs meant only for Kamal SSH deployment and external-facing URLs (like `proxy.host`). Using public IPs for inter-component communication routes traffic out through the external interface and back in, which may fail due to firewall rules, NAT hairpinning, or the accessory's service port not being open on the public firewall. CloudStack internal DNS resolves the accessory name to its private IP on the internal network, which is fast, reliable, and always available. For example, if a `wordpress` service needs to reach a `db` accessory, use `WORDPRESS_DB_HOST: db` — not `WORDPRESS_DB_HOST: <%= ENV['INFRA_DB_IP'] %>`. This applies to all inter-component references: database hosts, cache endpoints, message brokers, etc. The same rule applies to containers on the same host (where Kamal's shared Docker network also provides name resolution) and across VMs (where CloudStack internal DNS handles it)

Configuration

deploy.yml (base config)

The central configuration file at `config/deploy.yml`. Contains settings common to all environments — service identity, proxy behavior, registry, builder, SSH, common env vars, worker definitions, and deployment timings. Server hosts, accessories, and environment-specific settings go in destination files instead.

```
service: myapp
image: myapp

proxy:
  app_port: 80
  forward_headers: false
  healthcheck:
    path: /up
    interval: 3
    timeout: 5

ssh:
  user: root

registry:
  server: ghcr.io
  username: myorg
  password:
    - KAMAL_REGISTRY_PASSWORD

builder:
  arch: amd64
  cache:
    type: gha
    options: mode=max

env:
  clear:
    MY_VAR: hello
  # Do NOT put env.secret in the base config — see "Deep merge rules" below

servers:                      # only needed if using workers
  workers:
    cmd: "bin/worker"

readiness_delay: 15
deploy_timeout: 180
drain_timeout: 30
```

Destinations (environment-specific config)

Per-environment override files at `config/deploy.{env}.yml` (e.g. `deploy.preview.yml`, `deploy.production.yml`). Kamal deep-merges the destination file on top of the base `deploy.yml`. Deploy with: `kamal deploy -d preview`.

Destination files contain: server hosts (web + workers), environment-specific env vars, proxy host/ssl, and accessories.

```
# config/deploy.preview.yml
servers:
  web:
    hosts:
      - <%= ENV['INFRA_WEB_IP'] %>
  workers:
    hosts:
      - <%= ENV['INFRA_WORKER_IP_0'] %>

env:
  clear:
    APP_ENV: preview
  secret:
    - MY_SECRET              # common secrets, from .kamal/secrets-common
    - DATABASE_URL           # from .kamal/secrets.preview

proxy:
  host: <%= ENV['INFRA_WEB_IP'] %>.nip.io
  ssl: true

accessories:
  db:
    image: supabase/postgres:17.6.1.093
    host: <%= ENV['INFRA_DB_IP'] %>
    port: "5432:5432"
    cmd: "postgres -D /etc/postgresql -c shared_buffers=1GB"
    env:
      secret:
        - POSTGRES_PASSWORD
    directories:
      - /data/pgdata:/var/lib/postgresql/data
```

Deep merge rules:
- Scalar values are replaced
- Arrays are replaced (not appended)
- Hashes are recursively merged

IMPORTANT: Because arrays are replaced, `env.secret` must NOT be split between the base `deploy.yml` and destination files. If the destination file defines `env.secret`, it completely overwrites the base list — any secrets declared only in the base config will be silently lost. Put ALL secrets (common + environment-specific) in each `config/deploy.<env>.yml`. Note: this does not affect `.kamal/secrets-common` and `.kamal/secrets.<env>` — those are separate files that Kamal reads independently.

Enforce explicit destination with `require_destination: true` in the base config.

ERB Templating

Every YAML config file is processed through Ruby's ERB before YAML parsing: `ERB.new(File.read(file)).result`. This enables environment variable injection:

```
servers:
  web:
    hosts:
      - <%= ENV['WEB_IP'] %>
```

ERB is evaluated at deploy time on the machine running `kamal deploy`.

Proxy (kamal-proxy)

Built-in reverse proxy for zero-downtime deploys, automatic TLS, health checking, and request buffering.

Core settings

```
proxy:
  host: app.example.com       # Route only matching requests (optional)
  app_port: 3000               # Port the app listens on (default: 80)
  ssl: true                    # Let's Encrypt automatic TLS
  forward_headers: false       # Recommended when VMs are directly exposed to the internet with no upstream proxy
  response_timeout: 30         # Seconds (default: 30)
  ssl_redirect: true           # Redirect HTTP to HTTPS (default: true)
```

Multiple hosts:

```
proxy:
  hosts:
    - foo.example.com
    - bar.example.com
```

Custom TLS certificates

```
proxy:
  ssl:
    certificate_pem: CERTIFICATE_PEM    # Secret names
    private_key_pem: PRIVATE_KEY_PEM
```

Health checks

kamal-proxy checks the app before routing traffic:

```
proxy:
  healthcheck:
    path: /up            # Default: /up
    interval: 1          # Seconds between checks (default: 1)
    timeout: 5           # Seconds per check (default: 5)
```

Request/response buffering

```
proxy:
  buffering:
    requests: true
    responses: true
    max_request_body: 40_000_000
    max_response_body: 0
    memory: 2_000_000
```

Path-based routing

```
proxy:
  path_prefix: "/api,/oauth_callback"
  strip_path_prefix: false
```

Proxy on roles

- **Primary role (web)**: proxy enabled by default. Disable explicitly with `proxy: false` if needed
- **Other roles (workers, accessories, etc.)**: proxy disabled by default — no need to set `proxy: false`. Enable with `proxy: true` or custom proxy config

Proxy runtime settings

```
proxy:
  run:
```

```
    http_port: 80          # Default: 80
    https_port: 443        # Default: 443
    metrics_port: 9090
    log_max_size: "10m"
```

Environment Variables and Secrets

Clear variables

Non-sensitive values, placed in `env.clear` in either the base or destination config:

```
# config/deploy.yml — common to all environments
env:
  clear:
    BLOB_STORAGE_PATH: /data/blobs
```

```
# config/deploy.preview.yml — environment-specific
env:
  clear:
    APP_ENV: preview
```

Secrets

Secret names are listed in `env.secret` in each destination file; values are resolved from `.kamal/secrets-common` and `.kamal/secrets.<destination>` at deploy time. Because arrays are replaced (not merged) during deep merge, **all secrets — both common and environment-specific — must be listed in each `config/deploy.<env>.yml` **. Do NOT put `env.secret` in the base `deploy.yml` .

```
# config/deploy.preview.yml — ALL secrets for this environment
env:
  secret:
    - MY_SECRET              # common secret, resolved from .kamal/secrets-common
    - DATABASE_URL           # env-specific, resolved from .kamal/secrets.preview
```

`.kamal/secrets-common` and `.kamal/secrets.<destination>` are dotenv files evaluated in the shell context:

```
# .kamal/secrets-common
KAMAL_REGISTRY_PASSWORD=$KAMAL_REGISTRY_PASSWORD
MY_SECRET=$MY_SECRET
```

```
# .kamal/secrets.preview
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

Aliased secrets

Map a different secret name to the env var:

```
env:
  secret:
    - DB_PASSWORD:MAIN_DB_PASSWORD
```

Tag-based env variables

Per-host or per-tag environment overrides:

```
servers:
  - 1.1.1.1
  - 1.1.1.2: monitoring

env:
  clear:
    ROLE: standard
  tags:
```

```
    monitoring:
      ROLE: monitor
```

Secret adapters

Kamal can fetch secrets from password managers:

```
SECRETS=$(kamal secrets fetch --adapter 1password --account myaccount --from MyVault/MyItem DB_PASSWORD)
DB_PASSWORD=$(kamal secrets extract DB_PASSWORD $SECRETS)
```

Supported adapters: 1Password, LastPass, Bitwarden, AWS Secrets Manager, GCP Secret Manager, Doppler.

Docker Registry

Docker Hub (default)

```
registry:
  username: dockerhub_user
  password:
    - KAMAL_REGISTRY_PASSWORD
```

GitHub Container Registry

```
registry:
  server: ghcr.io
  username: github_user
  password:
    - KAMAL_REGISTRY_PASSWORD
```

AWS ECR

```
registry:
  server: <account-id>.dkr.ecr.<region>.amazonaws.com
  username: AWS
  password: <%= %x(aws ecr get-login-password) %>
```

Local registry

Kamal starts it automatically:

```
registry:
  server: localhost:5555
```

Builder

```
builder:
  arch: amd64                     # Required: amd64, arm64, or both
  dockerfile: Dockerfile.production  # Default: Dockerfile
  context: .                      # Default: Git HEAD
  target: production              # For multi-stage builds
  args:
    RUBY_VERSION: 3.2.0
  secrets:
    - GITHUB_TOKEN
  cache:
    type: registry                # or "gha" for GitHub Actions cache
    image: myapp-build-cache
    options: mode=max
```

Remote builder for cross-architecture:

```
builder:
```

```
  arch:
    - amd64
    - arm64
  remote: ssh://docker@docker-builder
  local: true    # Use local when arch matches (default)
```

Cloud Native Buildpacks (alternative to Dockerfile)

```
builder:
  pack:
    builder: heroku/builder:24
    buildpacks:
      - heroku/ruby
```

Servers and Roles

Simple server list (implicitly web role)

```
servers:
  - 1.1.1.1
  - 1.1.1.2
```

Role definitions in the base config

The base `deploy.yml` defines role behavior (cmd, proxy, options). Hosts go in the destination config:

```
# config/deploy.yml — role behavior
servers:
  workers:
    cmd: bin/jobs
    options:
      memory: 2g
      cpus: 4
    env:
      clear:
        WORKER_THREADS: "5"
```

```
# config/deploy.preview.yml — hosts per environment
servers:
  web:
    hosts:
      - <%= ENV['INFRA_WEB_IP'] %>
  workers:
    hosts:
      - <%= ENV['INFRA_WORKER_IP_0'] %>
```

Server tags

Tags allow per-host environment variable overrides:

```
servers:
  web:
    - 1.1.1.1
    - 1.1.1.2: experiments
    - 1.1.1.3: [experiments, monitoring]
```

Boot and Rollout

Staged rollout

```
boot:
  limit: 25%          # Deploy to 25% of hosts at a time (or integer)
```

```
    wait: 10            # Seconds between groups
    parallel_roles: true # Boot roles in parallel on same host
```

Container options

```
  options:
    restart: unless-stopped
    memory: 2g
    cpus: 4
```

Health checks for non-proxy roles

Roles without the proxy use Docker health checks (defined in the base config):

```
  # config/deploy.yml
  servers:
    workers:
      cmd: bin/worker
      options:
        health-cmd: bin/worker-healthcheck
        health-start-period: 5s
        health-interval: 5s
        health-retries: 5
```

Deployment timings

```
  deploy_timeout: 30     # Seconds to wait for container readiness (default: 30)
  drain_timeout: 30      # Seconds to wait for request draining (default: 30)
  readiness_delay: 7     # Wait time for containers without healthchecks (default: 7)
```

SSH

```
  ssh:
    user: root                # Default: root
    keys: [.kamal/ssh_key]    # Private key file(s)
```

Additional options:

```
  ssh:
    user: root
    port: "22"                # Default: 22
    proxy: root@proxy-host    # SSH jump host
    keys:
      - .kamal/ssh_key
    key_data:
      - SSH_PRIVATE_KEY        # Secret name containing the key
```

SSHKit tuning (for many servers)

```
  sshkit:
    max_concurrent_starts: 30   # Default: 30
    pool_idle_timeout: 900       # Default: 900
```

Hooks

Lifecycle scripts in `.kamal/hooks/` (no file extension). A non-zero exit code aborts the command.

Available hooks:
- `docker-setup` — after Docker is installed
- `pre-connect` — before connecting to servers
- `pre-build` — before building the image
- `pre-deploy` — before deploying

- `post-deploy` — after successful deployment
- `pre-app-boot` — before booting app container
- `post-app-boot` — after app container boots
- `pre-proxy-reboot` — before rebooting proxy
- `post-proxy-reboot` — after proxy reboots

Environment variables available in hooks:
- `KAMAL_PERFORMER` — local user
- `KAMAL_SERVICE_VERSION` — e.g. `app@150b24f`
- `KAMAL_VERSION` — full version hash
- `KAMAL_HOSTS` — comma-separated target hosts
- `KAMAL_COMMAND` / `KAMAL_SUBCOMMAND` — command being run
- `KAMAL_DESTINATION` — destination (e.g. `staging`)

Volumes and Asset Bridging

Volumes

```
# config/deploy.yml — volumes on the web VM
volumes:
  - /data/blobs:/data/blobs
```

Asset bridging

For apps with content-hashed assets (CSS/JS), Kamal mounts both old and new asset directories during deployment to avoid 404s:

```
asset_path: /public
```

Logging

```
logging:
  driver: json-file
  options:
    max-size: 100m
```

Can be set at root level or per-role.

Retain Containers

```
retain_containers: 5    # Keep last 5 old containers for rollback (default: 5)
```

YAML Anchors

Reuse configuration blocks with YAML anchors (prefix with `x-`):

```
# config/deploy.yml
x-worker-healthcheck: &worker-healthcheck
  health-cmd: bin/worker-healthcheck
  health-start-period: 5s
  health-retries: 5
  health-interval: 5s

servers:
  queue:
    cmd: bin/queue
    options:
      <<: *worker-healthcheck
  scheduler:
    cmd: bin/scheduler
```

```
      options:
        <<: *worker-healthcheck
```

Commands

Deployment

Command	Description
`kamal deploy`	Full deploy: build, push, boot, route traffic, prune
`kamal deploy -d staging`	Deploy to a specific destination
`kamal setup`	First-time setup: install Docker, boot accessories, deploy
`kamal redeploy`	Fast redeploy: skip setup/proxy/pruning
`kamal rollback <VERSION>`	Revert to a previous version

Deploy options: `--skip-push`, `--primary`, `--hosts=HOST1,HOST2`, `--roles=ROLE1`, `--skip-hooks`.

App management

Command	Description
`kamal app containers`	List running containers (use `-q` for rollback versions)
`kamal app logs`	Stream application logs
`kamal app exec <CMD>`	Run a command inside the app container
`kamal app details`	Detailed container info
`kamal app version`	Show running version

Accessories

Command	Description
`kamal accessory boot <NAME>`	Start an accessory
`kamal accessory boot all`	Start all accessories
`kamal accessory reboot <NAME>`	Restart an accessory — **required** after any config change (image tag, env, volumes, ports, cmd). Causes downtime.
`kamal accessory reboot all`	Restart all accessories
`kamal accessory logs <NAME>`	View accessory logs
`kamal accessory exec <NAME> <CMD>`	Run command in accessory container
`kamal accessory remove <NAME>`	Remove an accessory

Build

Command	Description
`kamal build push`	Build and push image
`kamal build pull`	Pull image on servers
`kamal build deliver`	Push then pull
`kamal build dev`	Build locally (tagged as 'dirty')

Other

Command	Description
`kamal config`	Display resolved config (YAML)
`kamal lock`	Check deploy lock status
`kamal audit`	View audit log

`kamal prune`	Clean up old images/containers
`kamal secrets print`	Print resolved secrets (debug)
`kamal secrets fetch`	Fetch from password manager adapters

Cron Jobs

Use a dedicated role that installs the crontab from the environment:

```
# config/deploy.yml — role definition
servers:
  cron:
    cmd: bash -c "(env && cat config/crontab) | crontab - && cron -f"
```

Note: cron doesn't propagate environment variables — the pattern above copies them into the crontab.

Multiple Apps on the Same Host

Multiple apps can share a host via kamal-proxy:
- Apps with `proxy.host` configured receive only matching requests
- An app without `proxy.host` acts as a catch-all for unmatched requests
- Automatic TLS via Let's Encrypt (`ssl: true`) does not work with multiple apps on the same host — use custom certificates instead

references/operations.md

Post-Deployment Operations

Reference for interacting with deployed infrastructure: finding IPs, SSH access, accessory access, container debugging.

Finding Deployment IPs

Every deploy workflow run produces a `provision-output` artifact containing a JSON file with all IPs (see workflows.md -- Deploy Output Reference for the full output spec). **Always clean up before downloading** to avoid reading stale data from a previous run:

```
# 1. Find the run ID for the environment you need
gh run list --workflow=deploy-preview.yml --limit=5

# 2. Clean up any previous artifact, then download
rm -rf ~/provision-output
gh run download <run-id> --name provision-output --dir ~/provision-output

# 3. Read the IPs
cat ~/provision-output/provision-output.json
```

The JSON contains:

Key	Description
`web_ip`	Public IP of the web VM -- used for SSH and app URL `https://<web_ip>.nip.io`)
`worker_ips`	JSON array of public worker VM IPs -- used for SSH
`accessory_ips`	JSON object with accessory IPs. Each key is the accessory name (e.g., `db` , `redis`) with `ip` (public, SSH only) and `internal_ip` (private, used by the app) fields.

Example `provision-output.json` :

```
{
  "web_ip": "200.234.x.x",
  "worker_ips": ["200.234.y.y"],
  "accessories": {
    "db": {
      "ip": "200.234.z.z",
      "internal_ip": "10.1.1.x"
    },
    "redis": {
      "ip": "200.234.w.w",
      "internal_ip": "10.1.1.y"
    }
  }
}
```

Fallback: get IPs from the workflow run summary

If you don't need the full JSON, the workflow run summary includes an IP table:

```
gh run view <run-id>
```

SSH Access

User is always `root` . Use the SSH key that matches the environment. See setup-and-deploy.md -- SSH Key Generation for how these keys are generated.

Resolve the key path first

Before any SSH command, determine the repo name and construct the key path. **Always use `-i` to specify the key** — never rely on the SSH client's default key selection:

```
# Get the repo name (without the owner prefix)
REPO_NAME=$(gh repo view --json name -q .name)

# Preview environment key
SSH_KEY=~/.ssh/$REPO_NAME

# Other environments (e.g., production)
SSH_KEY=~/.ssh/$REPO_NAME-production
```

Key locations

Environment	Key path
preview (default)	`~/.ssh/<repo-name>`
production	`~/.ssh/<repo-name>-production`
other `<env_name>`	`~/.ssh/<repo-name>-<env_name>`

`<repo-name>` is the name of the GitHub repository (not the owner/org prefix).

Connection commands

```
# Preview environment
ssh -i ~/.ssh/<repo-name> root@<web_ip>
ssh -i ~/.ssh/<repo-name> root@<accessory_ip>
ssh -i ~/.ssh/<repo-name> root@<worker_ip>

# Production (or other named environment)
ssh -i ~/.ssh/<repo-name>-production root@<web_ip>
ssh -i ~/.ssh/<repo-name>-production root@<accessory_ip>
ssh -i ~/.ssh/<repo-name>-production root@<worker_ip>
```

Use the accessory's `ip` field from the provision output for `<accessory_ip>`. For example, to SSH into the database VM: `root@<accessories.db.ip>`.

Accessory Access

Each accessory runs inside a Docker container on its own VM. Accessory VMs only expose SSH (port 22) — service ports are **not** reachable from the public internet. You must SSH into the accessory VM first, then connect locally via `docker exec`.

The container name follows Kamal's naming convention: `<service-name>-<accessory-name>`. For example, a database accessory named `db` runs as `<repo-name>-db`.

Understanding accessory IPs

- **`accessories.<name>.ip` -- public IP, used for** SSH access** to the accessory VM
- **`accessories.<name>.internal_ip` ** -- private IP on the CloudStack network. App containers connect to the accessory by its short hostname (e.g., `db`, `redis`) via CloudStack internal DNS — no IP needed.

Example: connecting to PostgreSQL

The examples below use a Postgres accessory named `db`. The same pattern applies to any accessory — replace the `docker exec` command with what's appropriate for the service.

```
# 1. SSH into the accessory VM
ssh -i ~/.ssh/<repo-name> root@<accessories.db.ip>

# 2. Connect to Postgres via the container
docker exec -it <repo-name>-db psql -U postgres
```

Run a one-off command

```
# From outside the accessory VM (combines SSH + docker exec)
ssh -i ~/.ssh/<repo-name> root@<accessories.db.ip> \
  'docker exec <repo-name>-db psql -U postgres -c "SELECT version();"'
```

Container Debugging

After SSHing into a VM, use these commands to inspect running containers:

```
# List running containers
docker ps

# View web app logs (last 100 lines)
docker logs $(docker ps -q --filter "label=service=<repo-name>") --tail 100

# Follow logs in real time
docker logs $(docker ps -q --filter "label=service=<repo-name>") -f

# Check if the app responds locally
curl -s localhost:80/up

# View kamal-proxy logs (web VM only)
docker logs kamal-proxy --tail 50

# Check accessory container logs (e.g. Postgres on the db accessory VM)
docker logs <repo-name>-db --tail 100

# Check disk mounts
df -h /data    # web VM or accessory VM

# Check container environment variables
docker exec $(docker ps -q --filter "label=service=<repo-name>") env

# Open a shell inside the app container
docker exec -it $(docker ps -q --filter "label=service=<repo-name>") sh
```

Common Pitfalls

Stale artifact files

`gh run download` **does not** clean the target directory -- it merges files, and existing files cause a collision error or are silently kept. **Always** `rm -rf ~/provision-output` before downloading.

Wrong SSH key for the environment

Each environment has its own SSH key. Using the preview key to SSH into a production VM (or vice versa) will fail with `Permission denied (publickey)`. Double-check the key path matches the environment.

Accessory VMs have no externally exposed service ports

You cannot `psql -h <accessories.db.ip>` or `redis-cli -h <accessories.redis.ip>` from your local machine. Accessory VM firewalls only allow SSH (port 22). You must SSH in first, then use `docker exec` to reach the service.

Getting the repo name

The repo name used in SSH key paths and container labels is the GitHub repository name (without the owner prefix). To confirm:

```
gh repo view --json name -q .name
```

references/postgres-recipe.md

supabase/postgres Accessory Recipe

Recipe for deploying the `supabase/postgres` image — a Postgres container image enriched with several extensions, as recommended by the **tech-stack** skill. Every section documents a non-obvious behavior discovered through iteration. Follow this recipe exactly to avoid re-learning these lessons.

Table of Contents

Complete Accessory Recipe

The full accessory definition goes in the environment-specific config (`config/deploy.<env>.yml`):

```
  # config/deploy.preview.yml
  accessories:
    db:
      image: supabase/postgres:17.6.1.095
      host: <%= ENV['INFRA_DB_IP'] %>
      port: "5432:5432"
            cmd:  "postgres  -D  /etc/postgresql  -c  shared_buffers=1GB  -c  effective_cache_size=3GB  -c  work_mem=10MB  -c
maintenance_work_mem=256MB -c max_connections=100"
      env:
        secret:
          - POSTGRES_PASSWORD
      directories:
        - /data/pgdata:/var/lib/postgresql/data
```

The `.kamal/secrets.<destination>` file (agent-owned, one per environment):

```
  POSTGRES_PASSWORD=$POSTGRES_PASSWORD
  DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

The `cmd` values shown above are for a `medium` plan (4 GiB RAM). Use `generate_pg_cmd.py` to compute plan-specific tuning -- see Tuning with `generate_pg_cmd.py`.

The `-D /etc/postgresql` Flag

Standard postgres Docker images use `-D /var/lib/postgresql/data` for the config directory. The `supabase/postgres` image is different: its configuration lives at `/etc/postgresql` and the data directory is separate.

The `cmd` **must** include `-D /etc/postgresql`:

```
  postgres -D /etc/postgresql -c shared_buffers=1GB ...
```

Without this flag, postgres will look for configuration in the wrong path and fail to start.

Volume Mount: `/data/pgdata`, Not `/data/`

The host disk is mounted at `/data/` by the workflow. For accessories, always use Kamal `directories` (never named Docker volumes) — `/data/` is an attached disk with scheduled snapshot policies for disaster recovery, and only data under `/data/` is backed up. The ext4 filesystem also creates a `lost+found` directory at the mount root, so PostgreSQL's `initdb` fails if the data directory is not empty. For that reason, always bind-mount a **subdirectory**:

```
  accessories:
    db:
      directories:
        - /data/pgdata:/var/lib/postgresql/data   # CORRECT -- clean subdirectory
        # - /data:/var/lib/postgresql/data         # WRONG -- lost+found breaks initdb
```

Container Env: Only `POSTGRES_PASSWORD`

The `supabase/postgres` image only accepts `POSTGRES_PASSWORD` as a container environment variable. Do **not** pass `POSTGRES_USER` or `POSTGRES_DB` -- the image's internal init scripts bootstrap with user `postgres` and database `postgres` by default, and injecting these vars conflicts with that process.

```
  # CORRECT
  accessories:
    db:
      env:
        secret:
          - POSTGRES_PASSWORD

  # WRONG -- will conflict with supabase/postgres init
  accessories:
    db:
      env:
        clear:
          POSTGRES_USER: myuser      # DO NOT
          POSTGRES_DB: mydb          # DO NOT
        secret:
          - POSTGRES_PASSWORD
```

DATABASE_URL Derived from POSTGRES_PASSWORD

`DATABASE_URL` is **not** a separate GitHub Secret. It is derived from `POSTGRES_PASSWORD` directly in the `.kamal/secrets` file. See env-vars.md -- Database Connection Variables for how the app uses this variable:

```
  POSTGRES_PASSWORD=$POSTGRES_PASSWORD
  DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

Breakdown:
- **User**: always `postgres` (the image bootstraps this; see above)
- **Database**: always `postgres`
- **Host**: `db` -- matches the accessory name. CloudStack DNS resolves it to the accessory VM's internal (private) IP within the isolated network
- **Port**: `5432`

The hostname `db` is deterministic (it matches the accessory name and VM name). The only GitHub Secret the user must create is:

```
  POSTGRES_PASSWORD=<random password>
```

For non-preview environments, use the suffixed variable:

```
  POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
  DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
```

Tuning with `generate_pg_cmd.py`

The script at `scripts/generate_pg_cmd.py` computes plan-appropriate PostgreSQL tuning parameters. It encodes the plan-to-RAM mapping and produces a complete `postgres` command string.

Usage:

```
python3 scripts/generate_pg_cmd.py --plan <plan>
```

Example output for `--plan medium`:

```
postgres -D /etc/postgresql -c shared_buffers=1GB -c effective_cache_size=3GB -c work_mem=10MB -c maintenance_work_mem=256MB -c max_connections=100
```

The agent runs this script with the chosen plan and uses the output as the value for `accessories.db.cmd` in `config/deploy.<env>.yml`.

Tuning algorithm

The script computes five PostgreSQL parameters from the plan's RAM:

Parameter	Formula	Notes
`shared_buffers`	RAM / 4	Main memory cache for table/index data
`effective_cache_size`	RAM * 3 / 4	Planner hint for OS cache availability
`work_mem`	max(RAM / max_connections / 4, 2MB)	Per-operation sort/hash memory
`maintenance_work_mem`	min(RAM / 16, 2GB)	VACUUM, CREATE INDEX, etc.
`max_connections`	100 (<=4GB), 200 (<=16GB), 400 (>16GB)	Scaled by available memory

Plan-to-RAM Mapping Table

Each plan corresponds to a VM with a fixed amount of RAM (see scaling.md -- VM Plans for the full vCPU/memory table). The plan name determines how `generate_pg_cmd.py` tunes PostgreSQL:

Plan	RAM (MiB)
`micro`	1024
`small`	2048
`medium`	4096
`large`	8192
`xlarge`	16384
`2xlarge`	32768
`4xlarge`	65536

Sync with Workflow

The `accessories` workflow input is a JSON array. It must include an entry for the database accessory:

```
[{"name": "db", "plan": "<chosen-plan>", "disk_size_gb": <size>}]
```

- **`name`**: must be `db` (matches the accessory name in `config/deploy.<env>.yml`)
- **`plan`**: one of the plans from the table above. Determines the VM size and therefore the RAM available for PostgreSQL tuning
- **`disk_size_gb`**: size of the data disk attached to the database VM (range: 10–4000 GB). This is the disk mounted at `/data/` where PostgreSQL stores its data (via the `/data/pgdata` subdirectory)

Example for a medium plan with a 50 GB data disk:

```
[{"name": "db", "plan": "medium", "disk_size_gb": 50}]
```

references/recovery.md

Disaster Recovery

Table of Contents

Overview

Every deployed environment has **automatic daily snapshots** of all data volumes (web disk and accessory disks). Recovery uses these snapshots to recreate data volumes in a new deployment, restoring the application to the state captured by the most recent snapshot.

Recovery is triggered by running the same deploy workflow with `recover: true` . The provisioning script replaces blank disk creation with disk-from-snapshot creation; everything else (VMs, networks, IPs, firewall rules) is provisioned normally.

How Data Protection Works

The provisioning script creates a **daily snapshot policy** on every data volume (web disk and each accessory disk). These snapshots:

- Run at 06:00 UTC daily
- Keep the 3 most recent snapshots (older ones are automatically deleted)
- Are replicated across all available zones (both ZP01 and ZP02), so recovery can target either zone
- Are tagged with `locaweb-cloud-provision-id=<network_name>` for reliable lookup
- Are created as RECURRING type (policy-driven)

When an environment is torn down, snapshot policies are deleted but **existing snapshots are preserved**. This means snapshots remain available for recovery even after the original deployment no longer exists.

When to Use Recovery

- **After teardown**: You tore down an environment but need the data back.
- **VM or disk failure**: The deployment is lost but snapshots remain in the zone.
- **Data corruption**: Restore to a known-good state from a previous snapshot.

Recovery is **not** a substitute for application-level backups (e.g., `pg_dump`). It restores entire disk images, not individual rows or files. The restored data reflects the last snapshot time, not the moment of failure.

Pre-Flight Requirements

The provisioning script runs mandatory pre-flight checks before proceeding with recovery. All three must pass:

| Check | What it verifies | Why |

|-------|----------------|-----|
No existing network	No network named `{repo}-{id}-{env}` in the target zone	Prevents recovering over a live deployment
No existing volumes	No web or accessory data disks in the target zone	Same as above -- forces explicit teardown first
Snapshots exist	A snapshot in `BackedUp` state exists for every expected volume (web + each accessory)	Cannot recover without data to restore

If any check fails, the workflow exits with an error message explaining what's wrong.

Practical implication: If the original deployment still exists in the target zone, either tear it down first or change the `zone` input to recover in a different zone. Teardown deletes volumes but preserves snapshots; snapshot policies replicate across all available zones, so recovery works in either zone.

Recovery Procedure

Trigger the deploy workflow with `recover: true`. The `zone` input determines where the recovered deployment will be created. Snapshot policies replicate snapshots across all available zones, so snapshots are available in either zone regardless of where the original deployment lived.

The provisioning script's pre-flight checks will verify that no existing deployment conflicts with recovery and that snapshots are present. If anything is wrong, the workflow fails with a clear error -- there's no need to check manually beforehand.

Caller Recovery Workflow

Recovery uses the existing deploy workflow -- no separate workflow is needed. Simply add `recover: true` to the infra job inputs:

```
# Trigger via workflow_dispatch, or modify the existing workflow temporarily
jobs:
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      env_name: "preview"
      zone: "ZP01"                                          # Zone for the recovered deployment (either zone works)
      accessories: '[{"name":"db","plan":"medium","disk_size_gb":20}]'
      recover: true                                         # Enable disaster recovery
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}

  deploy:
    needs: infra
    # ... same deploy job as the normal workflow
```

All other inputs (`web_plan`, `web_disk_size_gb`, `workers_replicas`, etc.) work the same. The disk size inputs are ignored during recovery because disk sizes are determined by the snapshots.

Via GitHub Actions UI: If the workflow uses `workflow_dispatch`, check the "Recover from snapshots" checkbox in the GitHub Actions run dialog.

Via CLI:

```
gh workflow run deploy-preview.yml -f recover=true
```

Monitoring the Recovery Run

```
# Watch the run
gh run list --workflow=deploy-preview.yml --limit=5
gh run watch <run-id>
```

Give the user a direct link to the job in the GitHub UI:

```
    echo "$(gh repo view --json url -q .url)/actions/runs/<run-id>/job/$(gh run view <run-id> --json jobs -q '.jobs[0].databaseId')"
```

When `recover: true`, the workflow **skips infrastructure caching** -- it always runs the full provisioning script to ensure snapshots are properly restored.

What Happens Under the Hood

The recovery flow reuses the normal provisioning pipeline. Only the disk creation step differs:

```
Normal:   Network → VMs → IPs → Firewall → Blank disks → Snapshot policies
Recovery: Network → VMs → IPs → Firewall → Disks from snapshots → Snapshot policies
```

Step by step:

1. **Pre-flight checks** -- verifies no existing deployment and that snapshots exist (see Pre-Flight Requirements)
2. **Snapshot discovery** -- finds the most recent snapshot for each expected volume (`{network_name}-webdata`, `{network_name}-{accessory}data`) in the target zone. Both MANUAL and RECURRING snapshot types are considered; the most recent `BackedUp` snapshot wins.
3. **Normal provisioning** -- creates the network, VMs, public IPs, static NAT, and firewall rules exactly as a fresh deployment.
4. **Disk creation from snapshots** -- instead of creating blank disks, creates volumes from the discovered snapshots. Each volume is tagged with `locaweb-cloud-provision-id` and attached to its VM.
5. **Snapshot policies** -- new daily snapshot policies are created on the recovered volumes, maintaining the same data protection as a fresh deployment.
6. **Cloud-init compatibility** -- the VM userdata scripts check `blkid` before formatting disks. Since recovered volumes already have ext4 filesystems with data, formatting is skipped and data is preserved as-is.
7. **Kamal deploy** -- the deploy job runs `kamal setup` on the fresh infrastructure, deploying the application. The app starts with the recovered data volumes mounted.

Post-Recovery Verification

After the workflow succeeds:

1. Health check

```
curl -s -o /dev/null -w "%{http_code}" https://<web_ip>.nip.io/up
```

2. Verify recovered data

SSH into the VMs and check that data is present. See operations.md for SSH access and container debugging commands.

For a database accessory:

```
# SSH into the database VM
ssh -i ~/.ssh/<repo-name> root@<accessories.db.ip>

# Check the data
docker exec -it <repo-name>-db psql -U postgres -c "SELECT count(*) FROM <table>;"
```

For the web disk:

```
# SSH into the web VM
ssh -i ~/.ssh/<repo-name> root@<web_ip>

# Check uploaded files exist
ls -la /data/uploads/    # or whatever subdirectory the app uses
```

3. Verify snapshot policies

New snapshot policies are created automatically on recovered volumes. No manual action needed -- the recovered deployment has the same daily snapshot protection as a fresh one.

4. Remove the `recover: true` flag

Once verification is complete, **remove `recover: true` ** from the workflow file (or uncheck the checkbox for `workflow_dispatch` triggers). If `recover: true` is left in place and the workflow runs again, the pre-flight checks will **fail hard** because the network and volumes created by the recovery already exist in the target zone. Removing the flag ensures subsequent workflow runs follow the normal deploy path (which skips provisioning when infrastructure is already cached).

Current Limitations

- **No existing deployment in target zone**: If the deployment still exists in the target zone, you must tear it down first. The pre-flight checks enforce this to prevent data loss. Recovery can target either zone -- snapshot policies replicate across all available zones.
- **Recovery point is the last snapshot**: Data written after the most recent snapshot is lost. The daily schedule means up to ~24 hours of data loss in the worst case (RPO). For lower RPO, create manual snapshots before planned operations.
- **Disk sizes match the snapshot**: The `web_disk_size_gb` and accessory `disk_size_gb` inputs are ignored during recovery -- the recovered volume inherits the size of the source snapshot's volume.
- **All volumes must have snapshots**: Recovery requires a snapshot for every expected volume (web + each accessory). If any snapshot is missing, recovery fails at pre-flight.

Creating Manual Snapshots

Daily snapshots run at 06:00 UTC. To create an immediate snapshot before a risky operation or planned teardown, use the CloudStack dashboard or API directly. Manual snapshots are also considered during recovery (both MANUAL and RECURRING types are searched).

The provisioning script does not provide a built-in manual snapshot command -- use the CloudStack UI at painel-cloud.locaweb.com.br to create snapshots from the Volumes section.

references/scaling.md

Scaling Guide

Table of Contents

VM Plans

Available plans (same options for `web_plan`, `workers_plan`, and accessory plans):

Plan	vCPUs	Memory (GiB)
`micro`	1	1
`small`	1	2
`medium`	2	4
`large`	2	8
`xlarge`	4	16
`2xlarge`	8	32
`4xlarge`	16	64

Choosing the Right Plan

Plan selection should consider three factors:

1. Runtime footprint

Different language runtimes (e.g., Go, Python, Node.js, Java) have varying baseline CPU and memory requirements. The specific framework, number of server workers, and application complexity also affect the resource footprint. Choose a plan that matches the actual resource consumption of your application, taking into account the target environment as well (for example, preview environments typically require fewer resources than production).

2. Accessory sizing

Each accessory (database, redis, etc.) has its own `plan` inside the `accessories` JSON input. Consider the expected workload for each:

- **Database (db)** (see postgres-recipe.md -- Tuning with generate_pg_cmd.py for plan-specific PostgreSQL tuning):
  - Small datasets (<1 GB), simple queries: `small` or `medium`

  - Medium datasets (1-10 GB), moderate queries: `medium` or `large`
  - Large datasets (>10 GB), complex queries or many connections: `large` or above
  - PostgreSQL benefits from available memory for OS page cache
- **Redis**: Typically `small` or `medium` is sufficient unless caching large datasets
- **Other accessories**: Size based on the specific service's requirements

3. Environment purpose

- **Preview/dev**: Use smaller plans (`micro` , `small`). These environments are for testing, not production traffic.
- **Production**: Size for expected load. Start with `medium` and scale up if needed.
- **Workers**: Depends on workload -- CPU-bound tasks (ML inference) need larger plans; I/O-bound tasks (queue processing) can use smaller plans.

Scaling the Web Tier

The web tier is **vertical only** (single VM). Increase `web_plan` for more capacity:

```
  with:
    web_plan: "medium"  # upgrade from "small"
```

Vertical scaling causes a restart with brief downtime. The VM is stopped, resized, and started again. Plan scaling during low-traffic windows when possible.

Scaling Workers

Workers scale **horizontally** by changing `workers_replicas` :

```
  with:
    workers_replicas: 3    # scale from 1 to 3 (0 = no workers)
    workers_plan: "small"
```

- `workers_replicas: 0` means no workers (disabled).
- Set to 1 or more to enable workers.

Scale up: re-deploy with a higher `workers_replicas` . New VMs are provisioned and containers deployed.

Scale down: re-deploy with a lower `workers_replicas` . Excess worker VMs are destroyed along with their public IPs, firewall rules, and static NAT mappings.

Workers can also scale vertically via `workers_plan` . **Vertical scaling of workers causes a restart with brief downtime** on the affected VMs.

Updating the host list when changing replicas

When changing `workers_replicas` , the host list in `config/deploy.<env>.yml` must match the new count. The provision job exports `INFRA_WORKER_IP_0` , `INFRA_WORKER_IP_1` , etc. — one per replica. Add or remove ERB entries accordingly:

```
  # config/deploy.preview.yml — example for 3 replicas
  servers:
    workers:
      hosts:
        - <%= ENV['INFRA_WORKER_IP_0'] %>
        - <%= ENV['INFRA_WORKER_IP_1'] %>
        - <%= ENV['INFRA_WORKER_IP_2'] %>
```

If the host list has fewer entries than `workers_replicas` , the extra worker VMs will be provisioned but Kamal will not deploy containers to them. If the host list has more entries than `workers_replicas` , the extra entries will resolve to empty strings and Kamal will fail.

Scaling Accessories

Each accessory scales **vertically only** (single VM per accessory). Change the `plan` inside the `accessories` JSON:

```
  with:
    accessories: '[{"name":"db","plan":"large","disk_size_gb":100},{"name":"redis","plan":"medium","disk_size_gb":10}]'
```

Vertical scaling causes a restart with brief downtime. The accessory container will restart after the VM is resized. Ensure the application handles transient disconnections gracefully (e.g., database reconnection logic).

To add a new accessory, add an entry to the `accessories` JSON array and re-deploy. To remove an accessory, remove its entry from the JSON array -- the teardown of the removed accessory's resources happens automatically.

Scaling the Database

When scaling a `supabase/postgres` database accessory, the VM plan change must be accompanied by updated PostgreSQL tuning parameters. The `cmd` in `config/deploy.<env>.yml` encodes memory-dependent settings (`shared_buffers` , `effective_cache_size` , etc.) that must match the new plan's RAM.

Procedure:

1. **Regenerate the `cmd` string** using `generate_pg_cmd.py` with the new plan:

```
    python3 plugins/cofounder/skills/app-deploy/scripts/generate_pg_cmd.py --plan <new_plan>
```

Example output for `--plan large` :

```
            postgres -D /etc/postgresql -c shared_buffers=2GB -c effective_cache_size=6GB -c work_mem=10MB -c
maintenance_work_mem=512MB -c max_connections=200
```

2. **Update `accessories.db.cmd` ** in `config/deploy.<env>.yml` with the output from step 1.

3. **Update the `plan` ** in the `accessories` JSON in the deploy workflow (`.github/workflows/deploy-<env>.yml`):

```
    accessories: '[{"name": "db", "plan": "large", "disk_size_gb": 50}]'
```

4. **Commit, push, and redeploy.** The provision job resizes the VM; Kamal reboots the accessory with the new `cmd` .

If the `cmd` is not updated to match the new plan, PostgreSQL will run with tuning parameters sized for the old VM — either wasting memory (scaled up) or risking OOM (scaled down).

See postgres-recipe.md -- Tuning with generate_pg_cmd.py for the tuning algorithm and plan-to-RAM mapping.

Other accessories with memory-dependent configuration

The database procedure above is a specific instance of a general rule: **when scaling any accessory, review whether its container configuration has parameters tied to available memory or CPU**. Examples:

- **Redis**: `maxmemory` (typically 50-75% of RAM)
- **Elasticsearch / OpenSearch**: JVM heap `-Xms` , `-Xmx` , typically 50% of RAM, max 32GB)
- **Any service with a `cmd` or `env` containing memory/CPU limits**

Update these values in `config/deploy.<env>.yml` (in the accessory's `cmd` or `env.clear` block) to match the new plan before redeploying.

Disk Sizes

Disk	Input	Default	Attached to
Web disk	`web_disk_size_gb`	20 GB	Web VM at `/data`
Accessory disks	`accessories` JSON `disk_size_gb`	20 GB	Each accessory VM at `/data`

Both web and accessory disks are mounted at `/data/` on their respective VMs. For main app roles (web, workers), use Kamal `volumes` ; for accessories, use `directories` . Never use named Docker volumes. Always map to a **subdirectory** of `/data/` (e.g., `/data/uploads` , `/data/pgdata`), never `/data/` root. `/data/` is an attached disk with scheduled snapshot policies for disaster recovery — data outside `/data/` is not backed up. Additionally, the ext4 filesystem

creates `lost+found` at the mount root, which breaks containers that expect a clean directory. See postgres-recipe.md -- Volume Mount for the database example.

All disk sizes must be in the range **10–4000 GB**. Disks can be **grown** by re-deploying with a larger value. **Shrinking is not supported** -- the workflow will fail with an error if you specify a smaller size than the current disk.

Choosing disk sizes

- **Preview environments**: 20 GB default is usually sufficient
- **Production web disk**: Size based on expected upload volume. If the app stores user uploads, media files, etc., plan for growth.
- **Production database disk**: Size based on expected data volume plus headroom for WAL, temporary files, and vacuuming. A good rule of thumb is 2-3x the expected data size.
- **Redis / other accessories**: Typically small disks are sufficient unless persisting large datasets.

Example -- production with larger disks:

```
with:
  web_disk_size_gb: 50
  accessories: '[{"name":"db","plan":"medium","disk_size_gb":100},{"name":"redis","plan":"small","disk_size_gb":10}]'
```

Workers are stateless and do not have data disks.

How Scaling Works

Scaling happens through the normal deploy workflow -- there is no separate "scale" action. Re-run the deploy workflow with updated inputs (see workflows.md -- Deploy Input Reference for all available inputs):

1. The provisioning script detects existing resources by name
2. For VMs: compares current service offering to desired; if different, stops the VM, scales it, and starts it again (expect brief downtime)
3. For disks: compares current size to desired; grows in-place if larger
4. For workers: creates new VMs if `workers_replicas` increased; destroys excess VMs if decreased
5. For accessories: adds new accessory VMs if new keys appear in the JSON; removes VMs if keys are removed
6. Kamal redeploys containers on all hosts (existing and new)

This means scaling and code deployment happen together in a single workflow run.

references/setup-and-deploy.md

Setup and Deploy Procedures

All setup steps are idempotent -- safe to re-run across agent sessions. Check for existing resources before creating new ones.

Table of Contents

GitHub Repository (Prerequisite)

Verify a git remote is configured:

```
git remote -v
```

If `origin` is not set, use the **repo-setup** skill to initialize the repository before continuing.

SSH Key Generation

See also operations.md -- SSH Access for using these keys to connect to VMs after deployment.

Preview environment key

Check if an SSH key already exists for this repo:

```
test -f ~/.ssh/<repo-name> && echo "Key exists" || echo "Key missing"
```

If the key does not exist, generate a new Ed25519 SSH key locally:

```
ssh-keygen -t ed25519 -f ~/.ssh/<repo-name> -N "" -C "<repo-name>-deploy"
chmod 600 ~/.ssh/<repo-name>
```

If the key already exists, reuse it -- do not overwrite.

This key will be:
- Stored as the `SSH_PRIVATE_KEY` GitHub secret (the private key)
- Used locally to SSH into preview environment VMs for debugging
- The public key is derived automatically by the deploy workflow at runtime

Additional environment keys

Generate a separate key for each additional environment (Step 8). For example, for the "production" environment:

```
test -f ~/.ssh/<repo-name>-production && echo "Key exists" || echo "Key missing"
```

```
ssh-keygen -t ed25519 -f ~/.ssh/<repo-name>-production -N "" -C "<repo-name>-deploy-production"
chmod 600 ~/.ssh/<repo-name>-production
```

This key will be:
- Stored as a suffixed GitHub secret matching the environment name: e.g., `SSH_PRIVATE_KEY_PRODUCTION`
- Used locally to SSH into that environment's VMs for debugging
- The caller workflow maps the suffixed secret to the workflow's standard `SSH_PRIVATE_KEY` input

CloudStack Credentials

First check if CloudStack secrets are already set in the repo:

```
gh secret list
```

If `CLOUDSTACK_API_KEY` and `CLOUDSTACK_SECRET_KEY` appear in the list, skip this step.

Otherwise, ask the user to set them via the GitHub UI as described in the Secrets the user must set via GitHub UI section.

If the user doesn't have a Locaweb Cloud account yet, recommend they go to locaweb.com.br/locaweb-cloud and look for the **"Contratar"** button to sign up. Once they have an account they can find their API keys in the CloudStack dashboard.

Database Credentials

See the `supabase/postgres` recipe for the full accessory configuration. See env-vars.md -- Database Connection Variables for how the app uses `DATABASE_URL`.

Check if the database secrets are already set in the repo:

```
gh secret list
```

If `POSTGRES_PASSWORD` already appears, skip this step.

Generate the Postgres password

Generate a random password for **each** environment:

```
# Preview password
python -c "import secrets; print(secrets.token_urlsafe(32))"

# Production password (different from preview)
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

DATABASE_URL

`DATABASE_URL` is **not** a separate GitHub Secret. It is derived from `POSTGRES_PASSWORD` in the `.kamal/secrets` file:

```
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

The hostname `db` resolves via CloudStack internal DNS to the database VM's private IP. User is always `postgres`, database is always `postgres`.

The default preview environment uses unsuffixed names: `POSTGRES_PASSWORD`.

Additional environments use suffixed names matching the environment name:
- `POSTGRES_PASSWORD_PRODUCTION` for the "production" environment
- The `.kamal/secrets.production` file derives `DATABASE_URL` from the suffixed variable:
`DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres`

Creating GitHub Secrets

First list existing secrets to avoid overwriting them:

```
gh secret list
```

Only create secrets that are **not already present**.

Security rule: Never accept secret values through the chat -- they would be stored in conversation history. For secrets the agent knows (generated passwords, local SSH keys), the agent can set them directly. For secrets only the user knows (CloudStack keys, app API keys), ask the user to set them via the GitHub UI (see Secrets the user must set via GitHub UI).

Secrets the agent can set directly

Infrastructure and database secrets per environment:

```
# SSH private key for preview (skip if already set)
gh secret set SSH_PRIVATE_KEY < ~/.ssh/<repo-name>

# SSH private key for additional environments, e.g. production (skip if already set)
gh secret set SSH_PRIVATE_KEY_PRODUCTION < ~/.ssh/<repo-name>-production

# Postgres password for preview (skip if already set)
gh secret set POSTGRES_PASSWORD --body "<generated password>"

# Postgres password for additional environments, e.g. production (skip if already set)
gh secret set POSTGRES_PASSWORD_PRODUCTION --body "<generated password>"
```

Note: `DATABASE_URL` does not need a GitHub Secret -- it is derived from `POSTGRES_PASSWORD` in the `.kamal/secrets` file.

Secrets the user must set via GitHub UI

Get the repository's secrets settings URL:

```
echo "$(gh repo view --json url -q .url)/settings/secrets/actions"
```

Give the user the URL and ask them to click **"New repository secret"** for each secret below. For each secret, tell the user:
1. The exact **Name** to enter.
2. Where to find the **Secret** value (e.g., which dashboard, settings page, or credential file to look in).

CloudStack credentials (if not already set):

Name	Where to find the value
`CLOUDSTACK_API_KEY`	painel-cloud.locaweb.com.br -> Contas -> *(sua conta)* -> Visualizar usuarios -> *(seu usuario)* -> Copiar Chave da API
`CLOUDSTACK_SECRET_KEY`	Same page -> Copiar Chave secreta

> **Note:** Warn that the keys may take some time to show up after page has loaded. If the user has never generated keys before, they should click the **"Gerar novas chaves"** icon on the top right corner of the user page.

App-specific secrets -- store each one **individually**:

Name	Where to find the value
`API_KEY`	*(describe where the user can find this value)*
`SMTP_PASSWORD`	*(describe where the user can find this value)*

After adding secrets, map them in the caller workflow's deploy job `env:` block:

```
# In deploy job
env:
  POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
```

```
    API_KEY: ${{ secrets.API_KEY }}
    SMTP_PASSWORD: ${{ secrets.SMTP_PASSWORD }}
```

This way, updating a single secret only requires creating/updating it in the GitHub UI -- no need to remember or rewrite the others.

Verify all secrets are set:

```
  gh secret list
```

Deploy Configuration Files

The agent creates the following files as part of setup. See env-vars.md for the full environment variables and secrets configuration reference.

deploy.yml

The Kamal deploy configuration. Contains service name, image, server hosts, environment variables, accessories, and other deployment settings. The agent generates this based on the application's requirements.

.kamal/secrets.\<destination\>

One secrets file per destination. Each maps secret environment variable names so Kamal can read them from the deploy environment:

```
  # .kamal/secrets.preview (unsuffixed — env var names match GitHub Secret names)
  POSTGRES_PASSWORD=$POSTGRES_PASSWORD
  DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
  API_KEY=$API_KEY
```

```
  # .kamal/secrets.production (suffixed — right side matches suffixed GitHub Secret names)
  POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
  DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
  API_KEY=$API_KEY_PRODUCTION
  STRIPE_LIVE_KEY=$STRIPE_LIVE_KEY_PRODUCTION
```

Since all deploys use `-d <destination>`, Kamal looks for `.kamal/secrets-common` and `.kamal/secrets.<destination>` — **not** `.kamal/secrets`. We use only per-destination files so each is a self-contained, verifiable manifest of what that environment needs.

Deploy Cycle

After committing workflows and pushing:

1. Share workflow link and monitor

Before starting to monitor, give the user a prominent clickable link to the workflow run:

```
  gh run list --limit=1 --json databaseId,url -q '.[0].url'
```

Present it prominently so the user can follow along in the GitHub UI:

> **Your deploy is running! Follow it live here:**
>
> **\<URL from the command above\>**

Then monitor the run:

```
  # Watch the latest run
  gh run watch

  # Or list runs and watch a specific one
  gh run list --limit=5
```

```
gh run watch <run-id>
```

2. If the workflow fails

Read the error:

```
# View the failed run's logs
gh run view <run-id> --log-failed
```

Common failure causes:
- **Missing secrets**: `gh secret list` to verify all required secrets exist
- **Dockerfile issues**: Build failures, wrong port, missing health check
- **Permission errors**: Ensure `permissions: contents: read, packages: write` in the caller workflow
- **Input errors**: Invalid zone, plan name, or type mismatches

Fix the issue, commit, and push. The preview workflow (triggered on push) will start a new run automatically.

3. Repeat until successful

Continue the cycle: read error -> fix -> commit/push -> watch run. Do not give up after one failure -- iterate.

4. On success, extract deployment info

```
# Get the web IP from the latest successful run
gh run view <run-id>

# Or download the provision-output artifact (clean first to avoid stale data)
rm -rf ~/provision-output
gh run download <run-id> --name provision-output --dir ~/provision-output
cat ~/provision-output/provision-output.json
```

The app URL is `https://<web_ip>.nip.io` (TLS works with nip.io).

App Verification (Health Check)

After the workflow succeeds, verify the application is healthy:

1. Run the health check

```
# Health check -- must return HTTP 200
curl -s -o /dev/null -w "%{http_code}" https://<web_ip>.nip.io/up
```

If HTTP 200 is returned, the deployment is verified and the Deployment Feedback Loop is complete.

2. If the health check fails

SSH into the VMs to inspect logs and container state. Use the SSH key generated earlier and the public IPs from the workflow output.

See SSH Debugging below for detailed commands.

After gathering diagnostic information, return to the **tech-stack** skill's Local Development Feedback Loop to diagnose and fix the issue locally before pushing again.

SSH Debugging

Use the locally saved SSH key and the public IPs from the workflow output to connect to VMs. Use the correct key for the environment: `~/.ssh/<repo-name>` for preview, `~/.ssh/<repo-name>-<env_name>` for other environments.

```
# Preview
ssh -i ~/.ssh/<repo-name> root@<web_ip>
```

```
ssh -i ~/.ssh/<repo-name> root@<accessory_ip>
ssh -i ~/.ssh/<repo-name> root@<worker_ip>

# Production (or other environment -- use the corresponding key)
ssh -i ~/.ssh/<repo-name>-production root@<web_ip>
ssh -i ~/.ssh/<repo-name>-production root@<accessory_ip>
ssh -i ~/.ssh/<repo-name>-production root@<worker_ip>
```

Useful debug commands on the VMs

```
# List running containers
docker ps

# View web app logs
docker logs $(docker ps -q --filter "label=service=<repo-name>") --tail 100

# Follow logs in real time
docker logs $(docker ps -q --filter "label=service=<repo-name>") -f

# Check if the app responds locally on port 80
curl -s localhost:80/up

# View kamal-proxy logs (web VM only)
docker logs kamal-proxy --tail 50

# Check Postgres container logs (accessory VM)
docker logs <repo-name>-db --tail 100

# Check disk mounts
df -h /data    # web VM or accessory VM

# Check container environment variables
docker exec $(docker ps -q --filter "label=service=<repo-name>") env
```

references/teardown.md

Teardown Guide

Table of Contents

Teardown Workflow

The teardown workflow requires only two inputs:

Input	Type	Required	Description
`env_name`	string	no (default: `preview`)	Environment name to tear down
`zone`	string	yes	CloudStack zone where the environment is deployed

And two secrets:

Secret	Required
`CLOUDSTACK_API_KEY`	yes
`CLOUDSTACK_SECRET_KEY`	yes

Caller Teardown Workflow

```
  # .github/workflows/teardown-preview.yml
  name: Teardown Preview
  on:
    workflow_dispatch:

  jobs:
    teardown:
      uses: gmautner/locaweb-cloud-provision/.github/workflows/teardown.yml@v1
      with:
        env_name: "preview"
        zone: "ZP01"
      secrets:
        CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
        CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
```

Create one teardown workflow per environment. For example:
- `teardown-preview.yml` with `env_name: "preview"`, `zone: "ZP01"`
- `teardown-production.yml` with `env_name: "production"`, `zone: "ZP01"`

Monitoring the teardown run

```
  gh run list --workflow=teardown-preview.yml --limit=5
  gh run watch <run-id>
```

Give the user a direct link to the job in the GitHub UI:

```
    echo "$(gh repo view --json url -q .url)/actions/runs/<run-id>/job/$(gh run view <run-id> --json jobs -q
'.jobs[0].databaseId')"
```

Inferring Zone and env_name

When tearing down an environment and the zone/env_name are not known, infer them from the existing deploy workflow:

1. **Read the committed deploy workflow** (e.g., `.github/workflows/deploy-preview.yml`):
   - `env_name` is in the `with:` block (e.g., `env_name: "preview"`)
   - `zone` is in the `with:` block (e.g., `zone: "ZP01"`)

2. **Confirm by checking the last run** using the GitHub CLI:

```
# List recent runs of the deploy workflow
gh run list --workflow=deploy-preview.yml --limit=5

# View a specific run's details (includes inputs)
gh run view <run-id>

# Download the provision-output artifact to see IPs (clean first to avoid stale data)
rm -rf ~/provision-output
gh run download <run-id> --name provision-output --dir ~/provision-output
cat ~/provision-output/provision-output.json
```

3. **Decode provision-output.json** -- the artifact contains:

```
{
  "web_ip": "200.234.x.x",
  "worker_ips": ["200.234.y.y"],
  "accessories": {
    "db": {
      "ip": "200.234.z.z",
      "internal_ip": "10.1.1.x"
    }
  }
}
```

These IPs confirm which deployment is active and help verify the environment before teardown.

Reading Last Run Outputs

To retrieve deployment information from a previous run (see also operations.md -- Finding Deployment IPs):

```
# Find the latest successful deploy run
gh run list --workflow=deploy-preview.yml --status=success --limit=1

# Get the run ID and download its artifact (clean first to avoid stale data)
rm -rf /tmp/output
gh run download <run-id> --name provision-output --dir /tmp/output
cat /tmp/output/provision-output.json
```

The step summary (visible in the GitHub UI) also shows:
- Infrastructure table: resource type and public IP for each VM
- Deployment summary: commit SHA, image tag, app URL, health check URL
- If a domain was configured: the domain name and the IP to point DNS at

What Teardown Destroys

The teardown script destroys resources in reverse creation order:

1. **Snapshot policies** on all data volumes (the policies are deleted, but **existing snapshots are preserved** -- they remain available for disaster recovery)
2. **Data volumes** (web disk, accessory disks) -- detached then deleted
3. **Static NAT mappings** on all public IPs
4. **Firewall rules** on all public IPs
5. **Public IPs** (released)
6. **All VMs** (web, workers, accessories -- destroyed with expunge)

7. **The isolated network** (after 5s wait for VM expunge)
8. **The SSH key pair**

All `cmk` failures during teardown are treated as non-fatal warnings (resources may already be partially deleted).

Important Notes

- **zone must match**: The teardown `zone` must match the zone where the environment was deployed. Resources in the wrong zone will not be found.
- **env_name must match**: The teardown `env_name` must match the deploy `env_name` exactly. The resource naming pattern is `{repo-name}-{repository-id}-{env_name}` .
- **Data disks are permanently deleted**: Teardown deletes data volumes (web and accessory disks). However, **snapshots taken before teardown are preserved** and remain in the CloudStack account. These snapshots can be used for disaster recovery.
- **Safe to re-run**: If some resources are already deleted (e.g., partial teardown), the script continues without failing.
- **Shared concurrency group**: Teardown shares the `infra-{repository}-{env_name}` concurrency group with the deploy workflow, so a teardown cannot run while a deploy is in progress (and vice versa).

references/workflows.md

Caller Workflow Reference

Table of Contents

Two-Job Pattern

Caller workflows use two jobs: **provision** (infrastructure provisioning) and **deploy** (application deployment with Kamal). The provision job calls the reusable `provision.yml@v1` workflow, which provisions VMs, networks, disks, and firewall rules. The deploy job runs Kamal to build, push, and deploy the Docker image.

This separation means:
- The provision job handles only infrastructure and outputs everything the deploy job needs
- The deploy job owns the Kamal lifecycle (install, SSH key, Docker cache, `kamal setup` `kamal deploy`)
- Application secrets flow directly from GitHub Secrets to the deploy job's `env:` block

Preview Environment (Default)

The default preview environment is triggered on push, immediately reflecting changes to the main branch -- matching a typical developer workflow. If no domain is provided, it uses nip.io for immediate access with TLS. Since `"preview"` is the default `env_name` , secrets use unsuffixed names.

> **Note:** Empty commits `git commit --allow-empty`) do not trigger `on: push` workflows. A push must include at least one file change for GitHub Actions to run.

Preview Workflow Example

```
# .github/workflows/deploy-preview.yml
name: Deploy Preview
on:
  push:
    branches: [main]
    paths-ignore: [".claude/**"]

permissions:
  contents: read
  packages: write

jobs:
```

```
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      env_name: "preview"
      zone: "ZP01"
      accessories: '[{"name":"db","plan":"medium","disk_size_gb":20}]'
      # workers_replicas: 2                  # Optional, default: 0 (0 = no workers)
      # workers_plan: "small"                # Optional, default: "small"
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}

  deploy:
    needs: infra
    runs-on: ubuntu-latest
    env:
      POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
    steps:
      - uses: actions/checkout@v4

      - name: Load infrastructure environment
        run: echo "${{ needs.infra.outputs.infra_env }}" >> "$GITHUB_ENV"

      - name: Set repo identity
        run: |
          echo "REPO_NAME=${{ github.event.repository.name }}" >> "$GITHUB_ENV"
          echo "REPO_FULL=${{ github.repository }}" >> "$GITHUB_ENV"
          echo "REPO_OWNER=${{ github.repository_owner }}" >> "$GITHUB_ENV"

      - uses: webfactory/ssh-agent@v0.9.0
        with:
          ssh-private-key: ${{ secrets.SSH_PRIVATE_KEY }}

      # Expose GHA cache env vars for Kamal's `docker buildx build --cache-from type=gha`.
      # Recommended by https://docs.docker.com/build/cache/backends/gha/
      - uses: crazy-max/ghaction-github-runtime@v3

      - uses: ruby/setup-ruby@v1
        with:
          ruby-version: "3.4"

      - run: gem install kamal --no-document

      - name: Deploy with Kamal
        env:
          KAMAL_REGISTRY_PASSWORD: ${{ secrets.GITHUB_TOKEN }}
        run: |
          kamal setup -d preview
          kamal accessory reboot all -d preview

      # Image is cached via GHA, not the registry -- clean up to prevent storage accumulation
      - uses: actions/delete-package-versions@v5
        with:
          package-name: ${{ github.event.repository.name }}
          package-type: container
          min-versions-to-keep: 1
```

After this runs successfully, the app is accessible at `https://<web_ip>.nip.io` . The `web_ip` is visible in the workflow run summary.

Additional Environments

Other environments can be created depending on your processes, changing the triggers and workflow inputs as needed. Each `env_name` creates fully isolated infrastructure.

Secret naming convention

Secrets flow directly from GitHub Secrets to the deploy job's `env:` block (see env-vars.md -- Secret Variables for the

full secrets configuration reference). The naming convention determines the GitHub Secret name and the environment variable name on the runner:

Preview (default environment) -- unsuffixed names:
- GitHub Secret: `POSTGRES_PASSWORD` → env var: `POSTGRES_PASSWORD`
- `.kamal/secrets.preview` : `POSTGRES_PASSWORD=$POSTGRES_PASSWORD`

Non-preview (e.g., production) -- suffixed names:
- GitHub Secret: `POSTGRES_PASSWORD_PRODUCTION` → env var: `POSTGRES_PASSWORD_PRODUCTION`
- `.kamal/secrets.production` : `POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION`

The env var on the runner matches the GitHub Secret name exactly. The `.kamal/secrets.<dest>` file maps the Kamal secret name (left side) to the env var name (right side).

Infrastructure secrets scoped to a specific environment also use the suffix:
- `SSH_PRIVATE_KEY` (preview), `SSH_PRIVATE_KEY_PRODUCTION` (production)

Secrets **common to all environments** (e.g., `CLOUDSTACK_API_KEY`, `CLOUDSTACK_SECRET_KEY`) don't need suffixes -- just pass them in every caller workflow.

Production environment example

A recommended additional environment is **"production"**, triggered on version tags (`v*`). A tag signals that the pointed commit is ready for production.

```
# .github/workflows/deploy-production.yml
name: Deploy Production
on:
  push:
    tags: ["v*"]

permissions:
  contents: read
  packages: write

jobs:
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      env_name: "production"
      zone: "ZP01"
      web_plan: "medium"
      web_disk_size_gb: 50
      accessories: '[{"name":"db","plan":"medium","disk_size_gb":50}]'
      # workers_replicas: 2                  # Optional, default: 0 (0 = no workers)
      # workers_plan: "small"                # Optional, default: "small"
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY_PRODUCTION }}

  deploy:
    needs: infra
    runs-on: ubuntu-latest
    env:
      POSTGRES_PASSWORD_PRODUCTION: ${{ secrets.POSTGRES_PASSWORD_PRODUCTION }}
    steps:
      - uses: actions/checkout@v4

      - name: Load infrastructure environment
        run: echo "${{ needs.infra.outputs.infra_env }}" >> "$GITHUB_ENV"

      - name: Set repo identity
        run: |
          echo "REPO_NAME=${{ github.event.repository.name }}" >> "$GITHUB_ENV"
          echo "REPO_FULL=${{ github.repository }}" >> "$GITHUB_ENV"
          echo "REPO_OWNER=${{ github.repository_owner }}" >> "$GITHUB_ENV"

      - uses: webfactory/ssh-agent@v0.9.0
```

```
          with:
            ssh-private-key: ${{ secrets.SSH_PRIVATE_KEY_PRODUCTION }}

        # Expose GHA cache env vars for Kamal's `docker buildx build --cache-from type=gha`.
        # Recommended by https://docs.docker.com/build/cache/backends/gha/
        - uses: crazy-max/ghaction-github-runtime@v3

        - uses: ruby/setup-ruby@v1
          with:
            ruby-version: "3.4"

        - run: gem install kamal --no-document

        - name: Deploy with Kamal
          env:
            KAMAL_REGISTRY_PASSWORD: ${{ secrets.GITHUB_TOKEN }}
          run: |
            kamal setup -d production
            kamal accessory reboot all -d production

        # Image is cached via GHA, not the registry -- clean up to prevent storage accumulation
        - uses: actions/delete-package-versions@v5
          with:
            package-name: ${{ github.event.repository.name }}
            package-type: container
            min-versions-to-keep: 1
```

To deploy to production: `git tag v1.0.0 && git push --tags` . The workflow checks out the tagged commit, so the Dockerfile and source code match the tag exactly.

Deploy Input Reference

All inputs passed to the `infra` job (the reusable `provision.yml@v1` workflow):

Input	Type	Default	When to set
`env_name`	string	`"preview"`	Name of the environment. Each env_name creates fully isolated infrastructure. Defaults to `"preview"` if omitted. **Must use only lowercase letters, digits, and underscores `[a-z0-9_]`).**
`zone`	string	`"ZP01"`	CloudStack zone. Usually leave as default. Use `ZP02` for geographic redundancy.
`web_plan`	string	`"small"`	Choose based on runtime footprint and environment. See scaling.md -- VM Plans for plan specs.
`web_disk_size_gb`	number	`20`	Persistent disk attached to the web VM at `/data` . Range: 10–4000 GB. Increase if the app stores files (uploads, media). Can only grow, never shrink.
`accessories`	string (JSON)	`"[]"`	JSON array of accessory VMs: `[{"name": "db", "plan": "medium", "disk_size_gb": 20}]` . * `disk_size_gb` is mandatory **for every accessory (range: 10–4000 GB), even if the accessory does not need persistent storage.** Accessory `name` must use only lowercase letters, digits, and underscores `[a-z0-9_]`).** Each object supports an optional `ports` field (comma-separated string, e.g. `"5432"` or `"80,443"`) to open additional firewall ports; port 22 (SSH) is always included.
`workers_replicas`	number	`0`	Number of worker VMs. `0` means no workers. Set to 1 or more to enable background processing.
`workers_plan`	string	`"small"`	VM size for workers. Choose based on worker workload intensity. See scaling.md -- Scaling Workers.
`automatic_reboot`	boolean	`true`	Enable automatic reboot after unattended security upgrades. Usually leave as default.
`automatic_reboot_time_utc`	string	`"05:00"`	When automatic reboots happen. Usually leave as default.
`recover`	boolean	`false`	Enable disaster recovery from snapshots. See recovery.md for the full procedure.

Inputs to leave at defaults

For most deployments, omit these (let defaults apply):
- `automatic_reboot` / `automatic_reboot_time_utc` -- security auto-updates are good defaults
- `recover` -- only set to `true` for disaster recovery (see recovery.md)

- `web_disk_size_gb` -- 20 GB is sufficient for most apps unless heavy file storage

Deploy Output Reference

The infra job exposes outputs consumed by the deploy job and optionally by downstream jobs:

Output	Type	Description
`web_ip`	string	Public IP of the web VM
`worker_ips`	string (JSON array)	Public IPs of worker VMs (e.g., `["1.2.3.4","5.6.7.8"]`)
`accessory_ips`	string (JSON object)	Accessory public IPs keyed by name (e.g., `{"db":"200.234.x.x","redis":"200.234.y.y"}`)
`infrastructure_changed`	string	`"true"` when fresh provision (cache miss), `"false"` when cached. Informational — the deploy job always runs `kamal setup` (idempotent).
`scaled_accessories`	string (JSON array)	Names of accessories whose VMs were rescaled (e.g., `["db"]`). Informational — the deploy job always reboots all accessories.
`infra_env`	string (multiline)	`KEY=VALUE` pairs for `GITHUB_ENV` (e.g., `INFRA_WEB_IP`, `INFRA_DB_IP`, `INFRA_WORKER_IP_0`). These are **public** IPs. The deploy job loads them to make IPs available for Kamal ERB templates — use them only in Kamal `host:` fields (SSH deployment), `servers.*.hosts`, and `proxy.host` (external URLs). **Never** use `INFRA_*_IP` in `env.clear` `env.secret` for inter-component communication; use the CloudStack internal DNS hostname (the accessory name, e.g., `db`) instead.

Complete Example (All Inputs)

Full-stack example with web, database, redis, and workers. Every input is shown with required/optional and default value annotations.

```
# .github/workflows/deploy-preview.yml
name: Deploy Preview
on:
  push:
    branches: [main]
    paths-ignore: [".claude/**"]

permissions:
  contents: read
  packages: write

jobs:
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      env_name: "preview"                    # Optional, default: "preview"
      zone: "ZP01"                           # Optional, default: "ZP01" (options: ZP01, ZP02)
      web_plan: "small"                      # Optional, default: "small"
      web_disk_size_gb: 20                   # Optional, default: 20 (grow only, never shrink)
      accessories: |-                        # Optional, default: "[]" (JSON array)
        [{"name":"db","plan":"medium","disk_size_gb":20},{"name":"redis","plan":"small","disk_size_gb":10}]
      workers_replicas: 2                    # Optional, default: 0 (0 = no workers)
      workers_plan: "small"                  # Optional, default: "small"
      automatic_reboot: true                 # Optional, default: true
      automatic_reboot_time_utc: "05:00"     # Optional, default: "05:00"
      recover: false                         # Optional, default: false (see recovery.md)
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}       # Required
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }} # Required
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}             # Required

  deploy:
    needs: infra
    runs-on: ubuntu-latest
    env:
      POSTGRES_PASSWORD: ${{ secrets.POSTGRES_PASSWORD }}
      API_KEY: ${{ secrets.API_KEY }}
      SMTP_PASSWORD: ${{ secrets.SMTP_PASSWORD }}
    steps:
```

```
        - uses: actions/checkout@v4

        - name: Load infrastructure environment
          run: echo "${{ needs.infra.outputs.infra_env }}" >> "$GITHUB_ENV"

        - name: Set repo identity
          run: |
            echo "REPO_NAME=${{ github.event.repository.name }}" >> "$GITHUB_ENV"
            echo "REPO_FULL=${{ github.repository }}" >> "$GITHUB_ENV"
            echo "REPO_OWNER=${{ github.repository_owner }}" >> "$GITHUB_ENV"

        - uses: webfactory/ssh-agent@v0.9.0
          with:
            ssh-private-key: ${{ secrets.SSH_PRIVATE_KEY }}

        # Expose GHA cache env vars for Kamal's `docker buildx build --cache-from type=gha`.
        # Recommended by https://docs.docker.com/build/cache/backends/gha/
        - uses: crazy-max/ghaction-github-runtime@v3

        - uses: ruby/setup-ruby@v1
          with:
            ruby-version: "3.4"

        - run: gem install kamal --no-document

        - name: Deploy with Kamal
          env:
            KAMAL_REGISTRY_PASSWORD: ${{ secrets.GITHUB_TOKEN }}
          run: |
            kamal setup -d preview
            kamal accessory reboot all -d preview

        # Image is cached via GHA, not the registry -- clean up to prevent storage accumulation
        - uses: actions/delete-package-versions@v5
          with:
            package-name: ${{ github.event.repository.name }}
            package-type: container
            min-versions-to-keep: 1
```

Workflow Permissions

The deploy caller workflow **must** include:

```
  permissions:
    contents: read
    packages: write
```

`packages: write` is required because the caller's deploy job pushes the container image to ghcr.io via Kamal and cleans up old package versions after deployment. The reusable infra workflow only needs `contents: read` (which it declares internally). The teardown workflow does not need `packages: write`.

No Docker Build Steps in Caller Workflows

Do **not** add any of these to the caller workflow:

- `docker/build-push-action` or `docker/login-action` actions
- `docker build`, `docker push`, or `docker login` commands
- Any step that builds or pushes a container image

The caller's deploy job handles the entire Docker lifecycle via Kamal: it checks out the application code, and runs `kamal setup` (or `kamal deploy`), which builds the image from the Dockerfile at the repo root, pushes it to ghcr.io, and deploys it to the VMs -- all in a single step. The `GITHUB_TOKEN` (provided automatically by GitHub Actions) is used as the registry credential `KAMAL_REGISTRY_PASSWORD`), so no separate registry login is needed.

No `secrets: inherit`

The reusable workflow lives in a **public repository** (`gmautner/locaweb-cloud-provision`). GitHub does not allow `secrets: inherit` when calling a reusable workflow from a different repository. Always pass secrets explicitly in the `secrets:` block of the infra job. Application secrets go in the deploy job's `env:` block, not through the reusable workflow.

Passing Outputs to Downstream Jobs

The infra job exposes outputs that can be consumed by the deploy job or additional downstream jobs:

```
jobs:
  infra:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/provision.yml@v1
    with:
      # ... inputs
    secrets:
      # ... secrets

  deploy:
    needs: infra
    # ... deploy steps (see examples above)

  notify:
    needs: infra
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo "Web IP: ${{ needs.infra.outputs.web_ip }}"
          echo "Worker IPs: ${{ needs.infra.outputs.worker_ips }}"
          echo "Accessory IPs: ${{ needs.infra.outputs.accessory_ips }}"
          echo "Infrastructure changed: ${{ needs.infra.outputs.infrastructure_changed }}"
```

Available outputs: see Deploy Output Reference.

scripts/generate_pg_cmd.py

```
#!/usr/bin/env python3
"""Generate the PostgreSQL cmd string tuned for a given VM plan.

Encodes the plan-to-RAM mapping and tuning algorithm from ADR-025.
Outputs the complete postgres command string for use in accessories.db.cmd.

Usage:
    python3 generate_pg_cmd.py --plan medium
    # Output: postgres -D /etc/postgresql -c shared_buffers=1GB ...
"""
import argparse
import sys

# Plan-to-RAM mapping (MiB)
PLAN_RAM_MIB = {
    "micro": 1024,
    "small": 2048,
    "medium": 4096,
    "large": 8192,
    "xlarge": 16384,
    "2xlarge": 32768,
    "4xlarge": 65536,
}


def compute_pg_params(plan):
    """Compute PostgreSQL tuning parameters for a given plan."""
    ram_mib = PLAN_RAM_MIB.get(plan)
    if ram_mib is None:
        raise ValueError(f"Unknown plan: {plan}. "
                         f"Valid plans: {', '.join(PLAN_RAM_MIB.keys())}")

    # max_connections: 100 (<=4GB), 200 (<=16GB), 400 (>16GB)
    if ram_mib <= 4096:
        max_connections = 100
    elif ram_mib <= 16384:
        max_connections = 200
    else:
        max_connections = 400

    # shared_buffers: RAM / 4
    shared_buffers_mib = ram_mib // 4

    # effective_cache_size: RAM * 3 / 4
    effective_cache_size_mib = ram_mib * 3 // 4

    # work_mem: max(RAM / max_conn / 4, 2MB)
    work_mem_mib = max(ram_mib // max_connections // 4, 2)

    # maintenance_work_mem: min(RAM / 16, 2GB)
    maintenance_work_mem_mib = min(ram_mib // 16, 2048)

    return {
        "shared_buffers": format_mib(shared_buffers_mib),
        "effective_cache_size": format_mib(effective_cache_size_mib),
        "work_mem": format_mib(work_mem_mib),
        "maintenance_work_mem": format_mib(maintenance_work_mem_mib),
        "max_connections": str(max_connections),
    }


def format_mib(mib):
    """Format MiB value as PostgreSQL-friendly string (e.g. 1GB, 512MB)."""
    if mib >= 1024 and mib % 1024 == 0:
```

```
        return f"{mib // 1024}GB"
    return f"{mib}MB"


def generate_cmd(plan):
    """Generate the complete postgres command string."""
    params = compute_pg_params(plan)
    parts = ["postgres", "-D", "/etc/postgresql"]
    for key in ("shared_buffers", "effective_cache_size", "work_mem",
                "maintenance_work_mem", "max_connections"):
        parts.extend(["-c", f"{key}={params[key]}"])
    return " ".join(parts)


def main():
    parser = argparse.ArgumentParser(
        description="Generate PostgreSQL cmd string tuned for a VM plan")
    parser.add_argument("--plan", required=True,
                        choices=list(PLAN_RAM_MIB.keys()),
                        help="VM plan name")
    args = parser.parse_args()
    print(generate_cmd(args.plan))


if __name__ == "__main__":
    main()
```

examples/secrets-common

```
# KAMAL_REGISTRY_PASSWORD comes from secrets.GITHUB_TOKEN in the workflow step -- NOT a user-created secret or PAT
KAMAL_REGISTRY_PASSWORD=$KAMAL_REGISTRY_PASSWORD
MY_SECRET=$MY_SECRET
```

examples/secrets.preview

```
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

examples/secrets.production

```
POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
```

examples/deploy.preview.yml

```
  servers:
    web:
      hosts:
        - <%= ENV['INFRA_WEB_IP'] %> # provision job exports web VM IP address INFRA_WEB_IP
    workers: # only needed if using workers
      hosts:
        - <%= ENV['INFRA_WORKER_IP_0'] %> # provision job exports INFRA_WORKER_IP_0, INFRA_WORKER_IP_1, etc.
        - <%= ENV['INFRA_WORKER_IP_1'] %>

  env:
    clear:
      APP_ENV: preview
    secret:
      - MY_SECRET # common secrets, obtained from .kamal/secrets-common
      - DATABASE_URL # obtained from .kamal/secrets.<environment> eg .kamal/secrets.preview

  proxy:
    host: <%= ENV['INFRA_WEB_IP'] %>.nip.io
    ssl: true

  accessories:
    db:
      image: supabase/postgres:17.6.1.093 # see references/postgres-recipe.md
      host: <%= ENV['INFRA_DB_IP'] %> # provision job exports accessory IP address INFRA_<accessory_name>_IP
      port: "5432:5432"
         cmd: "postgres -D /etc/postgresql -c shared_buffers=1GB -c effective_cache_size=3GB -c work_mem=10MB -c
maintenance_work_mem=256MB -c max_connections=100" # see references/postgres-recipe.md, use scripts/generate_pg_cmd.py
to generate
      env:
        secret:
          - POSTGRES_PASSWORD # obtained from .kamal/secrets.<environment> eg .kamal/secrets.preview
      directories:
        - /data/pgdata:/var/lib/postgresql/data # volume mount for accessory. Always mount at /data/<subdir>
```

examples/deploy.production.yml

```
  servers:
    web:
      hosts:
        - <%= ENV['INFRA_WEB_IP'] %> # provision job exports web VM IP address INFRA_WEB_IP
    workers: # only needed if using workers
      hosts:
        - <%= ENV['INFRA_WORKER_IP_0'] %> # provision job exports INFRA_WORKER_IP_0, INFRA_WORKER_IP_1, etc.
        - <%= ENV['INFRA_WORKER_IP_1'] %>

  env:
    clear:
      APP_ENV: production
    secret:
      - MY_SECRET # common secrets, obtained from .kamal/secrets-common
      - DATABASE_URL # obtained from .kamal/secrets.<environment> eg .kamal/secrets.production

  proxy:
    host: www.example.com
    ssl: true # when true, make sure the above domain points to the web VM IP

  accessories:
    db:
      image: supabase/postgres:17.6.1.093 # see references/postgres-recipe.md
      host: <%= ENV['INFRA_DB_IP'] %> # provision job exports accessory IP address INFRA_<accessory_name>_IP
      port: "5432:5432"
         cmd: "postgres -D /etc/postgresql -c shared_buffers=1GB -c effective_cache_size=3GB -c work_mem=10MB -c
maintenance_work_mem=256MB -c max_connections=100" # see references/postgres-recipe.md, use scripts/generate_pg_cmd.py
to generate
      env:
        secret:
            - POSTGRES_PASSWORD # expose ONLY this secret when using supabase/postgres image. User and db are always
postgres
      directories:
        - /data/pgdata:/var/lib/postgresql/data # volume mount for accessory. Always mount at /data/<subdir>
```

examples/deploy.yml

```
service: <%= ENV['REPO_NAME'] %> # do not change
image: <%= ENV['REPO_FULL'] %> # do not change

proxy:
  app_port: 80 # if changed, update your application's port
  forward_headers: false # do not change
  healthcheck:
    path: /up # if changed, update your application's healthcheck path
    interval: 3 # recommended, tune for your application
    timeout: 5 # recommended, tune for your application

ssh:
  user: root # do not change

registry:
  server: ghcr.io
  username: <%= ENV['REPO_OWNER'] %> # do not change
  password:
    - KAMAL_REGISTRY_PASSWORD # do not change

builder: # do not change
  arch: amd64
  cache:
    type: gha
    options: mode=max

env: # environment variables from clear variables, applicable to all environments
  clear:
    BLOB_STORAGE_PATH: /data/blobs
    MY_VAR: hello-from-e2e
  # IMPORTANT: do NOT put env.secret here -- arrays are replaced (not merged) by destination files,
  # so any secrets listed here would be lost when the destination file defines its own env.secret.
  # Put ALL secrets (common + environment-specific) in each config/deploy.<env>.yml instead.

volumes: # volumes on the web VM, applicable to all environments
  - /data/blobs:/data/blobs

servers: # only needed if using workers
  workers:
    cmd: "celery -A tasks worker --loglevel=info" # your worker command
    proxy: false

readiness_delay: 15 # recommended, tune for your application
deploy_timeout: 180 # recommended, tune for your application
drain_timeout: 30 # recommended, tune for your application
```

examples/.kamal/secrets-common

```
# KAMAL_REGISTRY_PASSWORD comes from secrets.GITHUB_TOKEN in the workflow step -- NOT a user-created secret or PAT
KAMAL_REGISTRY_PASSWORD=$KAMAL_REGISTRY_PASSWORD
MY_SECRET=$MY_SECRET
```

examples/.kamal/secrets.preview

```
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD}@db:5432/postgres
```

examples/.kamal/secrets.production

```
POSTGRES_PASSWORD=$POSTGRES_PASSWORD_PRODUCTION
DATABASE_URL=postgres://postgres:${POSTGRES_PASSWORD_PRODUCTION}@db:5432/postgres
```

Skill: computer-setup

Computer Setup

Install and configure all development prerequisites: package manager, mise (tool version manager), podman (container runtime), and GH CLI. Fully idempotent — safe to re-run across sessions.

Overview

This skill detects the current platform (macOS, Linux, or Windows via git bash) and walks through the installation of all required tools. Each step checks whether the tool is already present before attempting installation.

Detect Platform

```
uname -s 2>/dev/null || echo Windows
```

- `Darwin` → follow the **macOS** section
- `Linux` → follow the **Linux** section
- `MINGW64_NT*` / `MSYS_NT*` or similar → follow the **Windows** section

---

macOS

Phase 1 — Install tools

1. Install Homebrew

First check whether Homebrew is already installed:

```
command -v brew
```

If `brew` is found, skip to the next step.

If not found, extract the install command from https://brew.sh/ using the WebFetch tool, then tell the user to open their **Terminal app** (not Claude) and run that command there — it requires `sudo` which is not available inside Claude. Ask the user to come back and confirm once the installation finishes.

2. Install Podman

```
eval "$(/opt/homebrew/bin/brew shellenv 2>/dev/null || /usr/local/bin/brew shellenv 2>/dev/null || true)"
brew install podman
```

This is a no-op if podman is already installed.

3. Install and activate mise

```
brew install mise
```

This is a no-op if mise is already installed. Then ensure mise is activated in the user's shell profile:

```
grep -q 'mise activate' ~/.zprofile || echo 'eval "$(mise activate zsh --shims)"' >> ~/.zprofile
```

4. Install GH CLI

```
brew install gh
```

This is a no-op if gh is already installed.

5. Restart Claude

Ask the user to restart Claude so the new PATH takes effect and the session restarts with the cofounder agent as the main thread. To quit Claude, press **Command+Q** or select **Claude > Quit** on the upper left corner of the screen. Tell them to come back to this same chat session after restarting — they can find it by selecting the **Code** tab and looking for past sessions in the sidebar.

Phase 2 — Verify and set up (after restart)

6. Verify Homebrew

```
brew --version
```

7. Set up Podman machine

```
podman version
```

Interpret the output. If it shows client and server versions, podman is ready.
If it errors about needing a Linux VM:

```
podman machine init
```

Add `--memory 1024` if the computer has less than 16 GB of RAM.

IMPORTANT — before running `podman machine start` : **You MUST tell the user that a macOS system dialog may appear asking to install Rosetta. It can be hidden behind other windows — the user should look for it on their Desktop and press Install** to proceed. Claude cannot interact with this dialog. Wait for the user to confirm they are ready before running the command.

```
podman machine start
```

After running, if the command appears to hang, remind the user to check for the Rosetta dialog again.

Run connectivity test:

```
podman run -d --name podman-setup-test-nginx -p 18080:80 docker.io/library/nginx:alpine
```

Wait a few seconds, then:

```
curl -s -o /dev/null -w '%{http_code}' http://localhost:18080/
```

Expect `200` . Clean up:

```
podman rm -f podman-setup-test-nginx
```

8. Verify mise works

```
mkdir -p ~/test1 && cd ~/test1 && mise use node@24 && node --version && rm -rf ~/test1
```

9. Verify GH CLI

```
gh version
```

Linux

Phase 1 — Install tools

1. Install Podman

```
command -v podman
```

If `podman` is found, skip to the next step.

If not found, refer to https://podman.io/docs/installation#installing-on-linux using the WebFetch tool and follow the specific instructions for the user's Linux distribution. Detect the distro:

```
. /etc/os-release 2>/dev/null && echo "$ID"
```

Use the matching section from the Podman docs (e.g., Alpine, Arch, CentOS, Debian, Fedora, Ubuntu, etc.).

2. Install and activate mise

```
command -v mise
```

If `mise` is found, skip to activation below.

If not found, refer to https://mise.jdx.dev/installing-mise.html using the WebFetch tool and follow the specific instructions for the user's Linux distribution (detected above via `$ID`).

Then ensure mise is activated in the user's shell profile:

```
grep -q 'mise activate' ~/.profile || echo 'eval "$(mise activate bash --shims)"' >> ~/.profile
```

3. Install GH CLI

```
command -v gh
```

If `gh` is found, skip to the next step.

If not found, refer to https://github.com/cli/cli/blob/trunk/docs/install_linux.md using the WebFetch tool (or clone `https://github.com/cli/cli.git` to `/tmp/gh-cli-docs` and read `docs/install_linux.md` if WebFetch fails) and follow the specific instructions for the user's Linux distribution.

4. Restart Claude

If any of steps 1-3 performed an install, ask the user to **exit Claude** and **log out from their Linux session** (then log back in). This is needed to reload `.profile` so the new PATH takes effect. Tell them to come back to this same project directory and start a new Claude session. Otherwise skip to Phase 2.

Phase 2 — Verify and set up (after restart)

5. Set up Podman

```
podman version
```

Verify that podman is working. On Linux, podman runs natively — no machine

init/start is needed.

Run connectivity test:

```
podman run -d --name podman-setup-test-nginx -p 18080:80 docker.io/library/nginx:alpine
```

Wait a few seconds, then:

```
curl -s -o /dev/null -w '%{http_code}' http://localhost:18080/
```

Expect `200` . Clean up:

```
podman rm -f podman-setup-test-nginx
```

6. Verify mise works

```
mkdir -p ~/test1 && cd ~/test1 && mise use node@24 && node --version && rm -rf ~/test1
```

7. Verify GH CLI

```
gh version
```

Windows

> **Note:** It may seem odd to use bash and `.bash_profile` on Windows —
> Claude Code Desktop uses git bash as its shell environment.

1. Verify WSL2

```
wsl --status
```

Interpret the **entire** output, not just the version line. "Default Version: 2" may appear even when WSL2 is not yet functional — the output can still contain errors indicating that required Windows features are missing. Read all lines carefully and watch for any message about unsupported configurations, missing optional components, or virtualization not being enabled.

For example, the output may show "Default Version: 2" alongside errors like:

```
WSL2 is not supported with your current machine configuration.
Please enable the "Virtual Machine Platform" optional component and ensure
virtualization is enabled in the BIOS.
Enable "Virtual Machine Platform" by running: wsl.exe --install --no-distribution
```

If any such error appears, tell the user to:

1. Open **PowerShell as Administrator**
2. Run the command indicated in the error output (in the example above, `wsl.exe --install --no-distribution`) and wait for it to complete
3. Reboot the computer
4. Return to Claude Code after reboot

If WSL2 is not installed at all or the default version is not 2, tell the user to:

1. Open **PowerShell as Administrator**
2. Run `wsl --install`
3. Reboot the computer
4. Return to Claude Code after reboot

After reboot, re-run `wsl --status` to confirm. A second install run may be

needed in some cases.

2. Check Podman

```
podman version
```

If not installed:

```
winget install --exact --id RedHat.Podman --accept-source-agreements --accept-package-agreements
```

3. Install mise

```
mise version
```

If not installed:

```
winget install --exact --id jdx.mise --accept-source-agreements --accept-package-agreements
```

Then ensure mise is activated in the user's shell profile:

```
grep -q 'mise activate' ~/.bash_profile || echo 'eval "$(mise activate bash --shims)"' >> ~/.bash_profile
```

Then add the mise shims directory to the **Windows** PATH so that tools installed by mise are visible to Claude Code (which launches from Windows, not from bash):

```
powershell.exe -Command "[Environment]::SetEnvironmentVariable('Path', '$(cygpath -w "$LOCALAPPDATA/mise/shims")' + ';' + [Environment]::GetEnvironmentVariable('Path', 'User'), 'User')"
```

> `LOCALAPPDATA` is the default location mise uses on Windows when
> `XDG_DATA_HOME` is not set. If in doubt, run `mise doctor` and look for the
> `shims` entry to confirm the path.

4. Check GH CLI

```
gh version
```

If not installed:

```
winget install --exact --id GitHub.cli
```

5. Restart Claude

Ask the user to restart Claude so the new PATH takes effect and the session restarts with the cofounder agent as the main thread. To quit Claude, select **File > Exit** on the top left. Tell them to come back to this same chat session after restarting — they can find it by selecting the **Code** tab and looking for past sessions in the sidebar.

6. Set up Podman machine

After restart (or if no restart was needed):

```
podman version
```

Interpret the output. If it shows client and server versions, podman is ready. If it errors about needing a Linux VM:

```
podman machine init
```

Add `--memory 1024` if the computer has less than 16 GB of RAM. Then:

```
podman machine start
```

Run connectivity test:

```
podman run -d --name podman-setup-test-nginx -p 18080:80 docker.io/library/nginx:alpine
```

Wait a few seconds, then:

```
curl -s -o /dev/null -w '%{http_code}' http://localhost:18080/
```

Expect `200` . Clean up:

```
podman rm -f podman-setup-test-nginx
```

7. Verify mise works

```
mkdir -p ~/test1 && cd ~/test1 && mise use node@24 && node --version && rm -rf ~/test1
```

8. Verify GH CLI

```
gh version
```

Marketplace Auto-Update

Enable automatic updates for the marketplace this plugin belongs to. This step is platform-independent and runs on both macOS and Windows.

1. Retrieve the marketplace name

Find the name of the marketplace to which this cofounder plugin belongs. Store the result for the next step — referred to below as `<marketplace-name>` .

2. Enable auto-update

Read `~/.claude/plugins/known_marketplaces.json` and locate the entry whose key matches `<marketplace-name>` . If the entry already has `"autoUpdate": true` , skip this step. Otherwise, add `"autoUpdate": true` to that entry and write the file back. Preserve all other fields and formatting.

Use the Read tool to inspect the file, then the Edit tool to add the key. For example, if the marketplace name is `my-plugins` and the entry looks like:

```
"my-plugins": {
    "source": { ... },
    "installLocation": "...",
    "lastUpdated": "..."
}
```

Add `"autoUpdate": true` as the last field in that object:

```
"my-plugins": {
    "source": { ... },
    "installLocation": "...",
    "lastUpdated": "...",
    "autoUpdate": true
}
```

Troubleshooting

- **Homebrew/mise/podman not found after install**: Restart Claude so the new PATH takes effect. On Linux, also log out and back in to reload `.profile` .
- **Podman machine fails to start (macOS/Windows)**: Check that virtualization is enabled `sysctl kern.hv_support` on macOS). Ensure no conflicting hypervisor (e.g., Docker Desktop) holds the VM socket.
- **WSL issues on Windows**: "Default Version: 2" does not guarantee WSL2 is working — errors about missing components may follow. If the output mentions "Virtual Machine Platform" or similar, ask the user to run `wsl.exe --install --no-distribution` in PowerShell as Administrator, then reboot. Otherwise, use `wsl --install` .
- **Connectivity test fails**: The nginx container may need an extra second. Re-run the curl check. If it persists, check firewall rules and that port 18080 is free.

Bundled Resources

None — all steps are performed inline.

Skill: frontend-design

This skill guides creation of distinctive, production-grade frontend interfaces that avoid generic "AI slop" aesthetics. Implement real working code with exceptional attention to aesthetic details and creative choices.

The user provides frontend requirements: a component, page, application, or interface to build. They may include context about the purpose, audience, or technical constraints.

Design Thinking

Before coding, understand the context and commit to a BOLD aesthetic direction:
- **Purpose**: What problem does this interface solve? Who uses it?
- **Tone**: Pick an extreme: brutally minimal, maximalist chaos, retro-futuristic, organic/natural, luxury/refined, playful/toy-like, editorial/magazine, brutalist/raw, art deco/geometric, soft/pastel, industrial/utilitarian, etc. There are so many flavors to choose from. Use these for inspiration but design one that is true to the aesthetic direction.
- **Constraints**: Technical requirements (framework, performance, accessibility).
- **Differentiation**: What makes this UNFORGETTABLE? What's the one thing someone will remember?

CRITICAL: Choose a clear conceptual direction and execute it with precision. Bold maximalism and refined minimalism both work - the key is intentionality, not intensity.

Then implement working code (HTML/CSS/JS, React, Vue, etc.) that is:
- Production-grade and functional
- Visually striking and memorable
- Cohesive with a clear aesthetic point-of-view
- Meticulously refined in every detail

Frontend Aesthetics Guidelines

Focus on:
- **Typography**: Choose fonts that are beautiful, unique, and interesting. Avoid generic fonts like Arial and Inter; opt instead for distinctive choices that elevate the frontend's aesthetics; unexpected, characterful font choices. Pair a distinctive display font with a refined body font.
- **Color & Theme**: Commit to a cohesive aesthetic. Use CSS variables for consistency. Dominant colors with sharp accents outperform timid, evenly-distributed palettes.
- **Motion**: Use animations for effects and micro-interactions. Prioritize CSS-only solutions for HTML. Use Motion library for React when available. Focus on high-impact moments: one well-orchestrated page load with staggered reveals (animation-delay) creates more delight than scattered micro-interactions. Use scroll-triggering and hover states that surprise.
- **Spatial Composition**: Unexpected layouts. Asymmetry. Overlap. Diagonal flow. Grid-breaking elements. Generous negative space OR controlled density.
- **Backgrounds & Visual Details**: Create atmosphere and depth rather than defaulting to solid colors. Add contextual effects and textures that match the overall aesthetic. Apply creative forms like gradient meshes, noise textures, geometric patterns, layered transparencies, dramatic shadows, decorative borders, custom cursors, and grain overlays.

Light/Dark Mode System

Two observations to keep in mind:

1. **Design both modes intentionally.** Dark mode is not "invert the colors." Each mode must have its own curated palette — background tones, text contrast, accent vibrancy, shadow depth, and border opacity all shift independently. A dark theme that simply swaps white for black looks washed out and lifeless. Design each mode as a first-class experience with its own personality.
2. **Use CSS custom properties as the single source of truth.** Define all color tokens as `--color-*` variables on `:root` (light) and `[data-theme="dark"]` (or `.dark`). Components should never reference raw color values — only variables. This makes theming a one-place change and keeps both modes perfectly in sync. Offer three toggle states

— **Light**, **Dark**, and **System** — where System (the default) follows the OS-wide preference via `prefers-color-scheme: dark` . Persist the user's choice to `localStorage` so it survives reloads, but always start with System so the app respects the platform setting out of the box.

Interaction Feedback

- **Always provide visual feedback for operations.** Every user action that triggers processing must have a visible response — a button that saves without closing the dialog should show a brief success state (checkmark, color flash, toast notification, or subtle pulse). Silent success is indistinguishable from a broken button. Match the feedback weight to the action weight: a destructive delete deserves a more emphatic confirmation than a routine save.
- **Use cursor styles to signal interactivity.** Set `cursor: pointer` on all clickable elements (buttons, links, cards, toggles, tabs). Use `cursor: not-allowed` on disabled elements and `cursor: grab` / `cursor: grabbing` on draggable items. The cursor is the user's first hint that an element is actionable — if it stays as the default arrow, the affordance is invisible.

Favicon

Always generate a favicon for the app. Design it as part of the brand identity — it should echo the chosen aesthetic direction (color palette, visual motifs, overall tone). Deliver it as an SVG `<link rel="icon">` in `index.html` so it works across all sizes without extra files. Keep the shape simple and recognizable at 16×16px.

NEVER use generic AI-generated aesthetics like overused font families (Inter, Roboto, Arial, system fonts), cliched color schemes (particularly purple gradients on white backgrounds), predictable layouts and component patterns, and cookie-cutter design that lacks context-specific character.

Interpret creatively and make unexpected choices that feel genuinely designed for the context. No design should be the same. Vary between light and dark themes, different fonts, different aesthetics. NEVER converge on common choices (Space Grotesk, for example) across generations.

IMPORTANT: Match implementation complexity to the aesthetic vision. Maximalist designs need elaborate code with extensive animations and effects. Minimalist or refined designs need restraint, precision, and careful attention to spacing, typography, and subtle details. Elegance comes from executing the vision well.

Remember: Claude is capable of extraordinary creative work. Don't hold back, show what can truly be created when thinking outside the box and committing fully to a distinctive vision.

Skill: pre-flight-check

Pre-Flight Check

Validate that the current environment is suitable for project initialization before running any other cofounder skill.

When to Run

Execute the pre-flight check **after computer-setup and before repo-setup**. If the check fails, report the failure reasons to the user and stop — do not attempt to proceed with setup.

Running the Check

```
bash ${CLAUDE_PLUGIN_ROOT}/skills/pre-flight-check/scripts/preflight.sh
```

The script exits `0` and prints `PREFLIGHT_PASSED` on success, or exits `1` and prints `PREFLIGHT_FAILED` followed by one or more error lines on failure.

Conditions Checked

1. Must Not Be in the Home Directory

Running from the user's home folder (`~`) risks polluting it with project files.

On failure: Recommend creating and navigating to a project subfolder first (e.g., `mkdir ~/my-project && cd ~/my-project`).

2. No Pre-Existing Content Without a Git Repository

When **both** of these conditions are true simultaneously, the check fails:

- The directory contains files or folders other than `.claude` and `.venv`
- No local git repository has been initialized `.git/` does not exist)

This prevents accidentally initializing a project on top of untracked existing content.

On failure: Suggest using an empty directory, or initializing a git repository first to acknowledge the existing content.

3. Git Sync

When the directory has a git repository **and** at least one remote is configured, the script automatically synchronizes:

1. **Commit** any uncommitted local changes (staged or unstaged)
2. **Pull** remote commits using rebase to keep history linear
3. **Push** local commits to the remote

This ensures every session starts from a fully synchronized state.

On failure: The script reports a `GIT_SYNC_ERROR` with details (commit failed, merge conflict on pull, or push rejected). The user must resolve the issue manually and re-run the check.

Handling Failures

When the pre-flight check fails:

1. Display each error reason to the user in plain language
2. Provide the recommended remediation for each failure
3. **Do not proceed** with any setup skills — wait for the user to fix the environment and re-run the check

Bundled Resources

Scripts

- *`scripts/preflight.sh` ** — Runs all environment validations and reports pass/fail with specific error codes

scripts/preflight.sh

```
  #!/usr/bin/env bash
  set -euo pipefail

  errors=()

  # 1. Home folder check — must not be running from ~
  current_dir=$(pwd -P)
  home_dir="$HOME"
  if [[ "$current_dir" == "$home_dir" ]]; then
       errors+=("IN_HOME_DIR: Current directory is the user's home folder. Initialize the project within a subfolder
(e.g., ~/<project-name>).")
  fi

  # 2. Pre-existing content without a git repo
  #    Blocks only when BOTH conditions are true:
  #      - The folder contains items other than .claude, .venv
  #      - No local git repository has been initialized (.git/ absent)
  has_other_content=false
  has_git=false

  [[ -d ".git" ]] && has_git=true

  for item in * .[!.]* ..?*; do
      [[ -e "$item" ]] || continue
      case "$item" in
          .git|.claude|.venv) continue ;;
      esac
      has_other_content=true
      break
  done

  if $has_other_content && ! $has_git; then
       errors+=("EXISTING_CONTENT_NO_GIT: Directory contains pre-existing files but no git repository. This looks like
an attempt to initialize a project over existing content. Use an empty directory or initialize a git repo first.")
  fi

  # Report results
  if [[ ${#errors[@]} -gt 0 ]]; then
      echo "PREFLIGHT_FAILED"
      for err in "${errors[@]}"; do
          echo "  - $err"
      done
      exit 1
  fi

  # 3. Git sync — commit/push local changes and pull remote commits
  #    Only runs when a git repo with at least one remote exists.
  if $has_git && [[ -n "$(git remote 2>/dev/null)" ]]; then
      current_branch=$(git symbolic-ref --short HEAD 2>/dev/null || true)
      if [[ -z "$current_branch" ]]; then
          echo "PREFLIGHT_FAILED"
          echo "  - GIT_SYNC_ERROR: HEAD is detached — cannot sync."
          exit 1
      fi

      # Resolve the upstream tracking ref (e.g. "origin/main") if configured
      upstream=$(git rev-parse --abbrev-ref "@{upstream}" 2>/dev/null || true)

      # Commit any outstanding local changes
      if [[ -n "$(git status --porcelain 2>/dev/null)" ]]; then
          echo "SYNC: Committing local changes..."
          git add -A
          git commit -m "Auto-sync: commit outstanding changes before session" --no-gpg-sign || {
              echo "PREFLIGHT_FAILED"
```

```
            echo "  - GIT_SYNC_ERROR: Failed to commit local changes."
            exit 1
        }
    fi

    if [[ -n "$upstream" ]]; then
        # Upstream is configured — use it for pull/push (handles name mismatches)
        echo "SYNC: Pulling remote changes (upstream: $upstream)..."
        git pull --rebase || {
            echo "PREFLIGHT_FAILED"
            echo "  - GIT_SYNC_ERROR: Pull failed — possible merge conflict. Resolve manually and re-run."
            exit 1
        }

        echo "SYNC: Pushing local commits..."
        git push || {
            echo "PREFLIGHT_FAILED"
            echo "  - GIT_SYNC_ERROR: Push failed — check remote access and try again."
            exit 1
        }
    else
        # No upstream — try origin/<branch> as a best-effort fallback
        echo "SYNC: No upstream configured, trying origin/$current_branch..."
        if git rev-parse --verify "origin/$current_branch" >/dev/null 2>&1; then
            git pull --rebase origin "$current_branch" || {
                echo "PREFLIGHT_FAILED"
                echo "  - GIT_SYNC_ERROR: Pull failed — possible merge conflict. Resolve manually and re-run."
                exit 1
            }
        fi

        git push --set-upstream origin "$current_branch" || {
            echo "PREFLIGHT_FAILED"
            echo "  - GIT_SYNC_ERROR: Push failed — check remote access and try again."
            exit 1
        }
    fi

    echo "SYNC: Repository is up to date."
fi

echo "PREFLIGHT_PASSED"
exit 0
```

Skill: repo-setup

Repository Setup

Initialize a local git repository and create a matching remote on GitHub using the GH CLI, with device-code authentication that works reliably in all environments.

Prerequisites

The **computer-setup** skill must have been run first — it installs `gh` and `git`.

A GitHub account is required before proceeding. If the user does not have one, guide them through account creation:

1. Navigate to <https://github.com/>.
2. Click **Sign up** (or **Continue with Google** for social login).
3. Follow the prompts and verify the email address — without a verified email, repository creation will fail.

Recommend enabling two-factor authentication after sign-up. For more details, see Creating an account on GitHub.

Authentication

Device Code Flow

Run `gh auth status` — interpret whether it shows the user as authenticated.

If not authenticated, run:

```
    GH_FORCE_TTY=1 NO_COLOR=1 GH_PROMPT_DISABLED=1 gh auth login --hostname github.com --git-protocol https --scopes "repo,read:org,workflow" > /tmp/gh-auth.log 2>&1 & sleep 3 && cat /tmp/gh-auth.log
```

Retrieve the one-time code from the output and display it **very prominently** to the user along with the URL https://github.com/login/device. For example:

> **GitHub login required!**
>
> Your one-time code is: *`XXXX-XXXX`
>
> Open **https://github.com/login/device** in your browser and enter the code.

Ask the user to complete the browser authorization and confirm when done.

Verifying Authentication

After the user confirms, verify:

```
  gh auth status
```

Repository Initialization

Full Setup

Before running the init script, check if a remote is already configured:

```
git remote get-url origin 2>/dev/null
```

If `origin` is already set, the repo is fully configured — **do not ask the user for a name or visibility, and skip the init script entirely.**

If no remote is configured, ask the user for:
- **Repository name** — suggest the current folder name (basename of the working directory) as the default. Do not suggest other alternatives.
- **Visibility** — `private` (default) or `public`.

Then run the init script:

```
bash ${CLAUDE_PLUGIN_ROOT}/skills/repo-setup/scripts/repo-init.sh <repo-name> [private|public]
```

The script will:
1. Initialize a local git repo if `.git/` does not exist
2. Create an initial empty commit if the repo has no commits
3. Skip remote creation if `origin` is already configured locally
4. If the repo already exists on GitHub, add it as `origin` and push
5. Otherwise, create the GitHub repository with the given name and visibility (default: `private`), add it as `origin`, and push

Manual Steps

If finer control is needed, run each step individually:

```
# Initialize local repo
git init

# Create remote and link it (from the project directory)
gh repo create <repo-name> --private --source=. --remote=origin --push
```

Important Notes

- **Authentication must happen before repo creation.** Run the auth flow first if `gh auth status` fails.
- **Do not use `--web` flag** for `gh auth login` — it attempts to open a browser which may not work. The device code flow (default when `--web` is omitted) is more reliable.
- **Default visibility is private.** Always confirm with the user before creating a public repository.

Bundled Resources

Scripts

- **`scripts/repo-init.sh`** — Initializes local git repo and creates the matching GitHub remote in one step

scripts/repo-init.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Initialize a local git repo and create a matching remote on GitHub.
#
# Usage: bash repo-init.sh <repo-name> [visibility]
#   repo-name  : Name for the GitHub repository (e.g. "my-web-app")
#   visibility : "private" (default) or "public"

REPO_NAME="${1:?Usage: repo-init.sh <repo-name> [private|public]}"
VISIBILITY="${2:-private}"

# Validate visibility
if [[ "$VISIBILITY" != "private" && "$VISIBILITY" != "public" ]]; then
  echo "ERROR: visibility must be 'private' or 'public', got '$VISIBILITY'" >&2
  exit 1
fi

# Ensure gh is authenticated
if ! gh auth status &>/dev/null; then
  echo "ERROR: Not authenticated with GitHub. Run 'gh auth login' first." >&2
  exit 1
fi

# Initialize local git repo if needed
if [ ! -d .git ]; then
  echo "Initializing local git repository..."
  git init
fi

# Create initial commit if repo is empty
if ! git rev-parse HEAD &>/dev/null; then
  echo "Creating initial commit..."
  git commit --allow-empty -m "Initial commit"
fi

# Check if remote 'origin' already exists locally
if git remote get-url origin &>/dev/null; then
  echo "Remote 'origin' already set to: $(git remote get-url origin)"
  echo "Skipping remote creation."
  exit 0
fi

# Check if the repo already exists on GitHub
if gh repo view "$REPO_NAME" &>/dev/null; then
  echo "Repository '$REPO_NAME' already exists on GitHub."
  REMOTE_URL="$(gh repo view "$REPO_NAME" --json url --jq .url)"
  echo "Adding existing remote as origin: $REMOTE_URL"
  git remote add origin "$REMOTE_URL"
  BRANCH="$(git branch --show-current)"
  # Pull remote history first, then push
  git fetch origin
  git pull --rebase origin "$BRANCH" 2>/dev/null || true
  git push -u origin "$BRANCH"
else
  # Create the remote repository on GitHub
  echo "Creating $VISIBILITY repository '$REPO_NAME' on GitHub..."
  gh repo create "$REPO_NAME" \
    "--$VISIBILITY" \
    --source=. \
    --remote=origin \
    --push
fi
```

```
  echo ""
  echo "Repository created and pushed:"
  echo "  Local:  $(pwd)"
  echo "  Remote: $(git remote get-url origin)"
```

Skill: ssh-key-rotation

SSH Key Rotation

Rotates the SSH key for all VMs of a deployed environment. After rotation, only the new key grants access -- the old key is permanently revoked from every VM.

When to Use

1. **Explicit request**: The user asks to rotate, regenerate, or replace SSH keys.
2. **Missing local key**: The agent needs to SSH into a VM (for logs, debugging, database access, etc.) but the expected key file does not exist on disk.

Detecting a missing key

Before any SSH operation, the agent resolves the key path:

```
REPO_NAME=$(gh repo view --json name -q .name)

# Preview environment
SSH_KEY=~/.ssh/$REPO_NAME

# Other environments (e.g., production)
SSH_KEY=~/.ssh/$REPO_NAME-production
```

If the file does not exist (`test -f "$SSH_KEY"` fails), the key is missing and rotation is needed.

Warnings (Always Communicate Before Proceeding)

Before starting rotation, **always** warn the user:

1. **Downtime**: Rotation stops each VM, resets its SSH key, and restarts it. All VMs in the environment will experience downtime during the process (accessories first, then workers, then web -- to minimize user-facing downtime).
2. **Old key revoked**: The old SSH key is permanently erased from all VMs' `authorized_keys` files. Anyone using the old key will lose access immediately.
3. **All environments are independent**: Each environment (preview, production, etc.) has its own SSH key. Rotation only affects the specified environment.

Ask for explicit confirmation before proceeding.

Rotation Procedure

Step 1: Determine the environment

Identify which environment needs rotation:

- If the user specifies an environment, use it.
- If the agent discovered a missing key during a troubleshooting attempt, use the environment that was being targeted.
- If ambiguous, ask the user.

Step 2: Generate a new SSH key locally

Delete the old key file (if it exists) and generate a fresh one using the standard naming convention:

```
REPO_NAME=$(gh repo view --json name -q .name)
```

```
# Preview environment
rm -f ~/.ssh/$REPO_NAME ~/.ssh/$REPO_NAME.pub
ssh-keygen -t ed25519 -f ~/.ssh/$REPO_NAME -N "" -C "$REPO_NAME-deploy"
chmod 600 ~/.ssh/$REPO_NAME

# Other environments (e.g., production)
rm -f ~/.ssh/$REPO_NAME-production ~/.ssh/$REPO_NAME-production.pub
ssh-keygen -t ed25519 -f ~/.ssh/$REPO_NAME-production -N "" -C "$REPO_NAME-deploy-production"
chmod 600 ~/.ssh/$REPO_NAME-production
```

Step 3: Update the GitHub secret

Upload the new private key to the corresponding GitHub secret:

```
# Preview
gh secret set SSH_PRIVATE_KEY < ~/.ssh/$REPO_NAME

# Production (or other environment -- suffix matches env_name uppercased)
gh secret set SSH_PRIVATE_KEY_PRODUCTION < ~/.ssh/$REPO_NAME-production
```

Step 4: Determine the zone

The rotation workflow needs the `zone` where the environment is deployed. Infer it from the existing deploy workflow:

```
# Read the deploy workflow for the environment
cat .github/workflows/deploy-preview.yml | grep zone
# or for production:
cat .github/workflows/deploy-production.yml | grep zone
```

The `zone` value is in the `with:` block of the infra job (e.g., `zone: "ZP01"`).

Step 5: Create and run the rotation caller workflow

Create a caller workflow that invokes the reusable rotation workflow. This follows the same pattern as teardown workflows:

```
# .github/workflows/rotate-ssh-key-preview.yml
name: Rotate SSH Key Preview
on:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  rotate:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/rotate-ssh-key.yml@v1
    with:
      env_name: "preview"
    secrets:
      CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
      CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
      SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY }}
```

For other environments, adjust `env_name` and the `SSH_PRIVATE_KEY` secret reference:

```
# .github/workflows/rotate-ssh-key-production.yml
name: Rotate SSH Key Production
on:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  rotate:
    uses: gmautner/locaweb-cloud-provision/.github/workflows/rotate-ssh-key.yml@v1
    with:
```

```
        env_name: "production"
      secrets:
        CLOUDSTACK_API_KEY: ${{ secrets.CLOUDSTACK_API_KEY }}
        CLOUDSTACK_SECRET_KEY: ${{ secrets.CLOUDSTACK_SECRET_KEY }}
        SSH_PRIVATE_KEY: ${{ secrets.SSH_PRIVATE_KEY_PRODUCTION }}
```

Commit and push the workflow file, then trigger it:

```
git add .github/workflows/rotate-ssh-key-preview.yml
git commit -m "Add SSH key rotation workflow for preview"
git push

# Trigger the workflow
gh workflow run rotate-ssh-key-preview.yml
```

Step 6: Monitor the rotation

```
# Watch the run
gh run list --workflow=rotate-ssh-key-preview.yml --limit=5
gh run watch <run-id>
```

Give the user a direct link to follow in the GitHub UI:

```
gh run list --limit=1 --json databaseId,url -q '.[0].url'
```

If the workflow fails, read the logs:

```
gh run view <run-id> --log-failed
```

Step 7: Verify SSH access

After the workflow completes successfully, verify that the new key works:

```
REPO_NAME=$(gh repo view --json name -q .name)

# Get the web IP from the latest deploy run
rm -rf ~/provision-output
gh run list --workflow=deploy-preview.yml --status=success --limit=1
gh run download <run-id> --name provision-output --dir ~/provision-output
cat ~/provision-output/provision-output.json

# Test SSH with the new key
ssh -i ~/.ssh/$REPO_NAME -o ConnectTimeout=10 root@<web_ip> "echo 'SSH rotation successful'"
```

Step 8: Resume the original task

If rotation was triggered because the agent needed SSH access for troubleshooting, resume the original operation (checking logs, debugging, database access, etc.) using the new key.

Workflow Inputs

The reusable rotation workflow `rotate-ssh-key.yml@v1`) accepts:

Input	Type	Default	Description
`env_name`	string	`"preview"`	Environment name (must match the deployed environment)

Required secrets:

Secret	Description
`CLOUDSTACK_API_KEY`	CloudStack API key
`CLOUDSTACK_SECRET_KEY`	CloudStack secret key
`SSH_PRIVATE_KEY`	The **new** SSH private key (already updated in Step 3)

What the Rotation Does (Server-Side)

1. Verifies the SSH keypair and network exist in CloudStack (safety check)
2. Deletes the old keypair from CloudStack and registers the new public key under the same name
3. For each VM (accessories first, workers next, web last):
   - Stops the VM
   - Resets its SSH key via CloudStack API
   - Starts the VM
   - Connects via SSH with the new key and overwrites `authorized_keys` with only the new key
4. Prints a summary of results

Key File Naming Convention

Environment	Local key path	GitHub secret
preview (default)	`~/.ssh/<repo-name>`	`SSH_PRIVATE_KEY`
production	`~/.ssh/<repo-name>-production`	`SSH_PRIVATE_KEY_PRODUCTION`
other `<env_name>`	`~/.ssh/<repo-name>-<env_name>`	`SSH_PRIVATE_KEY_<ENV_NAME>` (uppercased)

Skill: tech-stack

Tech Stack

Go JSON API + React SPA served from a single binary and deployed as one container.

Architecture

Backend: Go stdlib `net/http` router, `pgx/v5` for Postgres, `sqlc` for query generation, `slog` for logging, embedded SQL migrations via `go:embed`.

Frontend: Vite + React + TypeScript, shadcn/ui components, Tailwind CSS, React Router. When making frontend design decisions (layouts, styling, component aesthetics, UI polish), use the **frontend-design** skill for guidance on creating distinctive, production-grade interfaces.

Project Layout

```
.
├── backend/
│   ├── cmd/server/main.go       # Entrypoint
│   ├── internal/
│   │   ├── config/              # Env var parsing
│   │   ├── database/
│   │   │   ├── migrations/*.sql # Embedded, forward-only
│   │   │   ├── queries/*.sql    # sqlc source
│   │   │   └── sqlc/            # Generated — do not edit
│   │   └── handler/             # HTTP handlers (JSON API)
│   ├── go.mod
│   ├── go.sum
│   └── sqlc.yaml
├── frontend/                    # Vite + React SPA
│   ├── src/
│   │   ├── components/ui/       # shadcn/ui (generated, editable)
│   │   └── pages/               # Route-level components
│   └── dist/                    # Build output (gitignored)
└── Dockerfile
```

Ensure the project `.gitignore` includes at least:

```
backend/cmd/server/server
frontend/dist/
frontend/node_modules/
.venv/
.env
.claude/launch.json
```

`backend/cmd/server/server` is the locally compiled Go binary produced by `go build`. It must not be committed.

`.claude/launch.json` is generated locally by Claude Code Desktop's Preview feature and contains platform-specific commands — it must not be committed.

`.env` holds **all** service connection strings and application secrets needed to run locally. It must **never** be committed. After creating or updating it, restrict permissions:

```
chmod 0600 .env
```

Example contents:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/postgres?sslmode=disable
REDIS_URL=redis://localhost:6379
N8N_WEBHOOK_URL=http://localhost:5678/webhook
```

The Go backend reads these values via `os.Getenv()`. During local development, load the file before starting the

server using `. .env` (POSIX dot syntax — not `source`, which is a bash builtin and may not work in all shells, particularly when commands run through `subprocess` or `with_server.py`):

```
set -a && . .env && set +a
```

This bridges the gap between local and deployed: locally `.env` provides the values; deployed, Kamal config provides them. The Go code stays the same (`os.Getenv("REDIS_URL")`).

Clear vs secret env vars in deployment

When translating `.env` entries to Kamal config for deployment, the agent must decide whether each env var is **clear** or **secret**:

- **Secret** — the URL contains a credential (password, API key, token). Goes in `env.secret` + `.kamal/secrets.<env>`. The credential is stored as a GitHub Secret, and the URL is derived in the secrets file. Example: `DATABASE_URL` contains `POSTGRES_PASSWORD`.
- **Clear** — the URL has no credentials. Goes in `env.clear` in the Kamal config. No GitHub Secret needed. Example: Redis without auth (`REDIS_URL: redis://redis:6379`).

The rule: **if the URL embeds a credential, it's a secret; otherwise it's clear.** When in doubt, make it a secret — it's safer and the only cost is a GitHub Secret entry.

In deployed environments, secrets are injected as environment variables by the deployment platform — see the **app-deploy** skill.

`mise.toml` should **not** be gitignored — it is committed to the repo so all developers use the same tool versions.

Key Decisions

Single-binary serving

The Go server handles everything: API routes under `/api/`, static assets under `/assets/`, and a catch-all that returns `index.html` for SPA routing. In development, Vite's dev server proxies API calls to the Go backend.

Postgres first

PostgreSQL is the primary external service. Use Postgres-backed alternatives whenever possible:
- **Queues:** `pgmq` — lightweight message queue with visibility timeout, archive, and batch operations. See references/pgmq.md
- **Pub/Sub:** `LISTEN` `NOTIFY` — no extension needed; combine with a persistence layer for durability. See references/notify-patterns.md
- **Caching:** unlogged tables
- **Scheduling:** `pg_cron` + `pg_net` — in-database cron plus async HTTP requests for triggering app endpoints on a schedule. **Do not use container-level cron.** See references/pg-cron.md
- **Search:** `pgroonga` — full-text search for all languages including CJK, with boolean queries, ranking, highlighting, and JSONB search. No configuration needed. When the app involves searchable content (products, articles, listings, messages, logs), **proactively propose adding search**. Falls back to native `tsvector` `tsquery` only when a simpler built-in solution suffices for a single well-supported language. See references/pgroonga.md
- **Vectors:** `pgvector` — embeddings storage and similarity search with HNSW and IVFFlat indexes. See references/pgvector.md
- **JSON validation:** `pg_jsonschema` — validate `json` `jsonb` columns against JSON Schema via CHECK constraints. See references/pg-jsonschema.md
- **Geospatial:** `postgis` — geometry types, spatial indexes, and geographic functions. See <https://postgis.net/>
- **HTTP from SQL:** `pg_net` — asynchronous HTTP/HTTPS requests from SQL; used with `pg_cron` for scheduled calls or from triggers for webhooks
- Other notable extensions: `pgjwt`, `pg_stat_statements`, `pgaudit`, `pg_hashids`

All 60+ bundled extensions from `supabase/postgres` are available.

When the application needs a capability that Postgres and its extensions cannot provide — a pre-built tool like n8n, a specialized system like Kafka, or a use case where a dedicated service is clearly superior — add it as an accessory. See Local Services for running it locally and `docs/INFRASTRUCTURE.md` for recording it. At deployment time, each accessory gets its own VM (see the **app-deploy** skill).

sqlc for all queries

All SQL lives in `backend/internal/database/queries/*.sql`. Run `cd backend && sqlc generate` after changes. **Never write raw SQL strings in Go handler code.**

Always include `emit_json_tags: true` in `sqlc.yaml` so that generated Go structs include lowercase JSON tags (e.g., `json:"id"` instead of exporting `ID` as-is). Without this, the API returns PascalCase field names that don't match frontend expectations.

Migrations at startup

Embedded SQL files applied in order before the server accepts traffic. Forward-only, numbered sequentially (`001_create_users.sql`, `002_add_tasks.sql`, ...). Each migration should be idempotent where possible (`CREATE TABLE IF NOT EXISTS`, `CREATE INDEX IF NOT EXISTS`).

The `go:embed` directive only accepts files in the same directory or subdirectories of the file that declares it — paths with `..` are rejected by the compiler. Place the embed directive in a Go file next to the `migrations/` directory (e.g., `backend/internal/database/migrate.go`), not in `cmd/server/main.go`.

Database connection retry

The Go backend should retry the database connection at startup (up to 10 attempts, 1-second delay between each). This handles parallel startup — Preview starts all servers simultaneously, so the backend may come up before the database is ready — and is also good practice for production deployments.

```
var pool *pgxpool.Pool
for i := range 10 {
    pool, err = pgxpool.New(ctx, os.Getenv("DATABASE_URL"))
    if err == nil {
        if err = pool.Ping(ctx); err == nil {
            break
        }
        pool.Close()
    }
    slog.Warn("database not ready, retrying", "attempt", i+1, "err", err)
    time.Sleep(time.Second)
}
if err != nil {
    slog.Error("failed to connect to database", "err", err)
    os.Exit(1)
}
```

Real-time updates via SSE

The Go backend listens for Postgres `NOTIFY` events and holds open a standard HTTP response with `Content-Type: text/event-stream` for each connected client. The React frontend uses the browser's built-in `EventSource` API. SSE is preferred over WebSockets to avoid adverse proxy configurations.

Deployment Constraints

These match the **app-deploy** skill requirements:

- Single container on **port 80** (controlled by `PORT` env var, defaulting to `8080` for local dev)
- Health check: `GET /up` → HTTP 200**
- Database via `DATABASE_URL` (preferred) or individual `POSTGRES_*` env vars — **fail hard if missing**
- File storage at `BLOB_STORAGE_PATH` (e.g. `/data/blobs`) — configured via `env.clear` in `deploy.yml`

- PostgreSQL as the primary data store (with 60+ bundled extensions via `supabase/postgres`). If the application needs services beyond what Postgres provides, additional accessories can be added via the deploy skill's Kamal layer
- No ORMs, no JavaScript frameworks beyond React, no CSS preprocessors

Dockerfile

Multi-stage: (1) build frontend with Node, (2) build Go binary, (3) minimal Alpine runtime with binary + `frontend/dist/` + CA certs. The Go binary embeds migrations; frontend assets are served from `/frontend/dist` on disk. The Node and Go versions in the Dockerfile must match the versions installed locally — run `go version` and `node --version` to check current versions before writing or updating the Dockerfile.

Local Development

All tools are on PATH via **mise** (set up by **computer-setup**). The database runs as a `supabase/postgres` container via **podman** (also set up by **computer-setup**), matching the production image.

> **Container naming convention:** Each project's database container is named `<repo_name>-db` (e.g., `myapp-db`), where `<repo_name>` is the basename of the project's root directory. This prevents collisions when multiple cofounder projects coexist on the same machine. Derive the name once at the start of the session and use it consistently for all `podman` commands.

> **Critical: `go.mod` lives in `backend/`, not in the project root.** All Go and sqlc commands (`go run`, `go build`, `go test`, `go mod tidy`, `sqlc generate`) **must** execute from the `backend/` directory. Always include `cd backend &&` inside the `bash -c` string. When a command chain involves multiple layers of shell invocation (bash → go), prefer writing a small helper script instead of nesting everything in a single `bash -c` string — this avoids the most common source of repeated build failures.

Project tool versions

On first setup (when `mise.toml` does not yet exist in the project root), lock the tool versions:

```
mise use go@1 sqlc@1 python@3.14 node@24 jq@1
```

This creates a `mise.toml` that is committed to the repo, ensuring all developers use the same versions. If a version upgrade is needed later, re-run `mise use` with the new version.

1. Start the database and local services

Before starting the container, check if any podman container is already using port 5432. If a container from **another project** is occupying the port, do **not** force-stop it. Instead, inform the user:

> "The container `<other_name>` from another project is currently using port 5432. Could you please stop it with `podman stop <other_name>` so we can start this project's database?"

Wait for the user to confirm before proceeding.

```
# Derive the container name from the repo directory
CONTAINER_NAME="$(basename "$(pwd)")-db"

# Start supabase/postgres container (matching production image)
 # Important: provide only the POSTGRES_PASSWORD environment variable. The database is started with both user and database name preset to `postgres`.
podman run -d \
  --name "$CONTAINER_NAME" \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  supabase/postgres:17.6.1.095

# Verify it's ready (uses container exec instead of pg_isready)
podman exec "$CONTAINER_NAME" pg_isready -U postgres
```

Local Services

When the application needs services beyond Postgres, run them as podman containers locally. Each service maps to an accessory that will be provisioned as a dedicated VM in deployment.

Naming convention

Every project container is named `<repo_name>-<accessory_name>` :

```
myapp-db
myapp-redis
myapp-n8n
```

The `-db` convention already exists for Postgres. Extend it to all accessories. This enables the cleanup pattern (see Stopping all project containers).

Start-or-create pattern

Containers persist across sessions. On a fresh session the containers from the previous session may already exist (stopped). Always use the start-or-create pattern instead of a bare `podman run` :

```
podman start myapp-redis 2>/dev/null || \
  podman run -d --name myapp-redis -p 6379:6379 redis:7-alpine
```

`podman start` succeeds silently if the container exists (running or stopped). If it doesn't exist, the fallback `podman run` creates it. This prevents "name already in use" errors on session resume.

Two accessory types

Distinguish backend-connected and standalone accessories with different procedures:

Backend-connected (Redis, Kafka, Meilisearch, etc.):

1. Start-or-create the container with naming convention and port mapping
2. Readiness check (e.g., `podman exec <name> redis-cli ping`)
3. Add env var to `.env` (e.g., `REDIS_URL=redis://localhost:6379`)
4. Add Go client library, read env var via `os.Getenv()`
5. Record in `docs/INFRASTRUCTURE.md`

Standalone (n8n, WordPress, Metabase, etc.):

1. Start-or-create the container with naming convention, port mapping, and local directory mount if persistence matters
2. Readiness check (e.g., `curl -s http://localhost:5678/healthz`)
3. Give the user a clickable `http://localhost:<port>` link
4. Record in `docs/INFRASTRUCTURE.md`
5. No Go code changes unless the backend also calls the tool's API — see "Hybrid accessories" below

Hybrid accessories

Some accessories are both user-facing AND called by the backend. For example, n8n where the user manages workflows in the n8n UI, but the Go backend triggers workflows via webhook.

For hybrid accessories:
- Treat as standalone for the user-facing aspect (give the browser link)
- ALSO add a backend env var for the API endpoint (e.g., `N8N_WEBHOOK_URL=http://localhost:5678/webhook`)
- Record the env var in `docs/INFRASTRUCTURE.md`
- The Type column can remain `standalone` — the env var in the Env Var column signals the backend dependency

Port conflict detection

Same pattern as the existing Postgres port check: before starting a container, check if another podman container is already occupying the port. If it belongs to another project, ask the user to stop it.

Common recipes

Accessory	Image	Port	Readiness check
Redis	`redis:7-alpine`	6379	`podman exec <name> redis-cli ping`
n8n	`n8nio/n8n:latest`	5678	`curl -s http://localhost:5678/healthz`
Meilisearch	`getmeili/meilisearch:latest`	7700	`curl -s http://localhost:7700/health`
WordPress	`wordpress:latest`	8080	`curl -s http://localhost:8080`
WAHA	`devlikeapro/waha:latest`	3000	`curl -s http://localhost:3000/api/health`

These are starting points. The agent should check the image's documentation for the correct ports and readiness endpoints.

Local vs. deployed hostnames

Service	Local `.env`)	Deployed hostname
Postgres	`localhost:5432`	`db:5432`
Redis	`localhost:6379`	`redis:6379`
n8n	`localhost:5678`	`n8n:5678`

Locally, all services are on `localhost` with mapped ports. In deployment, each accessory gets its own VM. The deployed hostname matches the **accessory name** — CloudStack internal DNS resolves it to the VM's private IP within the isolated network (see env-vars.md — Database Connection Variables for the documented pattern). Never use public IPs for inter-service communication.

The `.env` file (local) and Kamal config (deployed) each provide the correct values; the Go code reads them identically via `os.Getenv()`. For clear env vars (no credentials), the deployed value goes in `env.clear`. For secret env vars (with credentials), the deployed value goes in `.kamal/secrets` — see the `.env` section above.

2. Start the Go API (terminal 1)

```
bash -c 'set -a && . .env && set +a && cd backend && DEV_MODE=1 go run ./cmd/server'
```

3. Start the Vite dev server (terminal 2)

```
bash -c 'cd frontend && npm install && npm run dev'
```

Access the app at `http://localhost:5173` during development. Vite proxies `/api/*` and `/auth/*` to the Go backend.

Stopping all project containers

```
REPO_NAME="$(basename "$(pwd)")"

# Stop and remove all containers for this project
podman ps -a --filter "name=^${REPO_NAME}-" --format '{{.Names}}' | xargs -r podman stop
podman ps -a --filter "name=^${REPO_NAME}-" --format '{{.Names}}' | xargs -r podman rm
```

This relies on the naming convention `<repo_name>-<accessory_name>`) and removes all project containers at once.

Preview (Claude Code Desktop)

When `preview_*` tools are available (Claude Code Desktop), Preview manages the dev servers automatically — you do not need to start or stop them manually. Use `preview_screenshot`, `preview_click`, and `preview_snapshot` for quick visual checks during development. Reserve Playwright (via the **webapp-testing** skill) for comprehensive E2E test suites.

Accessories in Preview mode

Preview manages the Go backend and Vite dev server automatically, but does **not** manage podman containers. Before using Preview, ensure all accessory containers are running:

```
REPO_NAME="$(basename "$(pwd)")"
podman start "${REPO_NAME}-db" || true
podman start "${REPO_NAME}-redis" || true  # if applicable
```

Check `docs/INFRASTRUCTURE.md` for the full list of accessories. If a container doesn't exist yet, create it first following the Local Services recipes above.

Environment variables in Preview

The Go backend command in `.claude/launch.json` must load `.env` so that all service URLs (DATABASE_URL, REDIS_URL, etc.) are available. When generating or updating launch.json, use the same `. .env` pattern as the CLI command:

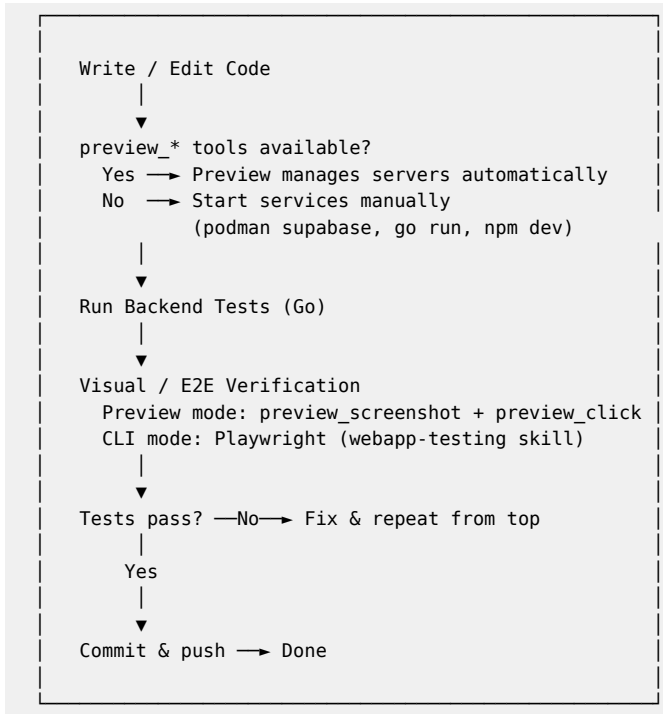
```
set -a && . .env && set +a && cd backend && DEV_MODE=1 go run ./cmd/server
```

This ensures Preview and CLI modes use the same env var source, and adding a new accessory only requires updating `.env` — not rewriting launch.json.

Local Development Feedback Loop

> **Preview mode:** If you have access to `preview_*` tools (Claude Code Desktop), Preview manages the dev servers — **do not start them manually**. Use `preview_screenshot`, `preview_snapshot`, and `preview_click` for visual verification instead of Playwright for quick checks. Use Playwright (via the **webapp-testing** skill) for comprehensive E2E test suites.

The core workflow is: **write code → spin up local instance → run tests → repeat until the feature works → commit & push.**



After committing and pushing, ask the user if they want to deploy to the cloud. If yes, use the **app-deploy** skill to run the **Deployment Feedback Loop**, which monitors the GitHub Actions workflow, verifies the health check, and handles deployment-specific failures.

Backend testing (Go)

Run unit tests against the local database:

```
bash -c 'set -a && . .env && set +a && cd backend && DEV_MODE=1 go test ./...'
```

- Test files live next to the code they test (`handler/todo_test.go` tests `handler/todo.go`).
- Use table-driven tests. Each test case gets a descriptive name.
- For database tests, use a test helper that runs migrations and wraps each test in a transaction that rolls back.
- Test the HTTP handlers via `httptest.NewServer` — send real HTTP requests, assert on status codes and JSON bodies.

Frontend testing (Playwright)

Use the **webapp-testing** skill for Playwright-based end-to-end testing. The `with_server.py` helper manages the full stack:

```
python skills/webapp-testing/scripts/with_server.py \
  --server "podman start $(basename $(pwd))-db || true" --port 5432 \
  --server "set -a && . .env && set +a && cd backend && DEV_MODE=1 go run ./cmd/server" --port 8080 \
  --server "cd frontend && npm run dev" --port 5173 \
  -- python test_script.py
```

Note: use `. .env` (dot) instead of `source .env` — `with_server.py` may run commands under `/bin/sh`, where `source` is not available.

Or, if services are already running (with env vars already loaded), write a standalone Playwright script:

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto('http://localhost:5173')
    page.wait_for_load_state('networkidle')
    # ... test interactions
    browser.close()
```

Follow the reconnaissance-then-action pattern: screenshot → identify selectors → execute actions → assert results.

sqlc workflow

Whenever SQL queries change:

```
bash -c 'cd backend && sqlc generate'
```

Then update the Go code that calls the generated functions. Never hand-write SQL in Go files.

Conventions

- **Thin handlers:** parse request → call database → return JSON. No business logic in handlers.
- **Logging:** `slog` exclusively. Never `fmt.Println` or `log.Println`.
- **Validation:** Server-side validation for all inputs. Never trust client-side validation alone.
- **Authorization:** Checks in every handler, not just middleware.
- **Frontend components:** `bash -c 'cd frontend && npx shadcn@latest add <component>'`
- **No ORMs.** SQL through sqlc only.
- **No CSS preprocessors.** Tailwind CSS only.
- **No additional JavaScript frameworks.** React + React Router only.

Authorization Best Practices

If the application has a user login area, **self sign-in with username and password** is acceptable for quickly prototyping the app, but **never for production** — the security of this method is weak.

Recommended approach: start with self sign-in (username + password) for prototyping, then move as soon as possible to one or both of:

- **Email with magic link** — practical, doesn't require memorization. Requires configuration of an SMTP gateway. See references/smtp-gateway.md
- **Google Auth** — practical, doesn't require memorization. Relies on Google's security mechanisms. Requires configuration of Google Auth (cost free). See references/google-auth.md

Dev Login for Testing

During local development, the agent (Playwright scripts or Claude Desktop Preview) needs to test pages behind authentication. Magic link and Google Auth flows cannot be completed in automated tests, so the backend must expose a **dev-only login endpoint** that bypasses the real auth flow.

How it works

The backend registers a `POST /api/dev/login` route **only when `DEV_MODE=1` ** is set. The guard must be at route registration time (not middleware) so the endpoint physically does not exist without the flag.

This endpoint accepts a user identifier (e.g., email), looks up the user, and creates a session using the exact same mechanism the real auth flow uses — same cookie name, same token format, same session store. The only difference is that it skips the external provider (magic link email or Google OAuth).

Test user

The dev login handler should create the user on the fly if it doesn't already exist `INSERT ... ON CONFLICT DO NOTHING`). This keeps the test user out of migrations, which run in all environments including production.

Security

- The endpoint must **never** be registered when `DEV_MODE` is unset — enforced by the guard at route registration time.
- Production deployments must **never** set `DEV_MODE` . The deploy skill does not include it.

Other References

- **references/smtp-gateway.md** -- SMTP gateway setup: Gmail (prototyping) and Locaweb (production) for sending e-mails (reminders, auth links, notifications, etc.)
- **references/google-auth.md** -- Google Auth OAuth setup: Google Cloud Console configuration, consent screen, credentials

Postgres Extension References

- **references/pgroonga.md** -- PGroonga full-text search: operators, ranking, highlighting, CJK support
- **references/pgmq.md** -- pgmq message queue: SQL examples for send, read, archive, delete
- **references/pg-cron.md** -- pg_cron + pg_net: scheduled jobs, HTTP triggers, common patterns
- **references/pgvector.md** -- pgvector similarity search: distance operators, HNSW/IVFFlat indexes, tuning
- **references/pg-jsonschema.md** -- pg_jsonschema validation: CHECK constraint pattern, core functions
- **references/notify-patterns.md** -- LISTEN/NOTIFY + persistence: pgmq for job queues, regular tables for data updates, polling fallback

references/google-auth.md

Quick Setup: Google Auth

Create a Google Cloud project

1. Go to <https://cloud.google.com/>
2. Click **Comece a usar sem custo financeiro**
3. Sign up and choose a payment method: credit card (Google Auth is free — no charges)

Configure the OAuth consent screen

1. Left navigation menu → **APIs e serviços** → **Credenciais**
2. Click **Configurar tela de consentimento** → **Vamos começar**
3. Choose a name for the app, select the pre-filled e-mail
4. Público: **Externo**
5. Contact info: your e-mail

Create OAuth credentials

1. Left navigation menu → **Clientes** → **Criar cliente**
2. Application type: **Aplicativo da Web**
3. Under **URIs de redirecionamento autorizados** → **Adicionar URI** → paste the redirect URI provided by Claude
4. Click **Criar**
5. Copy the **ID do cliente** (Client ID) and **Chave secreta do cliente** (Client Secret)

references/notify-patterns.md

LISTEN/NOTIFY + Persistence Patterns

Native `LISTEN` `NOTIFY` delivers real-time notifications but is **not durable** — if no listener is connected, the message is lost. When durability matters, combine it with a persistence layer so consumers can recover missed events.

Core idea

Layer	Role
`NOTIFY`	Real-time push — instant wake-up for connected listeners
Persistence (pgmq, regular table, etc.)	Durable record — survives missed notifications
Polling	Fallback — catches anything the listener missed

`NOTIFY` is an **optimization**, not a delivery guarantee. The persistence layer is the source of truth.

Pattern 1: pgmq + NOTIFY

Use when consumers need to **process jobs** reliably (task queues, background work).

Producer

```
BEGIN;

SELECT pgmq.send(
    queue_name => 'jobs',
    msg        => '{"job_id": 123}'
);

NOTIFY jobs_channel;

COMMIT;
```

Both the enqueue and the notification are in the same transaction — if it rolls back, neither happens.

Consumer

1. `LISTEN jobs_channel`
2. When notified → read the queue
3. If no notification arrives → periodic polling fallback (e.g., every 30 s)

```
SELECT *
FROM pgmq.read(
    queue_name => 'jobs',
    vt         => 30,
    qty        => 10
);
```

After processing, archive or delete:

```
SELECT pgmq.archive('jobs', msg_id => 42);
```

Pattern 2: Table update + NOTIFY

Use when the consumer just needs **fresh data** (live dashboards, page updates, cache invalidation).

Producer

```
BEGIN;

UPDATE orders SET status = 'shipped' WHERE id = 456;

NOTIFY orders_channel, '456';

COMMIT;
```

Consumer

- Connected listener receives the notification and refreshes instantly.
- If the notification is missed (disconnect, deploy, etc.), the next page load or periodic refresh reads the updated row — no data is lost because the table **is** the persistence layer.

No queue needed here — the regular table already holds the durable state.

When to use which

Scenario	Persistence layer	Why
Background jobs, task processing	pgmq	Need reliable delivery, visibility timeout, retry semantics
Live UI updates, dashboards	Regular table	Data is already persisted; notification is just an optimization for instant push
Cache invalidation	None (NOTIFY only may suffice)	Stale cache is tolerable; next read rebuilds it anyway

references/pg-cron.md

pg_cron — In-Database Job Scheduling

PostgreSQL extension that runs scheduled SQL commands inside the database using background workers. Jobs run as the user who scheduled them.

Setup

```
CREATE EXTENSION IF NOT EXISTS pg_cron;
```

For non-superuser roles, grant access:

```
GRANT USAGE ON SCHEMA cron TO my_role;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA cron TO my_role;
```

Schedule a Job

```
SELECT cron.schedule(
  'nightly-cleanup',        -- job name (unique identifier)
  '0 3 * * *',              -- cron expression
  $$DELETE FROM logs WHERE created_at < now() - interval '30 days'$$
);
```

Cron Expression Syntax

Five fields: `minute hour day_of_month month day_of_week`

Expression	Schedule
`* * * * *`	Every minute
`*/5 * * * *`	Every 5 minutes
`0 * * * *`	Every hour
`0 0 * * *`	Daily at midnight
`0 2 * * 0`	Weekly on Sunday at 2 AM
`0 0 1 * *`	Monthly on the 1st at midnight

Field ranges: minutes (0–59), hours (0–23), day of month (1–31), month (1–12), day of week (0–6, Sunday = 0).

Common Patterns

Periodic cleanup:

```
SELECT cron.schedule(
  'purge-expired-sessions',
  '*/15 * * * *',
  $$DELETE FROM sessions WHERE expires_at < now()$$
);
```

Materialized view refresh:

```
SELECT cron.schedule(
  'refresh-dashboard-stats',
  '0 * * * *',
  $$REFRESH MATERIALIZED VIEW CONCURRENTLY dashboard_stats$$
);
```

Vacuum a table:

```
SELECT cron.schedule(
```

```
    'vacuum-events',
    '0 4 * * *',
    $$VACUUM (ANALYZE) events$$
  );
```

Triggering Application Code via pg_net

`pg_net` is the recommended way to run scheduled application logic. Instead of container-level cron, schedule `pg_cron` jobs that fire HTTP requests to your app's endpoints using `pg_net`.

Setup:

```
  CREATE EXTENSION IF NOT EXISTS pg_net;
```

net.http_post

```
  net.http_post(
    url text,
    body jsonb default '{}'::jsonb,
    params jsonb default '{}'::jsonb,
    headers jsonb default '{"Content-Type": "application/json"}'::jsonb,
    timeout_milliseconds int default 1000
  ) returns bigint  -- request ID
```

net.http_get

```
  net.http_get(
    url text,
    params jsonb default '{}'::jsonb,
    headers jsonb default '{}'::jsonb,
    timeout_milliseconds int default 1000
  ) returns bigint  -- request ID
```

Both functions are **asynchronous** — the HTTP request is dispatched by a background worker after the transaction commits. The return value is a request ID. Responses are stored in `net._http_response` for 6 hours.

pg_cron + pg_net examples

Trigger a cleanup endpoint every 5 minutes:

```
  SELECT cron.schedule(
    'periodic-cleanup',
    '*/5 * * * *',
    $$SELECT net.http_post('http://localhost/tasks/cleanup', '{}')$$
  );
```

Trigger a daily report generation (offloaded to workers via pgmq):

```
  SELECT cron.schedule(
    'daily-report',
    '0 3 * * *',
    $$SELECT net.http_post('http://localhost/tasks/daily-report', '{}')$$
  );
```

Call an external webhook:

```
  SELECT cron.schedule(
    'webhook-ping',
    '*/5 * * * *',
    $$SELECT net.http_post('https://example.com/webhook', '{"source": "myapp"}')$$
  );
```

Pattern: heavy scheduled work. For resource-intensive tasks (large data processing, bulk emails, report generation), the pg_cron job should call a lightweight app endpoint that enqueues work to pgmq. Workers then pick up and execute the job, keeping the web VM free for request handling.

Checking responses

```
SELECT * FROM net._http_response ORDER BY created DESC LIMIT 10;
```

Columns: `id` , `status_code` , `headers` , `body` , `timed_out` , `error_msg` , `created` .

Manage Jobs

List all scheduled jobs:

```
SELECT * FROM cron.job;
```

Check execution history:

```
SELECT * FROM cron.job_run_details ORDER BY start_time DESC LIMIT 20;
```

Remove a job:

```
SELECT cron.unschedule('nightly-cleanup');
```

Notes

- Jobs execute using background workers and do not block normal database operations.
- Each job runs in its own transaction.
- Failed jobs are logged in `cron.job_run_details` with the error message.
- pg_cron runs in the database where the extension is installed — it cannot run commands in other databases.

Source: <https://github.com/citusdata/pg_cron>

references/pg-jsonschema.md

pg_jsonschema — JSON Schema Validation

PostgreSQL extension that validates `json` and `jsonb` values against JSON Schema documents (draft 2020-12 and earlier).

Setup

```
CREATE EXTENSION pg_jsonschema;
```

Core Functions

Function	Description
`json_matches_schema(schema json, instance json)`	Validate a `json` value
`jsonb_matches_schema(schema json, instance jsonb)`	Validate a `jsonb` value

Both return `boolean`.

Basic Validation

```
SELECT json_matches_schema(
  '{"type": "object"}',
  '{}'
);
-- true

SELECT json_matches_schema(
  '{"type": "object", "properties": {"name": {"type": "string"}}, "required": ["name"]}',
  '{"age": 30}'
);
-- false (missing required "name")
```

CHECK Constraint Pattern

The primary use case — enforce a JSON schema at the database level so invalid documents are rejected on insert/update:

```
CREATE TABLE customer (
  id SERIAL PRIMARY KEY,
  metadata jsonb,
  CHECK (
    jsonb_matches_schema(
      '{
        "type": "object",
        "properties": {
          "tags": {
            "type": "array",
            "items": {
              "type": "string",
              "maxLength": 16
            }
          }
        }
      }',
      metadata
    )
  )
);
```

Valid:

```
INSERT INTO customer (metadata) VALUES ('{"tags": ["vip", "darkmode-ui"]}');
-- OK
```

Invalid:

```
INSERT INTO customer (metadata) VALUES ('{"tags": [1, 3]}');
-- ERROR: new row for relation "customer" violates check constraint
```

Storing Schemas in a Table

For reusable schemas, store them in a dedicated table and reference by name:

```
CREATE TABLE json_schemas (
  name TEXT PRIMARY KEY,
  schema JSON NOT NULL
);

INSERT INTO json_schemas (name, schema) VALUES (
  'customer_metadata',
  '{
    "type": "object",
    "properties": {
      "tags": {"type": "array", "items": {"type": "string"}}
    }
  }'
);
```

Then validate against stored schemas in application code or triggers.

Source: <https://github.com/supabase/pg_jsonschema>

references/pgmq.md

pgmq — Lightweight Message Queue

PostgreSQL extension that provides a durable, ACID-compliant message queue inside the database. Messages are stored in regular tables (prefixed `q_`) and archived messages in tables prefixed `a_`.

Setup

```
CREATE EXTENSION pgmq;
```

Create a Queue

```
SELECT pgmq.create('my_queue');
```

Send Messages

Single message:

```
SELECT * FROM pgmq.send(
  queue_name => 'my_queue',
  msg => '{"foo": "bar"}'
);
-- Returns the message ID
```

With delay (invisible for N seconds):

```
SELECT * FROM pgmq.send(
  queue_name => 'my_queue',
  msg => '{"foo": "bar"}',
  delay => 5
);
```

Batch send:

```
SELECT pgmq.send_batch(
  queue_name => 'my_queue',
  msgs => ARRAY['{"a": 1}', '{"b": 2}', '{"c": 3}']::jsonb[]
);
```

Read Messages

Read without removing. The visibility timeout (`vt`) hides messages from other consumers for the specified seconds, providing at-most-once delivery within that window. If the consumer doesn't delete/archive before the timeout, the message becomes visible again.

```
SELECT * FROM pgmq.read(
  queue_name => 'my_queue',
  vt => 30,
  qty => 2
);
```

Returns empty result if the queue is empty or all messages are currently invisible.

Pop (Read + Delete)

Read and immediately delete in one atomic operation:

```
SELECT * FROM pgmq.pop('my_queue');
```

Archive Messages

Move messages to the archive table instead of deleting them:

```
-- Single message
SELECT pgmq.archive(queue_name => 'my_queue', msg_id => 2);

-- Multiple messages
SELECT pgmq.archive(queue_name => 'my_queue', msg_ids => ARRAY[3, 4, 5]);
```

Query archived messages:

```
SELECT * FROM pgmq.a_my_queue;
```

Delete Messages

Permanently remove a message (no archive):

```
SELECT pgmq.delete('my_queue', 6);
```

Drop a Queue

Remove the queue and all its messages:

```
SELECT pgmq.drop_queue('my_queue');
```

Typical Consumer Pattern

```
-- 1. Read a batch with visibility timeout
SELECT * FROM pgmq.read('tasks', vt => 60, qty => 1);

-- 2. Process the message in application code

-- 3a. On success: archive (or delete)
SELECT pgmq.archive('tasks', msg_id => 42);

-- 3b. On failure: message automatically becomes visible again after VT expires
```

Source: <https://github.com/pgmq/pgmq>

references/pgroonga.md

PGroonga — Full-Text Search for All Languages

PostgreSQL extension that provides fast full-text search using the Groonga engine. Unlike PostgreSQL's built-in `tsvector` `tsquery` (which requires language-specific dictionaries), PGroonga supports all languages including CJK (Chinese, Japanese, Korean) out of the box with no configuration.

Setup

```
CREATE EXTENSION IF NOT EXISTS pgroonga;
```

Create a Search Index

```
CREATE INDEX idx_memos_content ON memos USING pgroonga (content);
```

Multiple columns can be indexed:

```
CREATE INDEX idx_memos_search ON memos USING pgroonga (title, content);
```

Search Operators

`&@` — Single Keyword Match

```
SELECT * FROM memos WHERE content &@ 'database';
```

`&@~` — Query Syntax (Boolean Logic)

```
-- OR search
SELECT * FROM memos WHERE content &@~ 'PGroonga OR PostgreSQL';

-- AND search (default when space-separated)
SELECT * FROM memos WHERE content &@~ 'full text search';

-- NOT
SELECT * FROM memos WHERE content &@~ 'PostgreSQL -MySQL';
```

`LIKE` Compatibility

PGroonga accelerates existing `LIKE` queries — no query changes needed:

```
SELECT * FROM memos WHERE content LIKE '%engine%';
```

Note: `LIKE` through PGroonga is slower than native operators due to recheck overhead. Prefer `&@` or `&@~` for new code.

Relevance Ranking

```
SELECT *, pgroonga_score(tableoid, ctid) AS score
FROM memos
WHERE content &@ 'keyword'
ORDER BY score DESC;
```

Highlighting Search Terms

```
SELECT pgroonga_highlight_html(content, pgroonga_query_extract_keywords('search term'))
```

```
  FROM memos
  WHERE content &@~ 'search term';
```

Snippets (KWIC — Keyword in Context)

```
  SELECT pgroonga_snippet_html(content, pgroonga_query_extract_keywords('search term'))
  FROM memos
  WHERE content &@~ 'search term';
```

JSONB Search

PGroonga can index and search inside JSONB columns:

```
  CREATE INDEX idx_docs_metadata ON docs USING pgroonga (metadata);

  -- Containment
  SELECT * FROM docs WHERE metadata @> '{"category": "tech"}'::jsonb;

  -- Full-text search inside JSONB
  SELECT * FROM docs WHERE metadata &@ 'keyword';
```

When to Use PGroonga vs tsvector/tsquery

Feature	`tsvector` `tsquery`	PGroonga
Language support	Needs dictionaries per language	All languages, including CJK
Query syntax	Custom `& | !`)	Familiar `AND OR NOT`)
Relevance ranking	`ts_rank()`	`pgroonga_score()`
Highlighting	`ts_headline()`	`pgroonga_highlight_html()`
JSONB search	No	Yes

PGroonga works well for all languages. It adds particularly strong support for CJK and other languages where `tsvector` `tsquery` requires manual dictionary configuration or produces poor results.

sqlc Integration

All SQL lives in `backend/internal/database/queries/*.sql` and is generated via `sqlc generate` . PGroonga operators (`&@` , `&@~`) work in sqlc query files like any other operator.

```
  -- name: SearchMemos :many
  SELECT id, title, content, pgroonga_score(tableoid, ctid) AS score
  FROM memos
  WHERE content &@~ @query
  ORDER BY score DESC
  LIMIT @max_results;
```

```
  -- name: SearchMemosWithHighlight :many
  SELECT id, title,
         pgroonga_highlight_html(content, pgroonga_query_extract_keywords(@query)) AS content_highlighted,
         pgroonga_score(tableoid, ctid) AS score
  FROM memos
  WHERE content &@~ @query
  ORDER BY score DESC
  LIMIT @max_results;
```

The migration that creates the PGroonga index goes in `backend/internal/database/migrations/` alongside other migration files:

```
  -- NNN_add_search_index.sql
  CREATE EXTENSION IF NOT EXISTS pgroonga;
  CREATE INDEX IF NOT EXISTS idx_memos_search ON memos USING pgroonga (title, content);
```

Source: <https://pgroonga.github.io/>

references/pgvector.md

pgvector — Vector Similarity Search

PostgreSQL extension for storing embeddings and performing similarity search. Supports exact and approximate nearest-neighbor queries.

Setup

```
CREATE EXTENSION vector;
```

Create a Table with Vectors

Specify the number of dimensions (must match your embedding model):

```
CREATE TABLE items (
  id BIGSERIAL PRIMARY KEY,
  embedding VECTOR(1536)  -- e.g., OpenAI text-embedding-3-small
);
```

Insert Vectors

```
INSERT INTO items (embedding) VALUES ('[0.1, 0.2, 0.3, ...]');
```

Upsert:

```
INSERT INTO items (id, embedding) VALUES (1, '[0.1, 0.2, 0.3]')
  ON CONFLICT (id) DO UPDATE SET embedding = EXCLUDED.embedding;
```

Distance Operators

Operator	Distance Metric	Use Case
`<->`	L2 (Euclidean)	General purpose
`<=>`	Cosine	Normalized embeddings
`<#>`	Negative inner product	When embeddings are not normalized
`<+>`	L1 (Manhattan)	Sparse data

Similarity Queries

Nearest neighbors by L2 distance:

```
SELECT * FROM items ORDER BY embedding <-> '[0.1, 0.2, 0.3]' LIMIT 5;
```

Cosine similarity (convert distance to similarity):

```
SELECT *, 1 - (embedding <=> '[0.1, 0.2, 0.3]') AS similarity
FROM items
ORDER BY embedding <=> '[0.1, 0.2, 0.3]'
LIMIT 5;
```

Filter by distance threshold:

```
SELECT * FROM items WHERE embedding <-> '[0.1, 0.2, 0.3]' < 0.5;
```

Find similar to an existing row:

```
SELECT * FROM items
WHERE id != 1
```

```
  ORDER BY embedding <-> (SELECT embedding FROM items WHERE id = 1)
  LIMIT 5;
```

Indexing

Without an index, queries perform exact (brute-force) search. For large datasets, create an approximate nearest-neighbor (ANN) index.

HNSW (Recommended Default)

Better query performance, no training step, supports concurrent inserts during build:

```
  CREATE INDEX ON items USING hnsw (embedding vector_l2_ops);
```

Operator classes by distance metric:

Distance	Operator Class
L2	`vector_l2_ops`
Cosine	`vector_cosine_ops`
Inner product	`vector_ip_ops`
L1	`vector_l1_ops`

Tuning parameters:

```
  CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);
```

- `m` — Max connections per layer (default 16). Higher = better recall, more memory.
- `ef_construction` — Candidate list size during build (default 64). Higher = better recall, slower build.

Query-time recall tuning:

```
  SET hnsw.ef_search = 100;  -- default 40, higher = better recall, slower query
```

IVFFlat

Faster to build, lower memory, but requires data to already be present (trains on existing rows):

```
  CREATE INDEX ON items USING ivfflat (embedding vector_l2_ops) WITH (lists = 100);
```

Rule of thumb for `lists`: `rows / 1000` for up to 1M rows, `sqrt(rows)` for more.

Query-time tuning:

```
  SET ivfflat.probes = 10;  -- default 1, higher = better recall, slower query
```

Performance Tips

- Set `maintenance_work_mem` high for index builds: `SET maintenance_work_mem = '4 GB';`
- Use parallel workers for faster builds: `SET max_parallel_maintenance_workers = 7;`
- Monitor index build progress:

```
SELECT phase, round(100.0 * blocks_done / nullif(blocks_total, 0), 1) AS "%" FROM pg_stat_progress_create_index;
```

- For filtered queries, add a B-tree index on the filter column too.
- Partial indexes for frequent filter values:

```
CREATE INDEX ON items USING hnsw (embedding vector_l2_ops) WHERE (category_id = 123);
```

Vector Types

Type	Description	Max Dimensions

`vector`	32-bit float	2,000
`halfvec`	16-bit float (less memory)	4,000
`bit`	Binary (Hamming/Jaccard)	64,000
`sparsevec`	Sparse (only stores non-zero)	varies

Source: <https://github.com/pgvector/pgvector>

references/smtp-gateway.md

SMTP Gateway

An SMTP gateway is needed if the app sends e-mails (reminders, authentication links, etc.).

Recommended methods

Gmail SMTP — good for prototyping, low volume

1. Generate an App Password

1. Go to <https://myaccount.google.com/security>
2. **Segurança e Login** → **Verificação em duas etapas** → ensure it is enabled
3. Type **"app passwords"** in the search field
4. Fill **Nome do app:** `SMTP`
5. Copy the 16-character generated password back to Claude

2. Configure the web app

Setting	Value
SMTP Server	`smtp.gmail.com`
Port	`587` (TLS/STARTTLS)
Username	Your full Gmail address
Password	The App Password generated above

Locaweb SMTP — good for production use

Sign up at <https://www.locaweb.com.br/smtp-locaweb/>. If you already have the SMTP service, go to **Central do Cliente** to retrieve your credentials.

Skill: webapp-testing

Web Application Testing

To test local web applications, write native Python Playwright scripts.

Helper Scripts Available:
- `scripts/with_server.py` - Manages server lifecycle (supports multiple servers)

Always run scripts with `--help` first to see usage. DO NOT read the source until you try running the script first and find that a customized solution is abslutely necessary. These scripts can be very large and thus pollute your context window. They exist to be called directly as black-box scripts rather than ingested into your context window.

Decision Tree: Choosing Your Approach

```
User task → Is it static HTML?
    ├─ Yes → Read HTML file directly to identify selectors
    │         ├─ Success → Write Playwright script using selectors
    │         └─ Fails/Incomplete → Treat as dynamic (below)
    │
    └─ No (dynamic webapp) → Is the server already running?
        ├─ No → Run: python scripts/with_server.py --help
        │        Then use the helper + write simplified Playwright script
        │
        └─ Yes → Reconnaissance-then-action:
            1. Navigate and wait for networkidle
            2. Take screenshot or inspect DOM
            3. Identify selectors from rendered state
            4. Execute actions with discovered selectors
```

Example: Using with_server.py

To start a server, run `--help` first, then use the helper:

Single server:

```
python scripts/with_server.py --server "npm run dev" --port 5173 -- python your_automation.py
```

Multiple servers (e.g., backend + frontend):

```
python scripts/with_server.py \
  --server "cd backend && python server.py" --port 3000 \
  --server "cd frontend && npm run dev" --port 5173 \
  -- python your_automation.py
```

Including accessories

If the app depends on accessories beyond the database, start them in the `with_server.py` invocation. Use `podman start` (not `podman run`) — the containers should already exist from the Local Services setup:

```
python scripts/with_server.py \
  --server "podman start $(basename $(pwd))-db || true" --port 5432 \
  --server "podman start $(basename $(pwd))-redis || true" --port 6379 \
  --server "set -a && . .env && set +a && cd backend && DEV_MODE=1 go run ./cmd/server" --port 8080 \
  --server "cd frontend && npm run dev" --port 5173 \
  -- python test_script.py
```

Note: use `. .env` (dot) instead of `source .env` — `with_server.py` may run commands under `/bin/sh`, where `source` is not available.

Which accessories to include: Backend-connected accessories (Redis, Kafka, Meilisearch) must be running for tests to pass — include them. Standalone accessories (n8n, WordPress) are typically not exercised by E2E tests of the main app — omit them unless a test specifically interacts with their API.

To create an automation script, include only Playwright logic (servers are managed automatically):

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True) # Always launch chromium in headless mode
    page = browser.new_page()
    page.goto('http://localhost:5173') # Server already running and ready
    page.wait_for_load_state('networkidle') # CRITICAL: Wait for JS to execute
    # ... your automation logic
    browser.close()
```

Reconnaissance-Then-Action Pattern

1. **Inspect rendered DOM**:

```
    page.screenshot(path='/tmp/inspect.png', full_page=True)
    content = page.content()
    page.locator('button').all()
```

2. **Identify selectors** from inspection results

3. **Execute actions** using discovered selectors

Common Pitfalls

- **Don't** inspect the DOM before waiting for `networkidle` on dynamic apps
- **Do** wait for `page.wait_for_load_state('networkidle')` before inspection
- **Don't** assume the default Vite port (5173) is available. If another dev server is already using the port, Vite silently picks the next one (5174, 5175, ...). Always check the Vite startup output for the actual `Local:` URL before writing test scripts.

Installing Playwright

Create a virtualenv first, then install Playwright inside it:

```
# macOS/Linux:
python -m venv .venv
bash -c 'source .venv/bin/activate && pip install playwright && python -m playwright install chromium'
```

Activate the virtualenv before running Playwright scripts:

```
# macOS/Linux:
bash -c 'source .venv/bin/activate && python your_test.py'
```

Windows note: Use `.venv\Scripts\activate` instead of `source .venv/bin/activate`, or call `.venv\Scripts\python.exe` directly:

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install playwright
.venv\Scripts\python.exe -m playwright install chromium
.venv\Scripts\python.exe your_test.py
```

Authenticating in Tests

Many app features are behind a login wall. Magic link and Google Auth flows cannot be completed in automated tests, so the backend provides a **dev-only login endpoint** (`POST /api/dev/login`) when running locally with `DEV_MODE=1`. See the **tech-stack** skill for the backend design.

In Playwright scripts, call the dev login endpoint using `page.request.post()` before navigating to authenticated pages. Because this method shares the browser context's cookie jar, the session cookie is automatically available for all subsequent navigations.

In Claude Desktop Preview, use `preview_click` to submit the login form through the UI — either a dev-mode login

shortcut (if the app renders one when `DEV_MODE=1`) or the regular login form with the test user credentials.

Best Practices

- **Use bundled scripts as black boxes** - To accomplish a task, consider whether one of the scripts available in `scripts/` can help. These scripts handle common, complex workflows reliably without cluttering the context window. Use `--help` to see usage, then invoke directly.
- Use `sync_playwright()` for synchronous scripts
- Always close the browser when done
- Use descriptive selectors: `text=` , `role=` , CSS selectors, or IDs
- Add appropriate waits: `page.wait_for_selector()` or `page.wait_for_timeout()`

Reference Files

- **examples/** - Examples showing common patterns:
  - `element_discovery.py` - Discovering buttons, links, and inputs on a page
  - `static_html_automation.py` - Using file:// URLs for local HTML
  - `console_logging.py` - Capturing console logs during automation

scripts/with_server.py

```
#!/usr/bin/env python3
"""
Start one or more servers, wait for them to be ready, run a command, then clean up.

Usage:
    # Single server
    python scripts/with_server.py --server "npm run dev" --port 5173 -- python automation.py
    python scripts/with_server.py --server "npm start" --port 3000 -- python test.py

    # Multiple servers
    python scripts/with_server.py \
      --server "cd backend && python server.py" --port 3000 \
      --server "cd frontend && npm run dev" --port 5173 \
      -- python test.py
"""

import subprocess
import socket
import time
import sys
import argparse

def is_server_ready(port, timeout=30):
    """Wait for server to be ready by polling the port."""
    start_time = time.time()
    while time.time() - start_time < timeout:
        try:
            with socket.create_connection(('localhost', port), timeout=1):
                return True
        except (socket.error, ConnectionRefusedError):
            time.sleep(0.5)
    return False


def main():
    parser = argparse.ArgumentParser(description='Run command with one or more servers')
      parser.add_argument('--server', action='append', dest='servers', required=True, help='Server command (can be
repeated)')
    parser.add_argument('--port', action='append', dest='ports', type=int, required=True, help='Port for each server
(must match --server count)')
    parser.add_argument('--timeout', type=int, default=30, help='Timeout in seconds per server (default: 30)')
    parser.add_argument('command', nargs=argparse.REMAINDER, help='Command to run after server(s) ready')

    args = parser.parse_args()

    # Remove the '--' separator if present
    if args.command and args.command[0] == '--':
        args.command = args.command[1:]

    if not args.command:
        print("Error: No command specified to run")
        sys.exit(1)

    # Parse server configurations
    if len(args.servers) != len(args.ports):
        print("Error: Number of --server and --port arguments must match")
        sys.exit(1)

    servers = []
    for cmd, port in zip(args.servers, args.ports):
        servers.append({'cmd': cmd, 'port': port})

    server_processes = []
```

```
    try:
        # Start all servers
        for i, server in enumerate(servers):
            print(f"Starting server {i+1}/{len(servers)}: {server['cmd']}")

            # Use shell=True to support commands with cd and &&
            process = subprocess.Popen(
                server['cmd'],
                shell=True,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE
            )
            server_processes.append(process)

            # Wait for this server to be ready
            print(f"Waiting for server on port {server['port']}...")
            if not is_server_ready(server['port'], timeout=args.timeout):
                raise RuntimeError(f"Server failed to start on port {server['port']} within {args.timeout}s")

            print(f"Server ready on port {server['port']}")

        print(f"\nAll {len(servers)} server(s) ready")

        # Run the command
        print(f"Running: {' '.join(args.command)}\n")
        result = subprocess.run(args.command)
        sys.exit(result.returncode)

    finally:
        # Clean up all servers
        print(f"\nStopping {len(server_processes)} server(s)...")
        for i, process in enumerate(server_processes):
            try:
                process.terminate()
                process.wait(timeout=5)
            except subprocess.TimeoutExpired:
                process.kill()
                process.wait()
            print(f"Server {i+1} stopped")
        print("All servers stopped")


if __name__ == '__main__':
    main()
```

examples/console_logging.py

```
from playwright.sync_api import sync_playwright

# Example: Capturing console logs during browser automation

url = 'http://localhost:5173'  # Replace with your URL

console_logs = []

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page(viewport={'width': 1920, 'height': 1080})

    # Set up console log capture
    def handle_console_message(msg):
        console_logs.append(f"[{msg.type}] {msg.text}")
        print(f"Console: [{msg.type}] {msg.text}")

    page.on("console", handle_console_message)

    # Navigate to page
    page.goto(url)
    page.wait_for_load_state('networkidle')

    # Interact with the page (triggers console logs)
    page.click('text=Dashboard')
    page.wait_for_timeout(1000)

    browser.close()

# Save console logs to file
with open('/tmp/console.log', 'w') as f:
    f.write('\n'.join(console_logs))

print(f"\nCaptured {len(console_logs)} console messages")
print(f"Logs saved to: /tmp/console.log")
```

examples/element_discovery.py

```
from playwright.sync_api import sync_playwright

# Example: Discovering buttons and other elements on a page

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()

    # Navigate to page and wait for it to fully load
    page.goto('http://localhost:5173')
    page.wait_for_load_state('networkidle')

    # Discover all buttons on the page
    buttons = page.locator('button').all()
    print(f"Found {len(buttons)} buttons:")
    for i, button in enumerate(buttons):
        text = button.inner_text() if button.is_visible() else "[hidden]"
        print(f"  [{i}] {text}")

    # Discover links
    links = page.locator('a[href]').all()
    print(f"\nFound {len(links)} links:")
    for link in links[:5]:  # Show first 5
        text = link.inner_text().strip()
        href = link.get_attribute('href')
        print(f"  - {text} -> {href}")

    # Discover input fields
    inputs = page.locator('input, textarea, select').all()
    print(f"\nFound {len(inputs)} input fields:")
    for input_elem in inputs:
        name = input_elem.get_attribute('name') or input_elem.get_attribute('id') or "[unnamed]"
        input_type = input_elem.get_attribute('type') or 'text'
        print(f"  - {name} ({input_type})")

    # Take screenshot for visual reference
    page.screenshot(path='/tmp/page_discovery.png', full_page=True)
    print("\nScreenshot saved to /tmp/page_discovery.png")

    browser.close()
```

examples/static_html_automation.py

```
from playwright.sync_api import sync_playwright
import os

# Example: Automating interaction with static HTML files using file:// URLs

html_file_path = os.path.abspath('path/to/your/file.html')
file_url = f'file://{html_file_path}'

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page(viewport={'width': 1920, 'height': 1080})

    # Navigate to local HTML file
    page.goto(file_url)

    # Take screenshot
    page.screenshot(path='/tmp/static_page.png', full_page=True)

    # Interact with elements
    page.click('text=Click Me')
    page.fill('#name', 'John Doe')
    page.fill('#email', 'john@example.com')

    # Submit form
    page.click('button[type="submit"]')
    page.wait_for_timeout(500)

    # Take final screenshot
    page.screenshot(path='/tmp/after_submit.png', full_page=True)

    browser.close()

print("Static HTML automation completed!")
```